

机器学习高级话题

强化学习

张寅

zhangyin98@zju.edu.cn

浙江大学计算机学院

前言

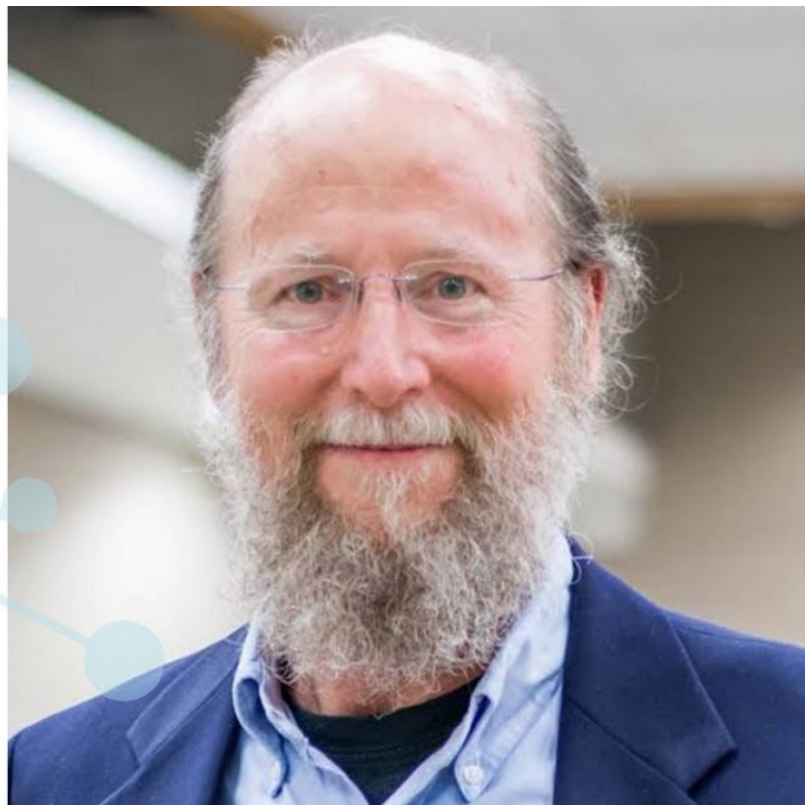


Richard Stuart Sutton 现任阿尔伯塔大学计算机科学教授、阿尔伯塔机器智能研究所研究员兼首席科学顾问

他在现代计算强化学习领域做出多项重要贡献，包括**时序差分学习**和**策略梯度**方法，并获得2024年度图灵奖。

前言

THE BITTER LESSON IN AI

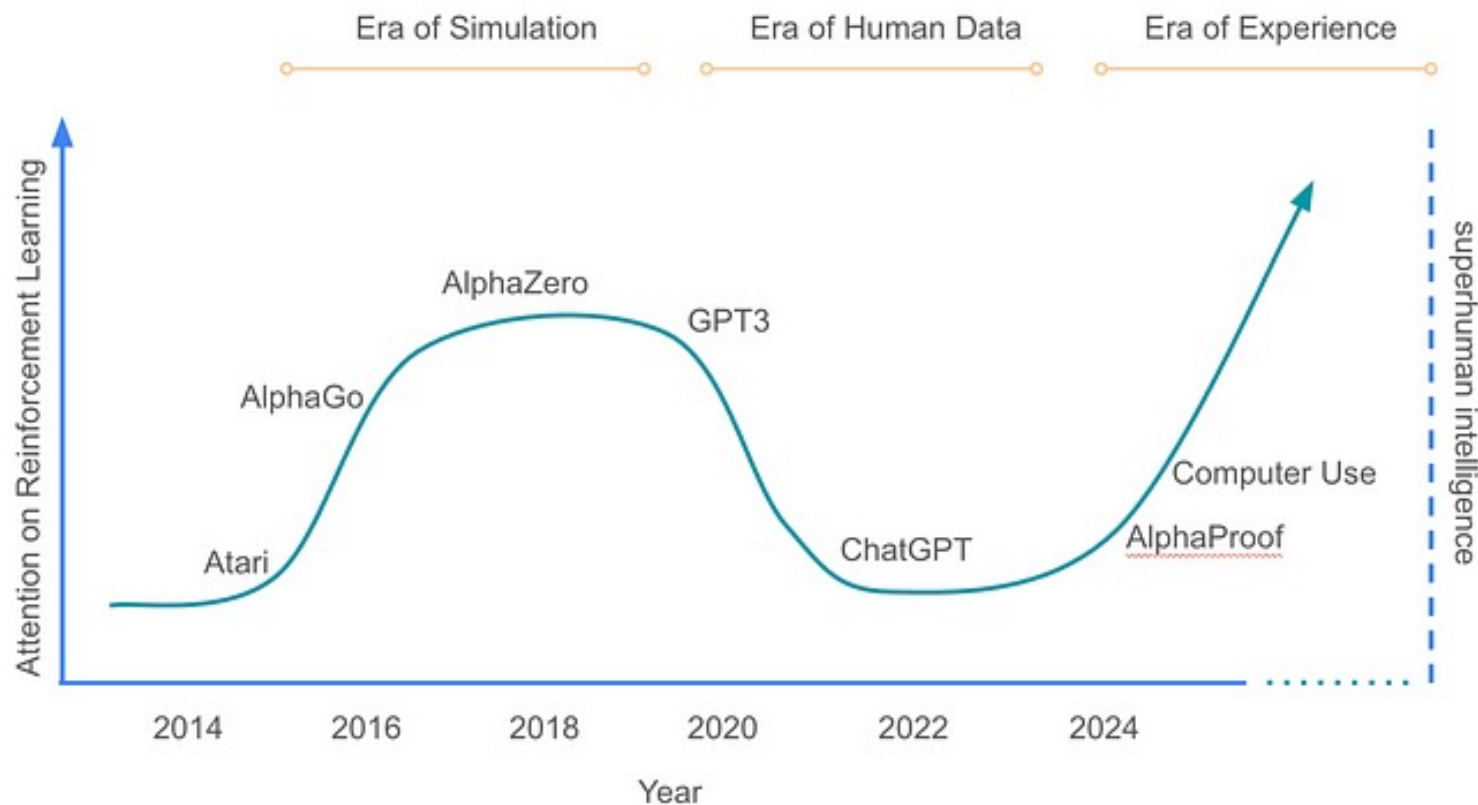


从 70 年人工智能研究中可以汲取的最大教训是：

由于摩尔定律，依赖人类知识的方法往往会以降低计算通用性为代价来增加系统复杂度，只能寻求短期内的显著改进；

长期来看真正重要的只有对计算资源的运用

前言



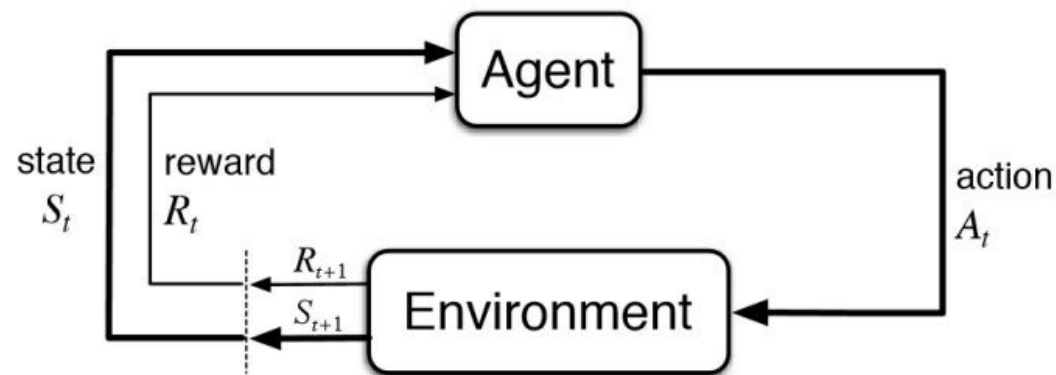
“Welcome to the Era of Experience”
指出：

当前 AI 主要依赖大量人类生成的数据进行训练，这种方法无法让 AI 学到超越人类理解的新见解，尤其是数学、编程和科学等关键领域

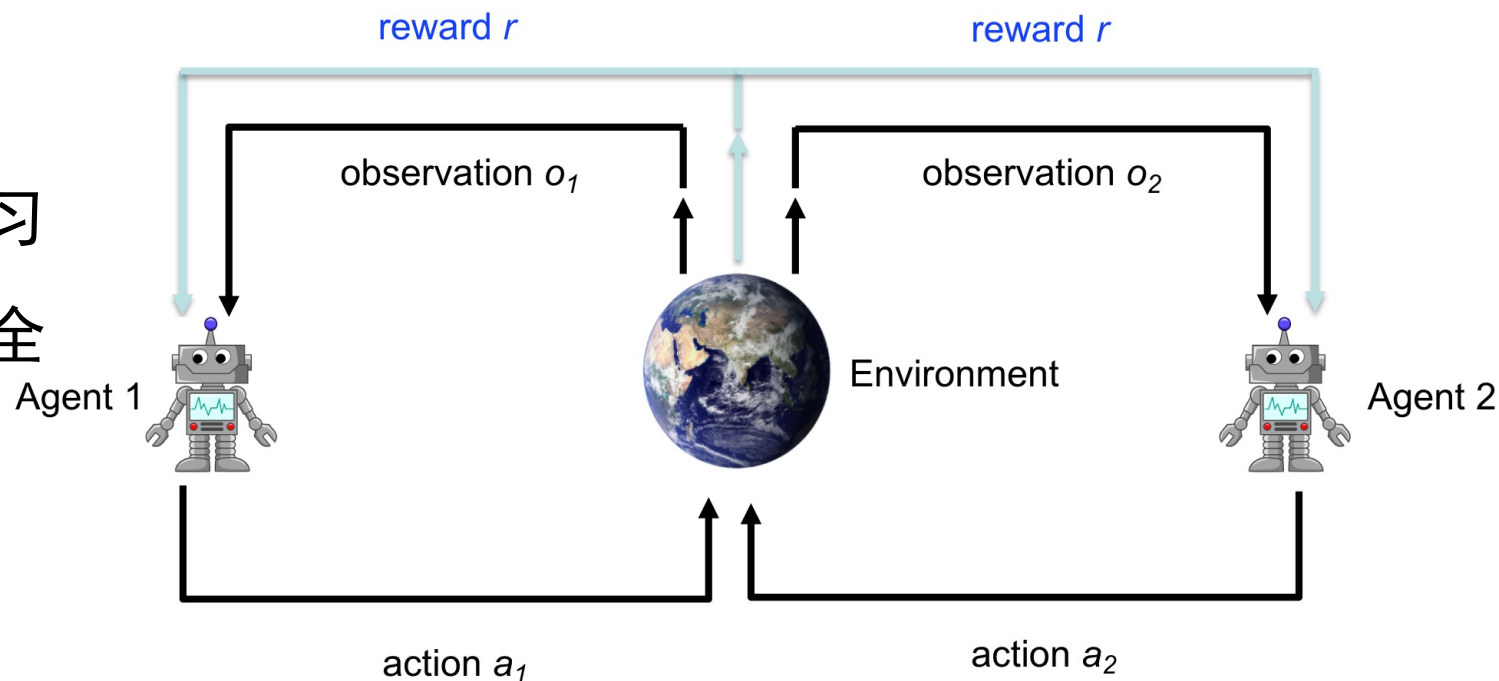
因此需要一种新设想或方法框架，让 AI 智能体通过与环境交互产生的数据（即经验）来学习，实现超越人类的智能

强化学习

智能体通过与环境交互，学习到具体策略，进而最大化累计奖励。



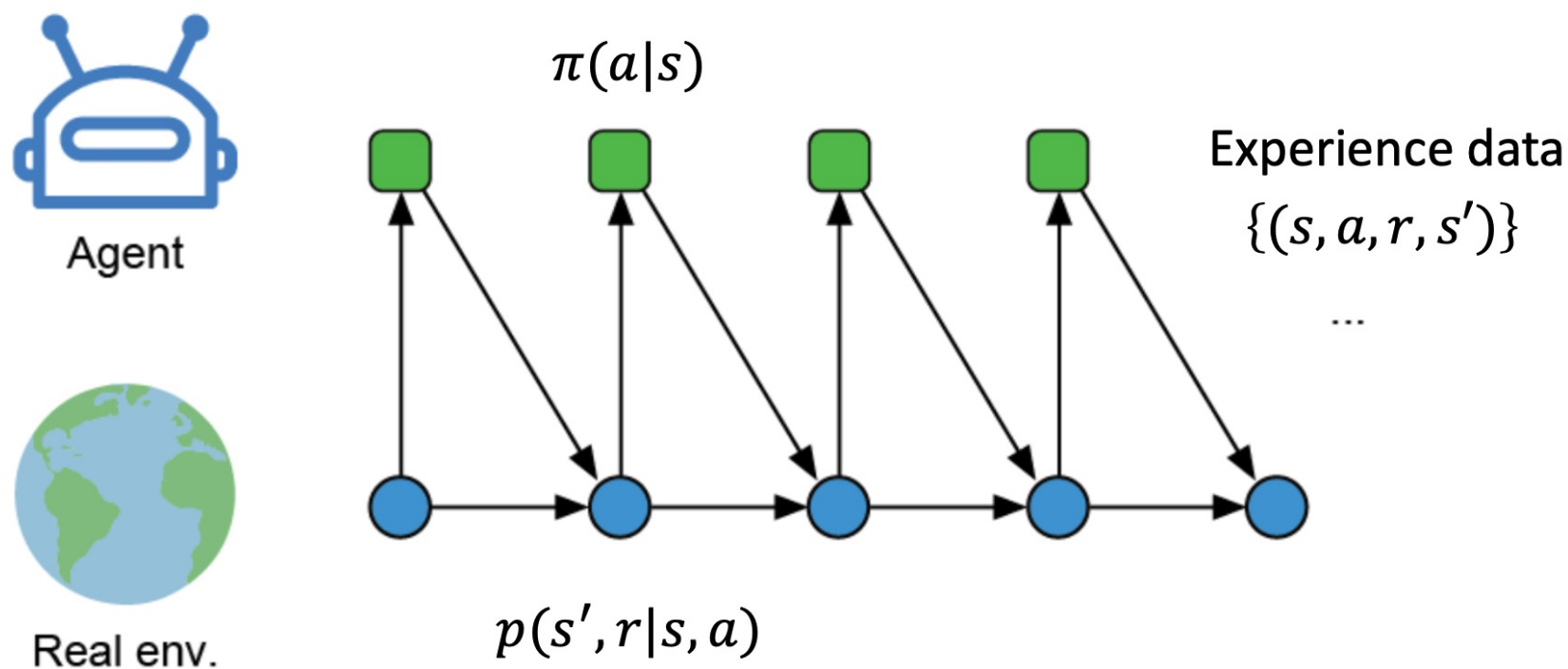
多个智能体通过与环境交互，学习到各自的具体策略，进而最大化全局累计奖励。



基于模型的强化学习

为什么需要基于模型的强化学习？

- 无模型深度强化学习算法的样本效率非常低
- 结合模型学习和强化学习，基于模型训练政策，能提高样本效率



基于模型的强化学习

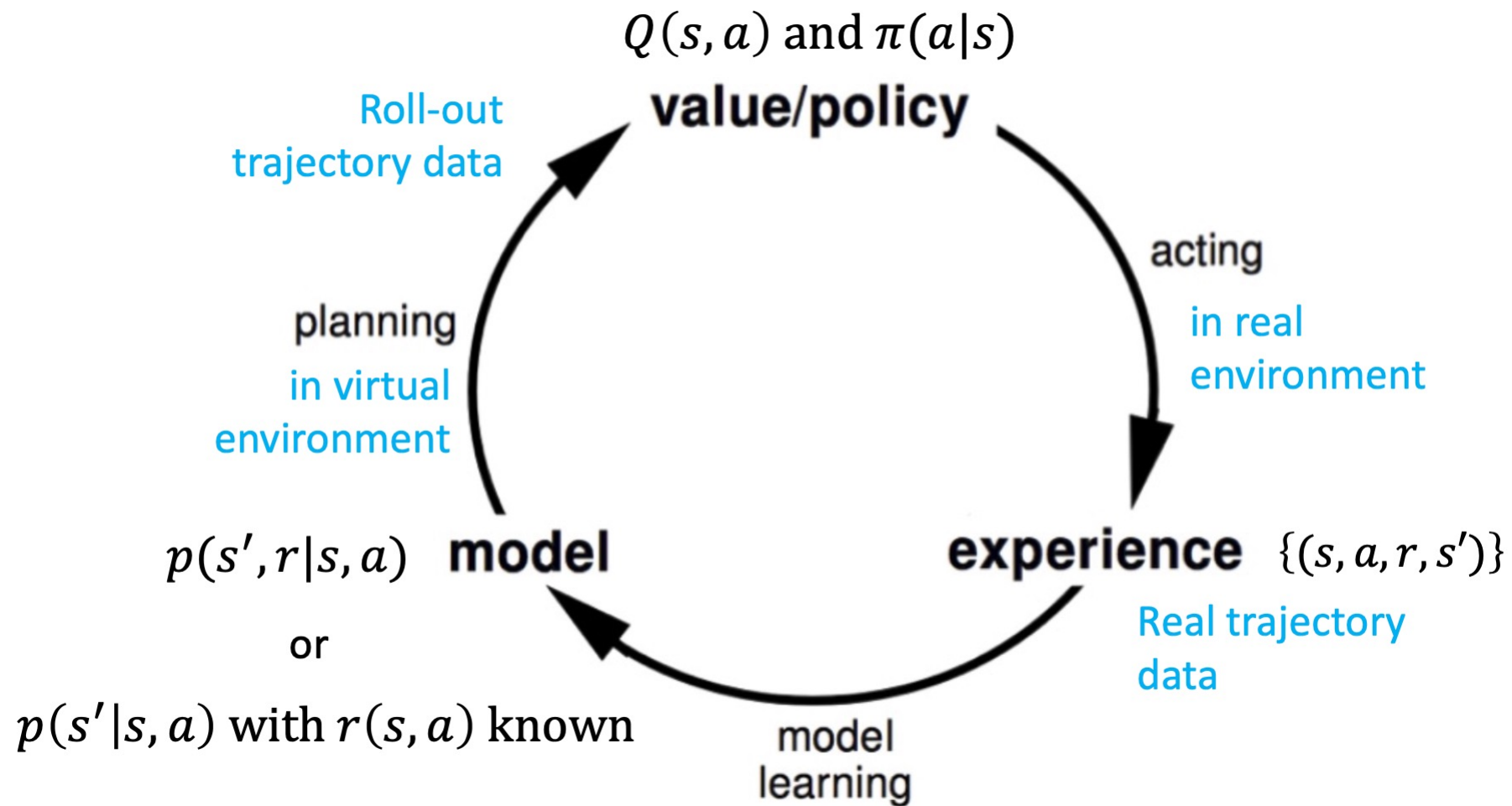
基于模型的强化学习

- 可以直接用on-policy强化学习算法更新策略
- 可以选择不需要更多真实交互数据
- 在实验上始终比无模型强化学习方法具备更高的采样效率
- 但是受限于模型预测的累积误差

无模型强化学习

- 具有最佳渐近性能
- 非常适合具有大数据的深度学习架构
- 但是off-policy方法不稳定性
- 样本效率极低，需要大量的训练数据

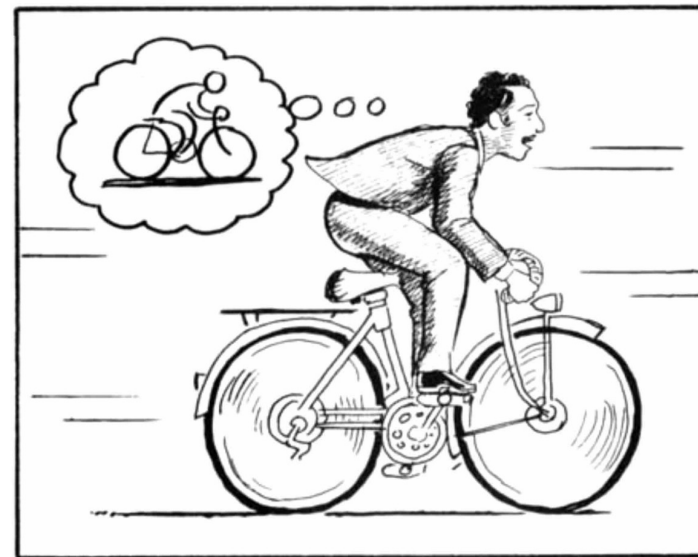
基于模型的强化学习



高性能模型学习

为什么MBRL如此高效？

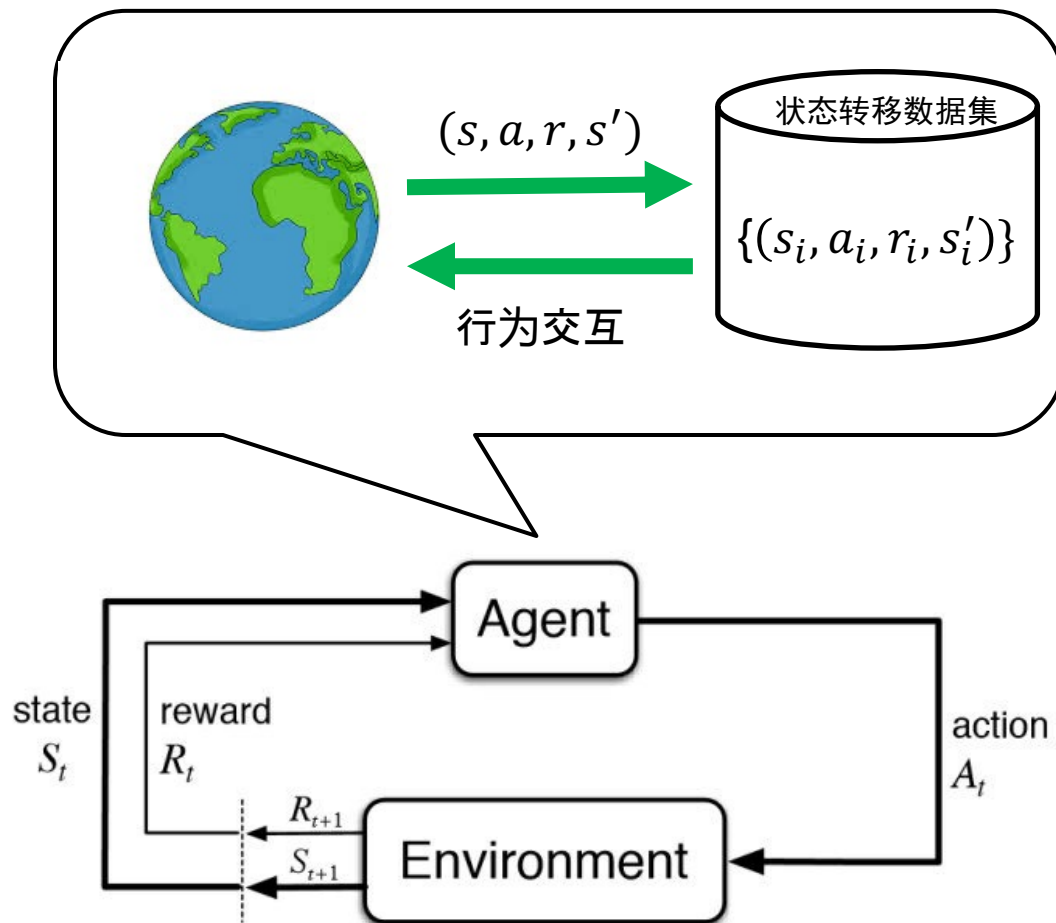
- 强大的世界模型
 - 模型架构: RNN, CNN, Transformer, etc.
 - 模型集成: MoE, Disagreement, Population, etc.
 - 辅助任务: VAE, SPR, Prototype, Simsam, etc.
- 规划&搜索机制
- 多步价值估计 (Multi-step Value Estimation)
 - 资格迹: VTrace, ReTrace, GAE, etc.
 - Multi-step Return



从经验中学习

经验回放缓存：我当初是怎样做的决策？现在应该怎样做决策？

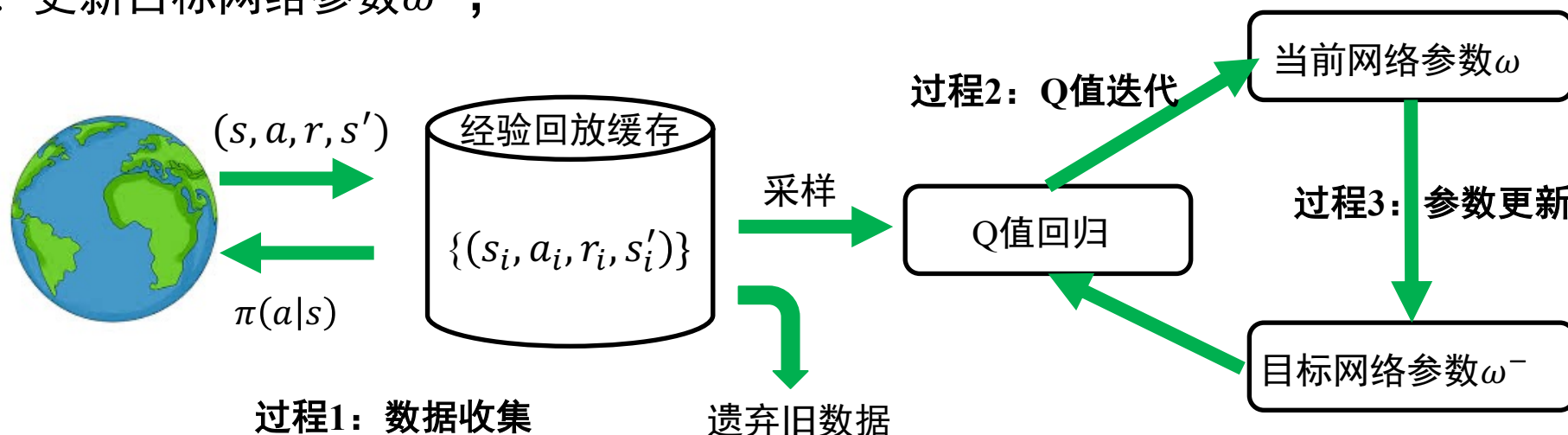
- 智能体与环境交互得到的历史经验存储在经验回放缓存中；
- 智能体从经验缓存中采样数据，用来训练当前的策略；
- 历史交互数据在训练模型时能否发挥更高效的作用？



深度Q学习

经典深度Q学习：

1. 基于当前策略采取行为 a_i ，观测 (s_i, a_i, r_i, s'_i) 并加入到缓存 B ；
2. 均匀从缓存 B 中采样得到小组数据 $\{(s_j, a_j, r_j, s'_j)\}$ ；
3. 利用目标 Q_{ω^-} -网络计算 $y_j \leftarrow r_j + \gamma \max_{a'_j} Q_{\omega^-}(s'_j, a'_j)$ ；
4. $\omega \leftarrow \omega - \alpha \frac{dQ_{\omega}}{d\omega}(s_j, a_j)(Q_{\omega}(s_j, a_j) - y_j)$ ；
5. 更新目标网络参数 ω^- ；

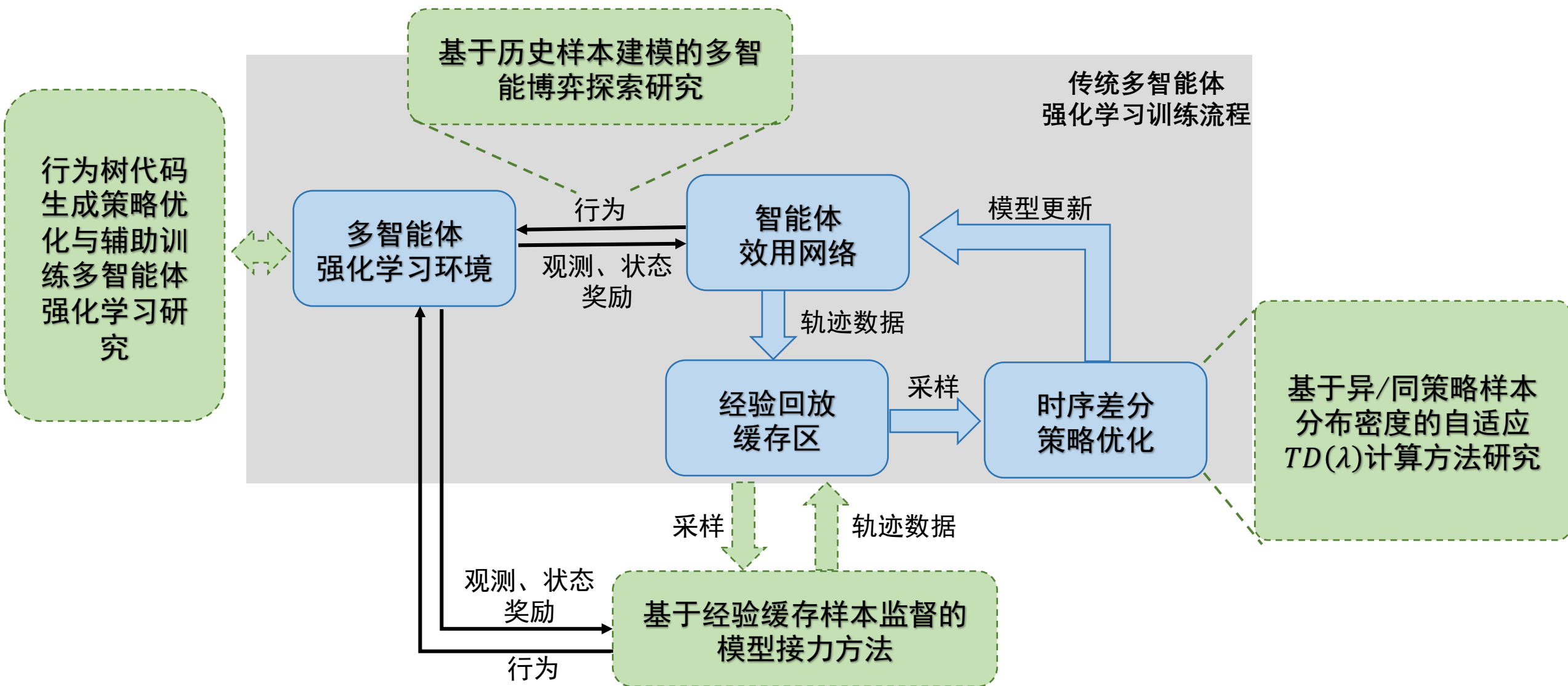


历史数据

拿到历史数据后可以：

1. 用历史轨迹样本数据训练一个模型
 - 基于经验缓存样本监督的模型接力方法
2. 用历史轨迹样本引导智能体的探索行为
 - 基于历史样本建模的多智能博弈探索研究
3. 将历史轨迹样本通过重要性采样的方式映射到当前策略样本概率
 - 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 大模型作为外部人类知识样本的集大成，可以作为专家辅助强化学习训练
 - 行为树代码生成策略优化与辅助训练多智能体强化学习研究

组织结构



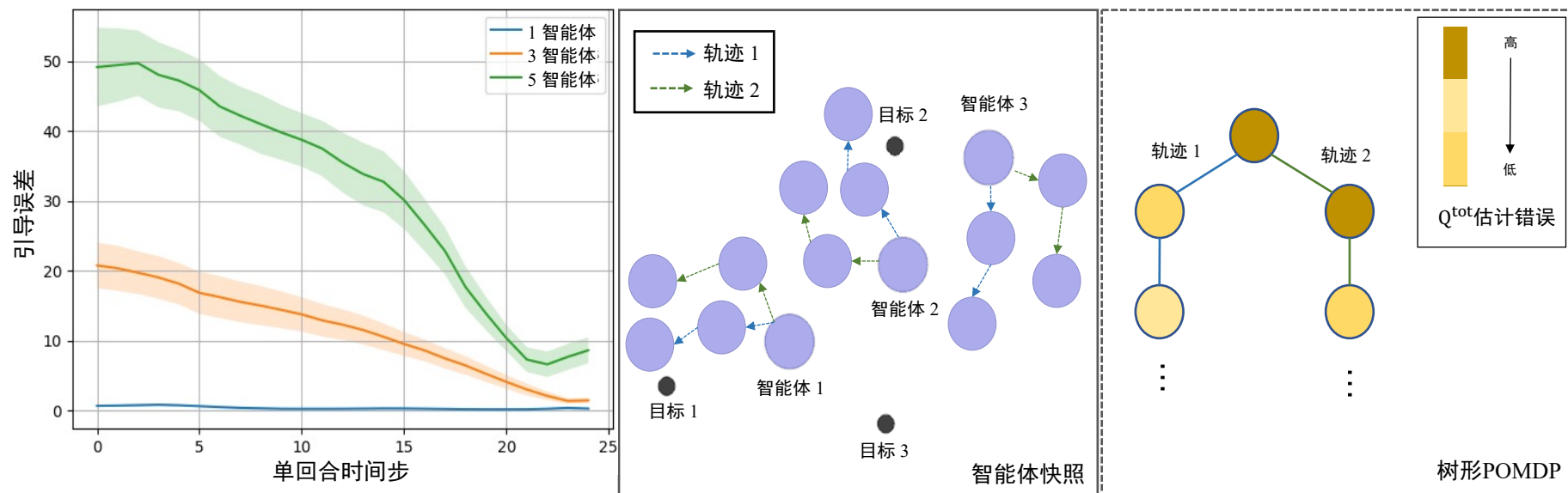
主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

基于经验缓存样本监督的模型接力方法：自举误差

自举误差：智能体输出的预期状态-行为价值，与环境真实采样得到的累计奖励的差。

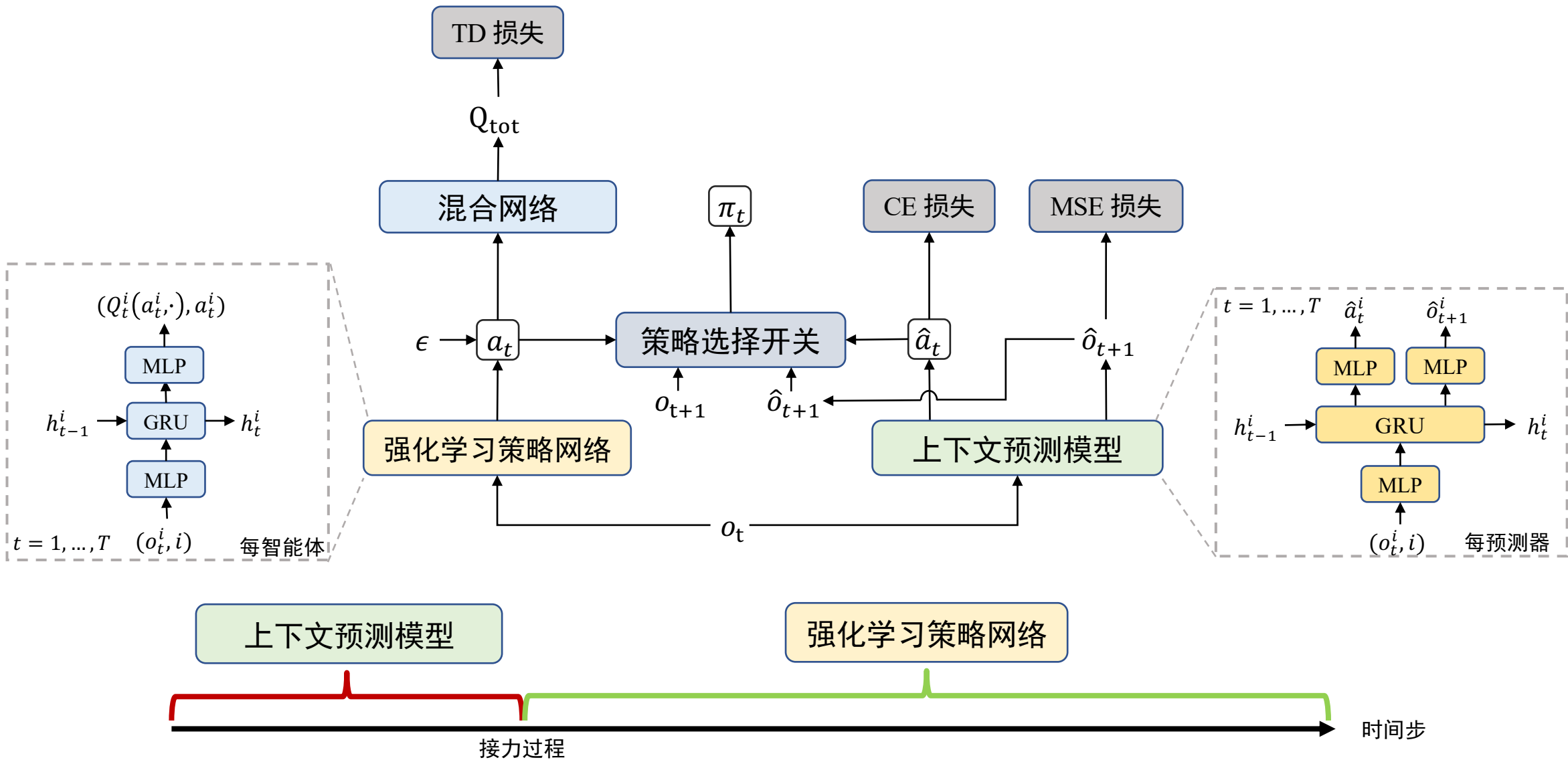
- 自举误差随MDP树向根节点回传；
- 智能体数量增大，自举误差显著增大；
- 给定状态和行为，直接学习状态到行为的有监督映射。



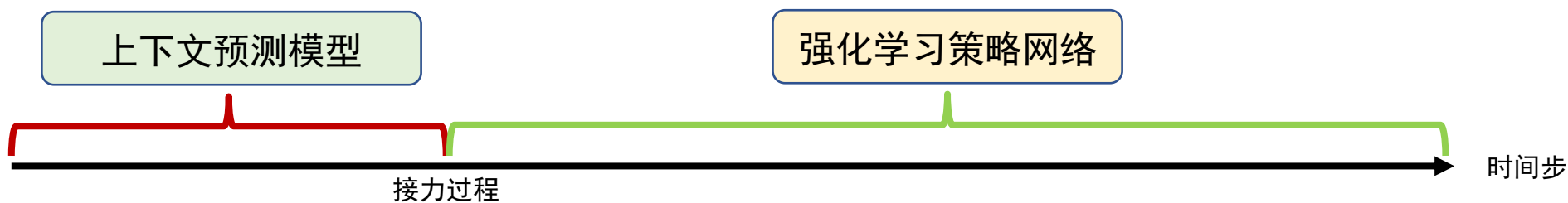
基于经验缓存样本监督的模型接力方法：自举误差

- 在多智能体强化学习的初期阶段，由于Q值学习随机性比较高且没有收敛，因此每个决策步骤中的自举误差很大。同时自举误差也随着树形MDP向根节点回传，导致早期决策不可信。
- 既然早期决策不可信，论文提出了先监督学习后强化学习的接力决策架构。通过将经验缓存中的轨迹建模成蒙特卡洛轨迹树的形式，通过UCB计算采样出最优轨迹，并以行为克隆的模式学习轨迹进行训练。在决策时先用监督学习策略，再用基线多智能强化学习算法接力训练。

基于经验缓存样本监督的模型接力方法：模型架构



基于经验缓存样本监督的模型接力方法：模型接力



- 在轨迹模拟中，通过上下文预测模型预测的下时刻的观测是否准确，来控制是否应该由强化学习模型接力：

$$\pi_{,t} = (\mathbb{1}_t)\pi_{,t}^{Context} + (1 - \mathbb{1}_t)\pi_{,t}^{Util}$$

$$\mathbb{1}_t = \begin{cases} 1, & |\hat{o}_t - o_t| \leq d_t \text{ and } \mathbb{1}_{t-1} = 1 \\ 0, & \text{others} \end{cases}$$

上下文预测器

上下文预测器一方面要完成智能体行为的选择，一方面要完成下一刻观测的预测

- 如果给定轨迹 $\{(s_t, a_t, o_t, s_{t+1}, a_{t+1}, o_{t+1}, \dots)\}$ ，训练一个上下文预测模型，该模型以 o_t 为输入，以 a_t 和 o_{t+1} 为输出，其中 a_t 的训练是分类任务 $\Phi_{act}(\hat{a}_t|o_t, \omega)$ ，采用交叉熵损失函数， o_{t+1} 的训练任务为回归任务 $\Phi_{obs}(\hat{o}_{t+1}|o_t, \omega)$ ，采用均方损失函数，即：

$$L_{context}(\omega) = \sum_{i=0}^b (CE(a_t, \Phi_{act}(\hat{a}_t|o_t, \omega)) + MSE(o_{t+1}, \Phi_{obs}(\hat{o}_{t+1}|o_t, \omega)))$$

- 上下文预测模型可以通过观测量预测最优行为，同时回归预测采取该行为后的下一时刻的观测量。如果预测下一时刻观测与推演后真实观测相似，则继续采用上下文预测模型决策；如果不相似，则后续任务由强化学习模型接手预测。

观测聚类

- 由于状态和观测是连续值，直接建立树的节点是不现实的，同时上下文预测值与真实值比对时几乎无法保证完全相同，因此观测值需要被映射到具备一定误差阈值的原型。从观测到原型的映射可以采用EM算法完成聚类：

$$\theta^{(it+1)} = \arg \max_{\theta} \sum_{z=1}^{N_{clu}} \log P(s, z | \theta) \times P(z | s, \theta^{(it)})$$

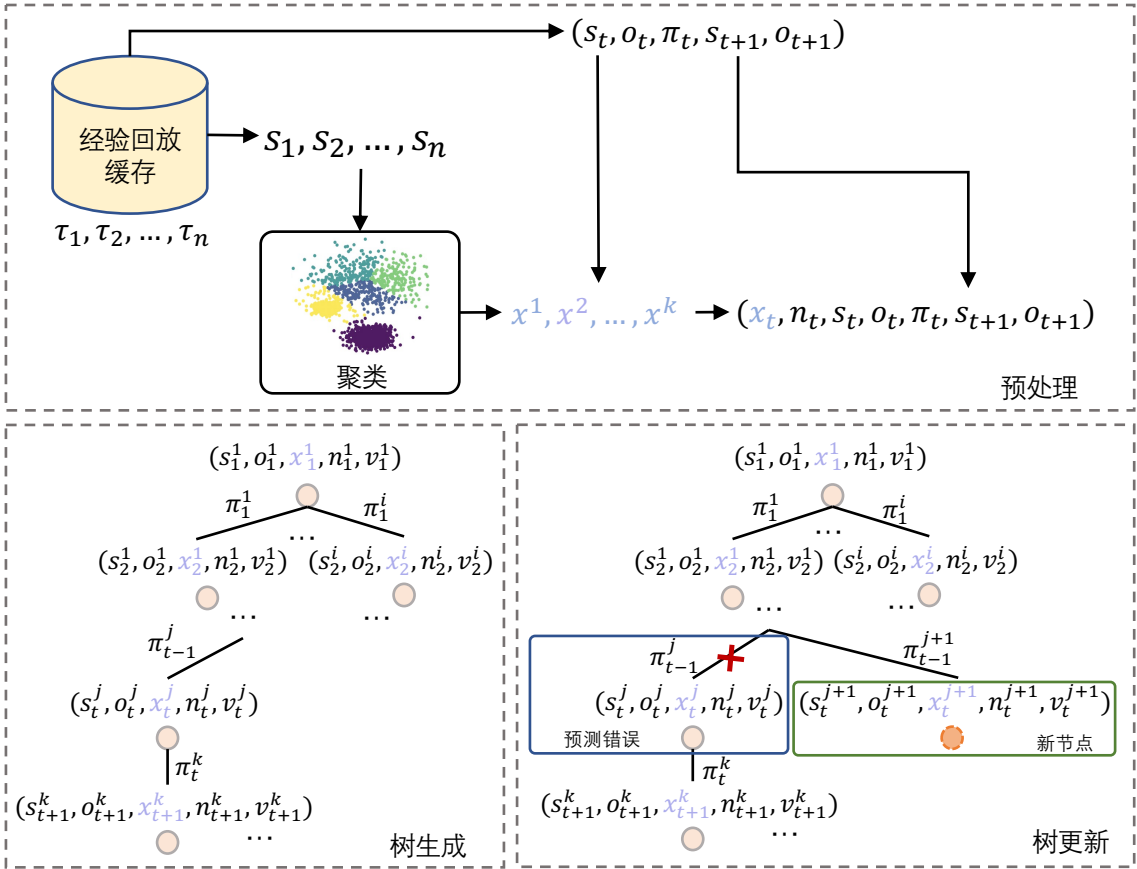
$$cs_i^{(it+1)} = \frac{1}{|C_i^{(it)}|} \sum_{s_j \in C_i^{(it)}} s_j$$

$$cs_i^{(it+1)} = \frac{1}{n+1} (cs_i^{(it)} \times n), \quad n = |C_i^{(it)}|$$

$$s \in \arg \min_{cs_i \in C_i} dist(s, cs_i), \quad i = 1, 2, \dots, n$$

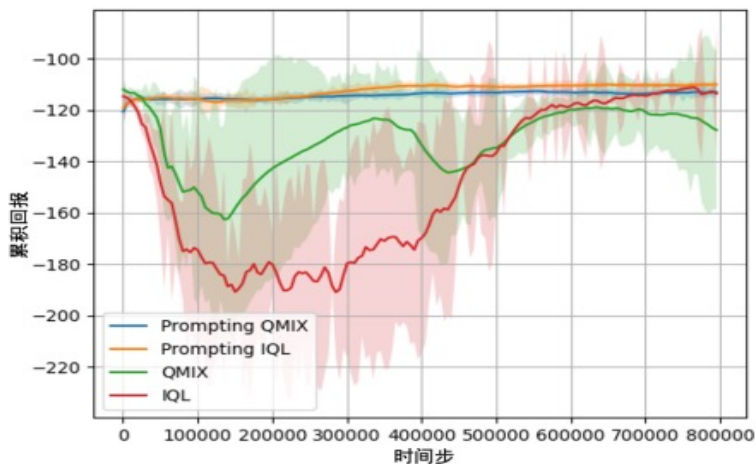
蒙特卡洛轨迹树生成

- 将经验缓存中的轨迹数据进行聚类形成原型；
- 将轨迹数据转换为原型数据；
- 将一定量的原型轨迹建模为蒙特卡洛轨迹树；
- 从轨迹树中从根节点到叶子节点进行选择，
选出当前最优原型轨迹路径。

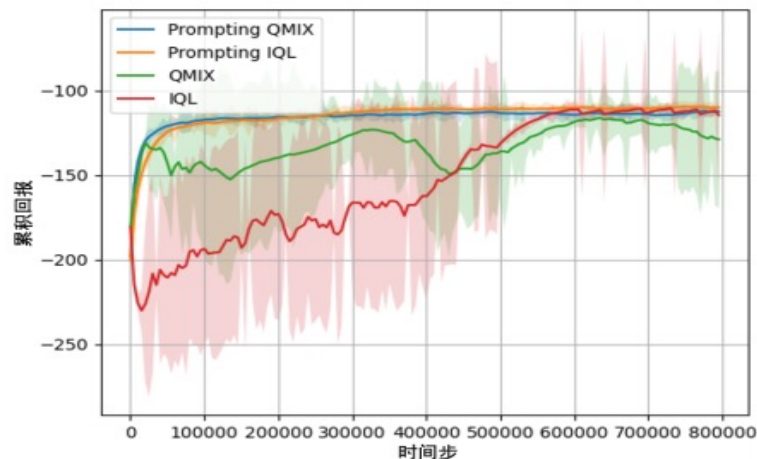


多智能体粒子世界环境模拟

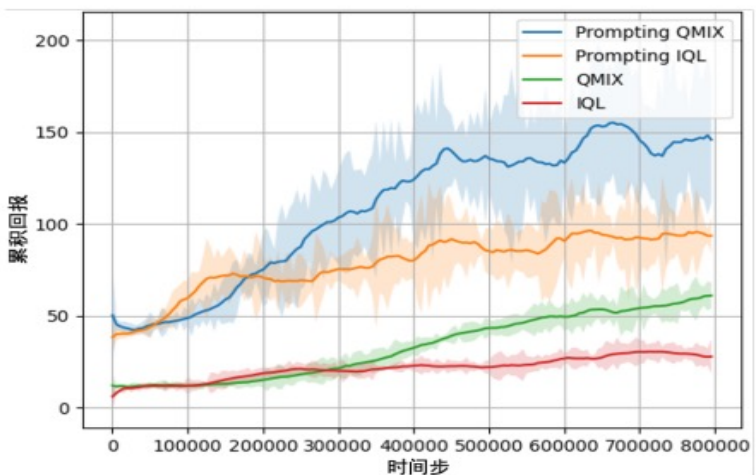
算法收敛速度和表现有较大的提升



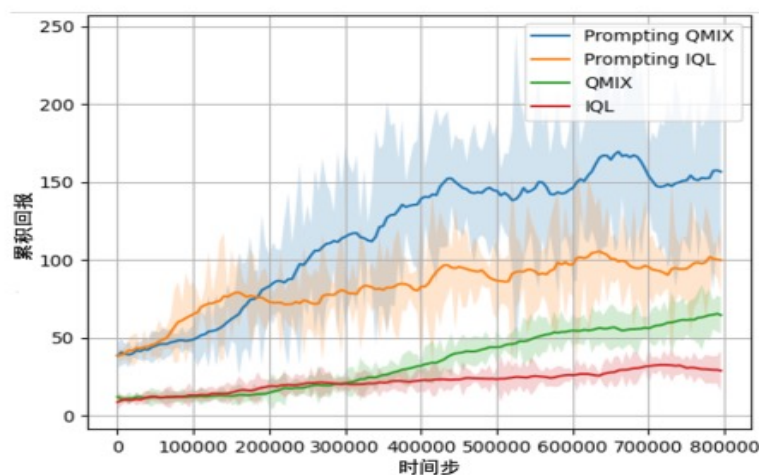
(a) Spread 场景训练曲线



(b) Spread 场景测试曲线



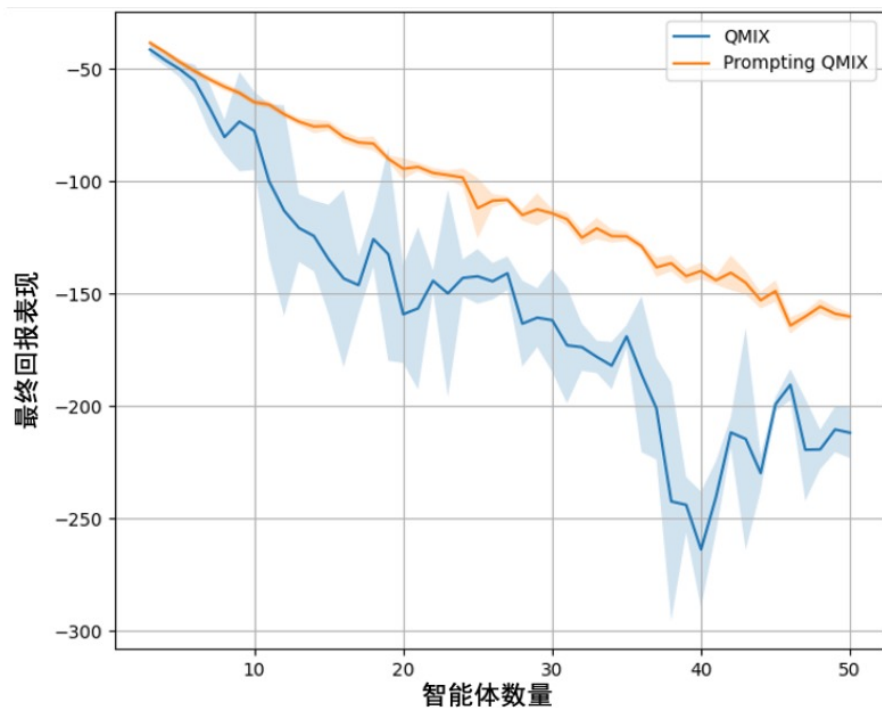
(c) Tag 场景训练曲线



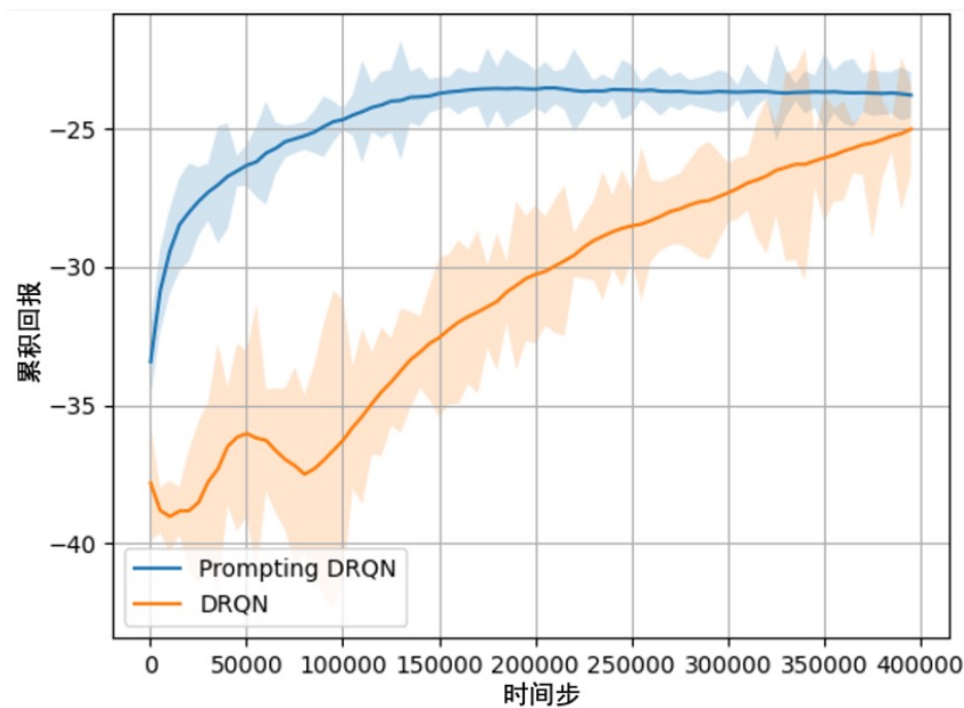
(d) Tag 场景测试曲线

多智能体粒子世界环境模拟

本研究对极大数量智能体环境的支持，以及对单智能体强化学习的兼容。



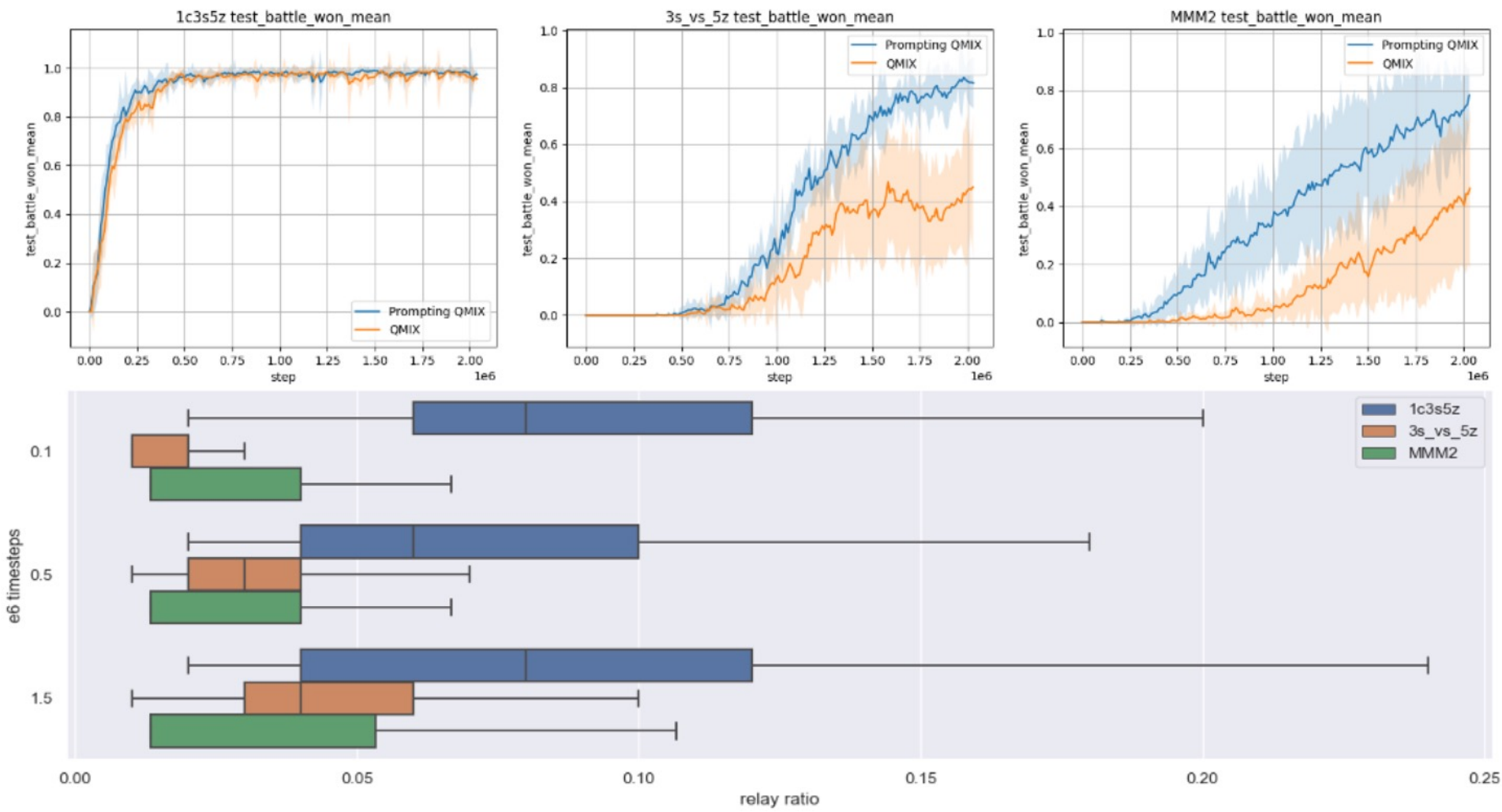
(a) 智能体数量与最终回报



(b) 单智能体 DRQN 测试曲线

SMAC环境模拟

随着训练的进行，接力比率增大，即上下文预测器决策更多的时间步



主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

基于历史样本建模的多智能博弈探索研究

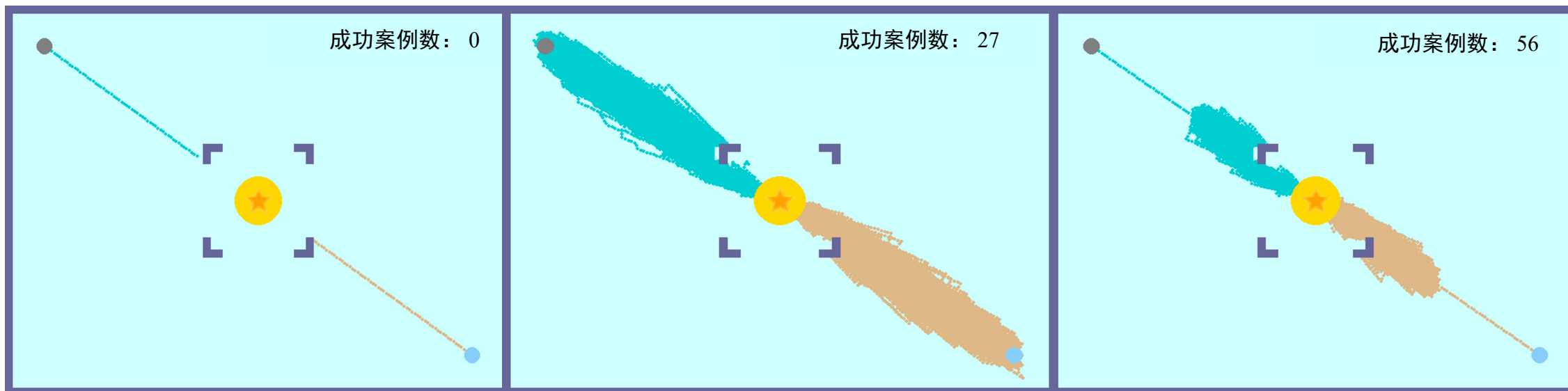
- ϵ -greedy探索方法可以被视为颤抖手决策问题：
 - ϵ -greedy以 p 为概率进行探索，随机选择一个行为，以 $1 - p$ 为概率进行利用，选择价值最大的行为；
 - 颤抖手决策问题，即以较大概率完成最大价值决策，但是以小概率的可能性出现手滑，选错其他的选项；

		player 1	
		L	R
player 2	T	(1 , 1)	(1.9 , 0)
	B	(0 , 1.9)	(2 , 2)

- 假定 $\epsilon = 0.2$ ，颤抖手均衡解为 (T, L) 然而最优解是 (B, R)

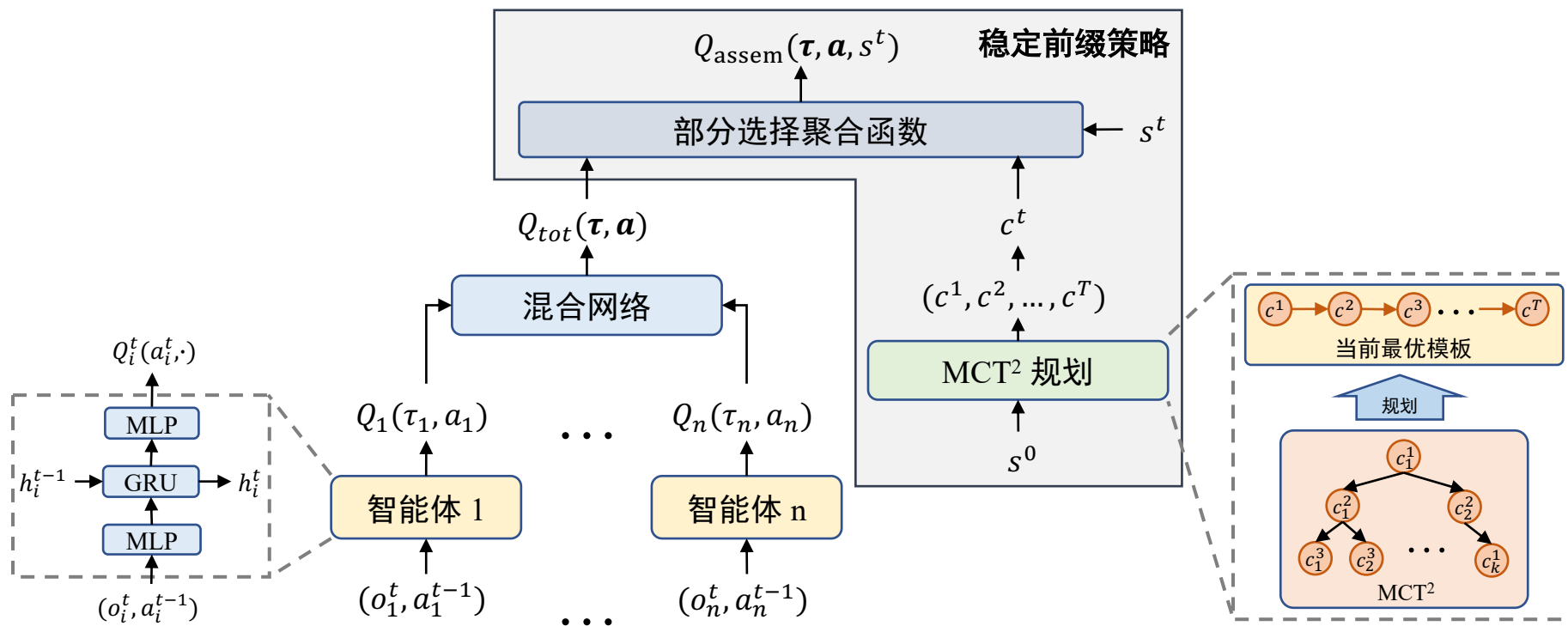
基于历史样本建模的多智能博弈探索研究

- 给定的策略是两个智能体朝向终点运动，终点附近有墙阻挡；
 - 如果不进行探索，则智能体被墙挡住，智能体陷入局部最优；
 - 如果在全路径都进行探索，智能体可能会进行很多不必要的探索过程；
 - 如果智能体先按一定的策略决策一段时间后，再开始探索，则可以节省很多的探索预算，有更大的几率探索更重要的部分。



算法整体架构

- QMIX方法作为baseline;
- 蒙特卡洛轨迹树建模, 规划出当前最优模版;
- 智能体符合模版路线时不探索, 跳出模版时开始探索。



蒙特卡洛轨迹树

- 将历史原型转换更新到蒙特卡洛轨迹树中后，根据当前的轨迹树进行UCB计算采样到一条从根节点到叶子节点的轨迹，并将该轨迹作为模板轨迹：

$$c = \arg \max_{cn \in \text{children}(n)} (v(cn) + c_{ucb} \cdot \pi(cn|n) \cdot \sqrt{(\frac{\log \Gamma_{cn}}{1 + \Gamma_n}))})$$

- 每rollout得到一条轨迹后，都将该轨迹转化成原型轨迹，并将该轨迹集成到MCTT中：

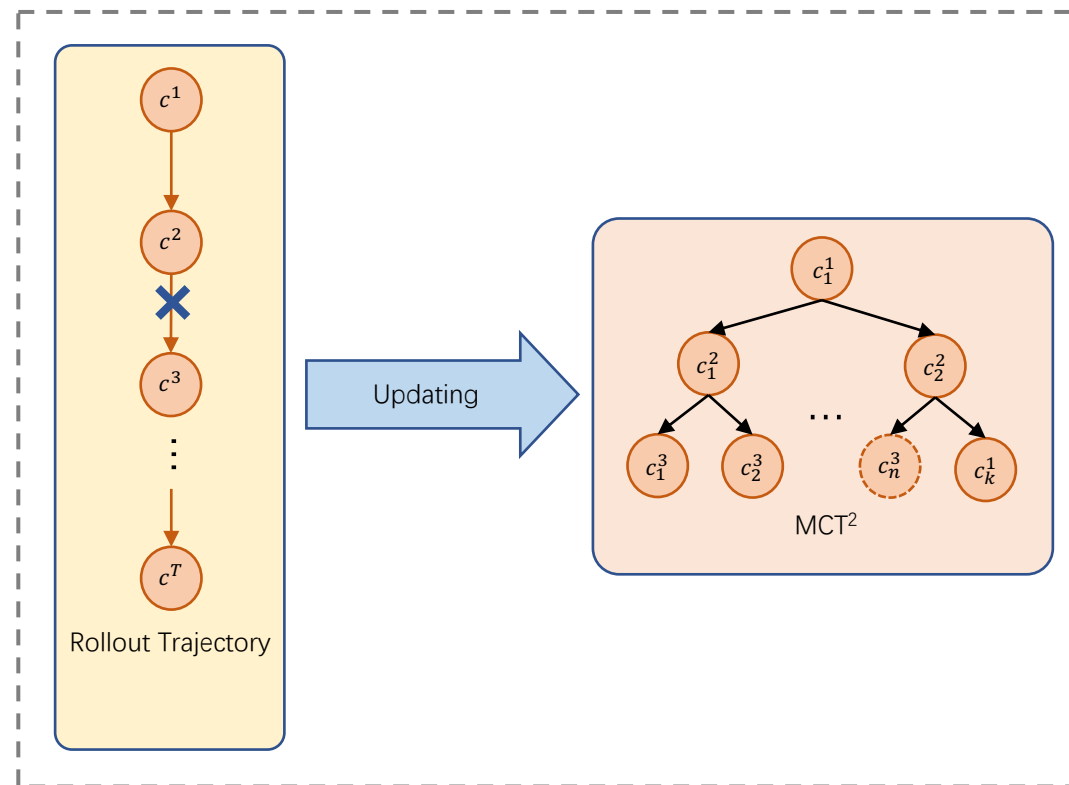
$$v(n) = \lambda v(n) + (1 - \lambda) \times \sum_{cn \in \text{children}(n)} \pi(cn|n) \times (R_{n \rightarrow cn} + \gamma v(cn))$$

- 为解决策略偏移的问题，每隔一个周期都摧毁轨迹树，并根据最新经验回放缓存中的数据重新构建MCTT，在扩展根节点时加入狄利克雷噪声：

$$\pi(cn|n) := (1 - \rho)\pi(cn|n) + \rho \mathcal{N}_{\mathcal{D}}(\xi)$$

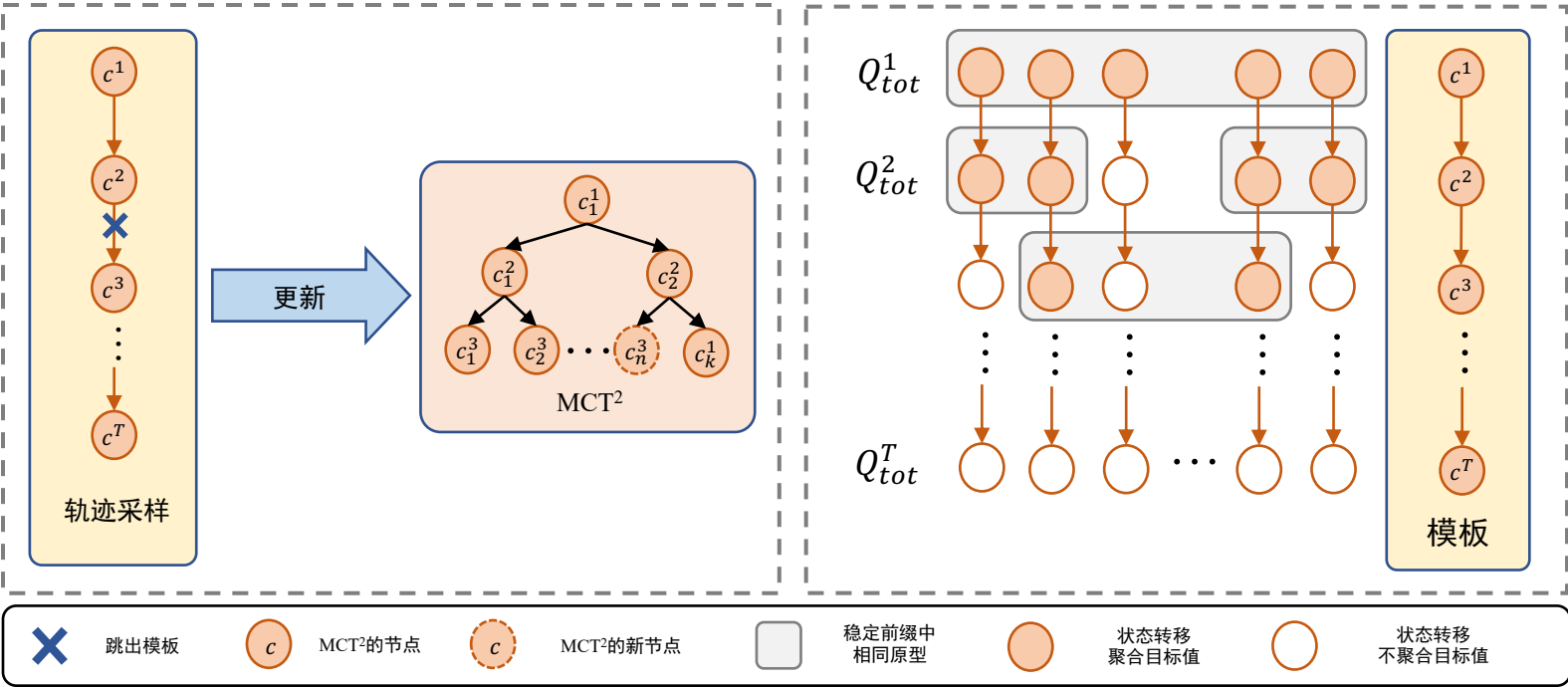
蒙特卡洛轨迹树

- 使用EM算法，将状态 s 聚类成为原型（聚类中心）；
- 将replay buffer中的数据转换为原型转移数据；
- 根据原型，构建蒙特卡洛轨迹树，其枝干仅以原型-原型转换为扩展，而不是以行为进行扩展；
- 即智能体不论采用什么策略，只要其进入到的下一刻的状态是正确即判断其依旧在模板中；



智能体策略训练

- 训练过程中从经验回放缓存中采取一定数量的轨迹数据，取每条轨迹的初始状态并转换成原型；
- 从MCTT中根节点开始规划，为每条轨迹规划出当前最优的模板路径；
- 将真实轨迹转换为真实原型轨迹，将真实原型轨迹与当前最优模板路径相对比，找到每条轨迹中不符合模板的部分；
- 将符合模板部分的 Q_{tot} 相聚合；



智能体策略训练

- 将符合模板部分的 Q_{tot} 相聚合；

$$Q_{assem}^t(s^t) = \frac{(\sum_{i=1}^{bs} Q_{tot}(\tau_i, a_i) \cdot \mathbb{1}(c^t = \phi(s^t)))}{\sum_{i=1}^{bs} \mathbb{1}(c^t = \phi(s^t))}$$

- 计算TD target值的时候根据是否符合模板选用聚合的 Q_{assem} 或者原本的 Q_{tot} ：

$$y^t = r^t + \gamma[\mathbb{1}(c^{t+1} = \phi(s^{t+1})) \cdot Q_{assem}^{t+1}(s^{t+1}) + (1 - \mathbb{1}(c^{t+1} = \phi(s^{t+1}))) \cdot Q_{tot}(\tau, a^{t+1})]$$

- 计算TD error后采用MSE loss进行训练。

算法表现能力

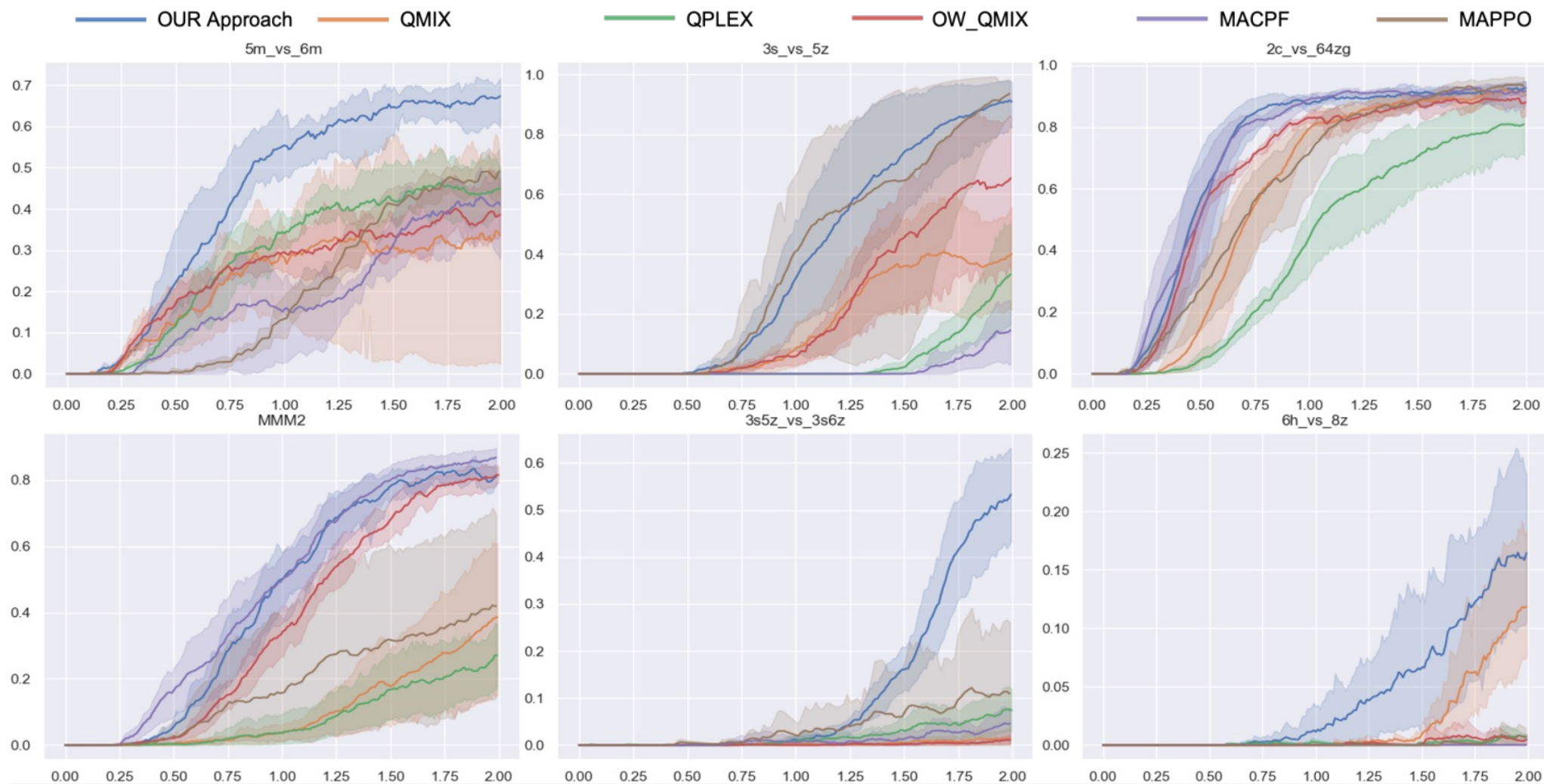
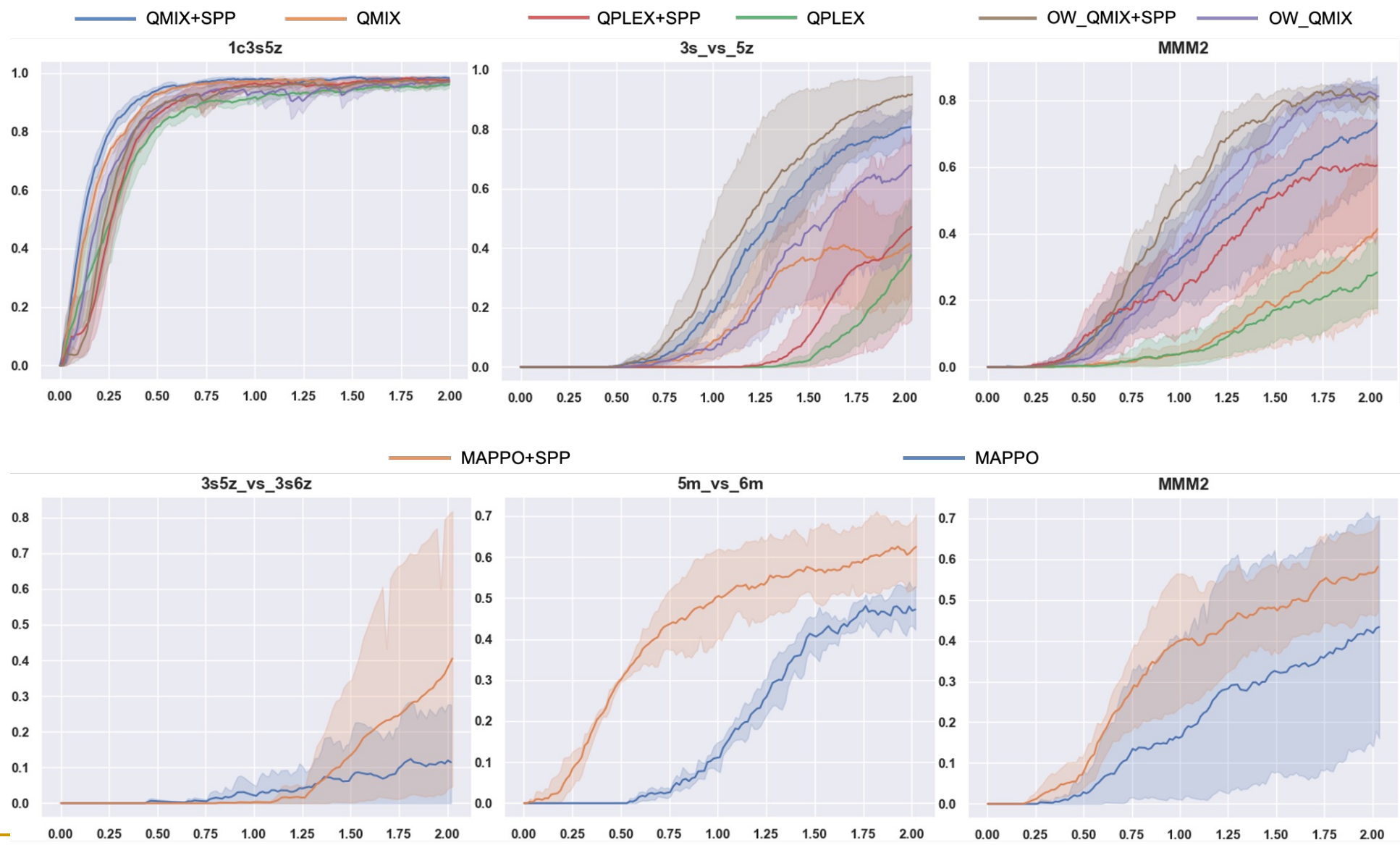


图 4.5 稳定前缀策略框架在 SMAC 场景的主要 9 个任务中的表现

算法适配能力

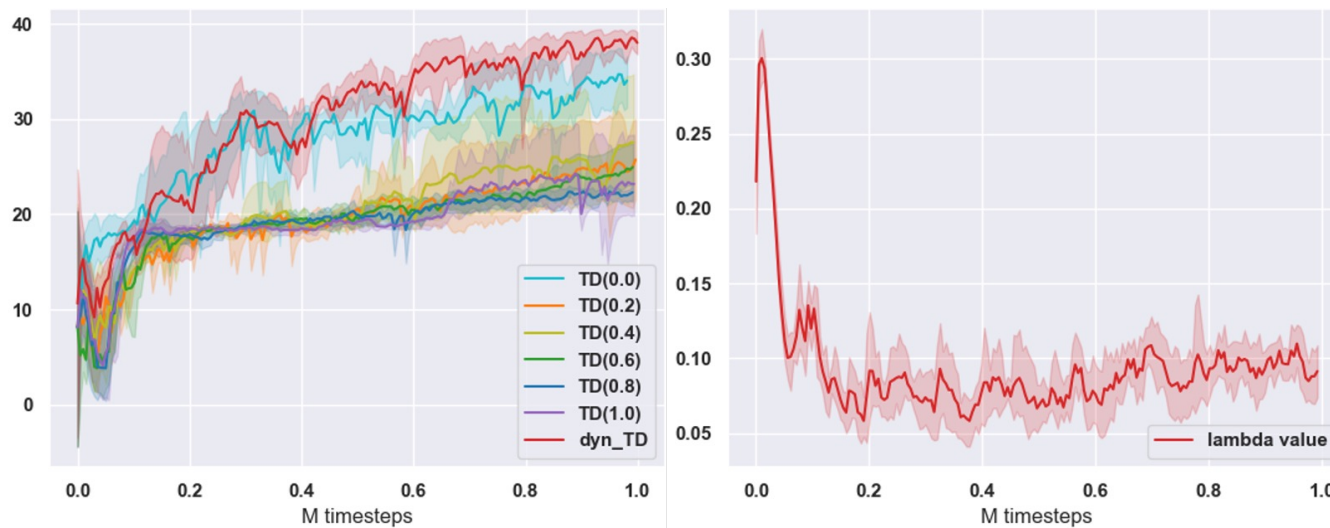
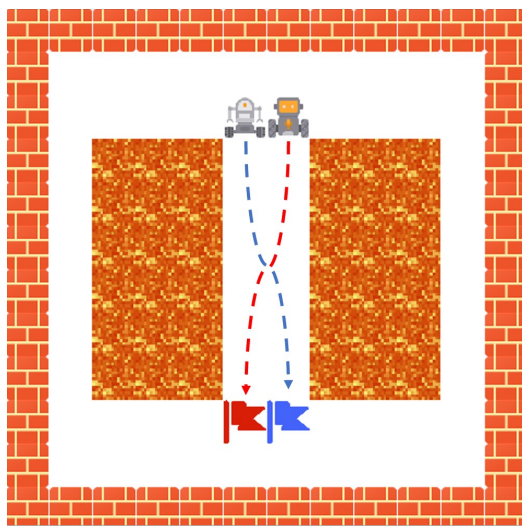


主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

基于异/同策略样本分布密度的自适应TD(λ)计算方法研究

- 计算target 的时候应该采用蒙特卡洛采样得到的累积回报，还是采用从已存在的Q函数中自举得到新的价值？
- TD(λ)的方式平衡TD更新与MC采样？
- 传统的TD(λ)方法中的 λ 被设定为一个不变的定值超参，且该超参的值选定会极大影响算法的收敛表现。



自适应TD(λ)计算

- SMIX(λ)算法使用离策略进行训练，通过避免贪婪假设来实现稳定的集中价值函数，并将SMIX(λ)的概念与 $Q(\lambda)$ 连接起来；
- ETD(λ)通过对TD(λ)更新进行适当加权，确保了线性情况下的收敛性；
- RIIT论文讨论了QMIX算法的单调性和实现技巧，并在训练过程之前提供了 λ 值建议；
- 重要性采样是校正行为策略与目标策略之间差异的最简单方法，但存在较大的方差。 $Q^*(\lambda)$ 引入了一种基于 Q 基线的偏离策略校正，它避免了方差膨胀但不能保证任意行为策略与目标策略的收敛性；
- **ATD (λ)**方法引入了一种自适应 λ 值计算方法，即基于采样的状态转移出现在当前策略的可能性，并在训练过程中确定自适应计算 λ 值，而不是在训练过程之前预设一个超参数。

离策略强化学习算法不稳定原因

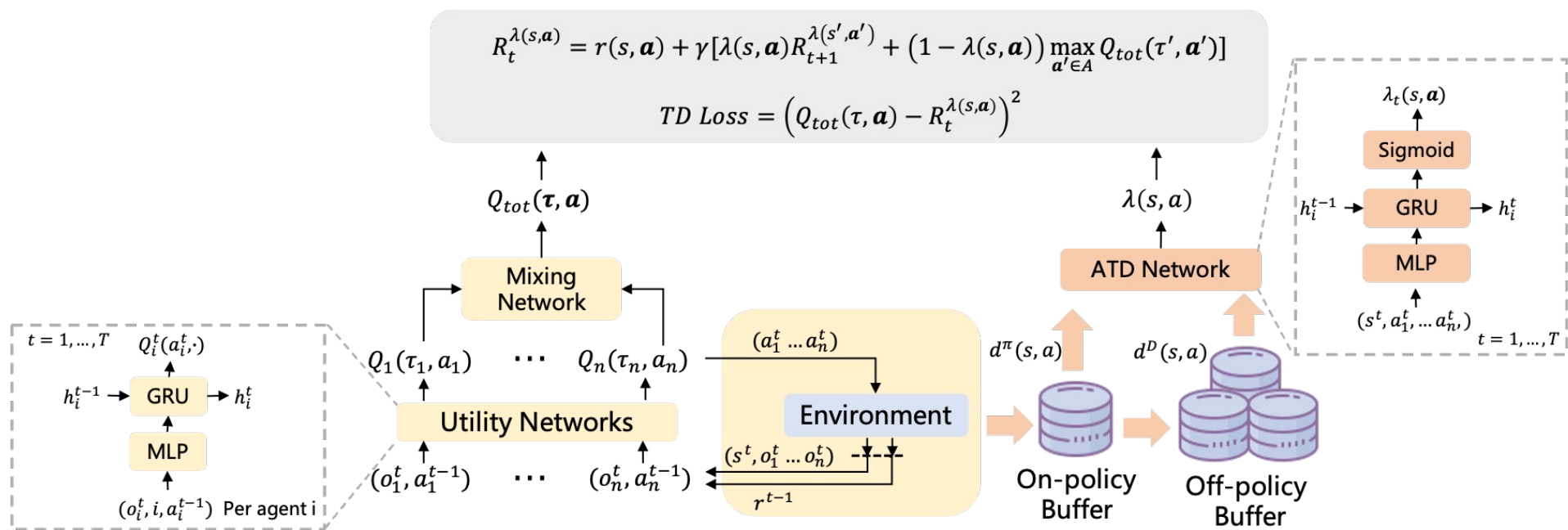
- 行为 μ 策略（旧）与当前 π 策略（新）不同带来的差异。

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\doteq \boldsymbol{\theta}_t + \rho_t \alpha \left(R_{t+1} + \gamma \boldsymbol{\theta}_t^\top \boldsymbol{\phi}_{t+1} - \boldsymbol{\theta}_t^\top \boldsymbol{\phi}_t \right) \boldsymbol{\phi}_t \\ &= \boldsymbol{\theta}_t + \alpha \left(\underbrace{\rho_t R_{t+1} \boldsymbol{\phi}_t}_{\mathbf{b}_t} - \underbrace{\rho_t \boldsymbol{\phi}_t (\boldsymbol{\phi}_t - \gamma \boldsymbol{\phi}_{t+1})^\top}_{\mathbf{A}_t} \boldsymbol{\theta}_t \right),\end{aligned}$$

$$\begin{aligned}\mathbf{A} &= \lim_{t \rightarrow \infty} \mathbb{E}[\mathbf{A}_t] = \lim_{t \rightarrow \infty} \mathbb{E}_\mu \left[\rho_t \boldsymbol{\phi}_t (\boldsymbol{\phi}_t - \gamma \boldsymbol{\phi}_{t+1})^\top \right] \\ &= \sum_s d_\mu(s) \mathbb{E}_\mu \left[\rho_k \boldsymbol{\phi}_k (\boldsymbol{\phi}_k - \gamma \boldsymbol{\phi}_{k+1})^\top \middle| S_k = s \right] \\ &= \sum_s d_\mu(s) \mathbb{E}_\pi \left[\boldsymbol{\phi}_k (\boldsymbol{\phi}_k - \gamma \boldsymbol{\phi}_{k+1})^\top \middle| S_k = s \right] \\ &= \sum_s d_\mu(s) \boldsymbol{\phi}(s) \left(\boldsymbol{\phi}(s) - \gamma \sum_{s'} [\mathbf{P}_\pi]_{ss'} \boldsymbol{\phi}(s') \right)^\top \\ &= \boldsymbol{\Phi}^\top \mathbf{D}_\mu (\mathbf{I} - \gamma \mathbf{P}_\pi) \boldsymbol{\Phi},\end{aligned}$$

ATD方法主体架构

- 将智能体与环境交互得到数据分别加入一个大的replay buffer和一个小的replay buffer中；
- 大replay buffer中存储更多的不同时刻的策略采样得到的轨迹（off-policy），而小replay buffer存储的轨迹更贴近当前的策略（on-policy）；



异策略数据的同策略程度

- 即采样的状态转移出现在当前策略的可能性
- 定义 d 为稳定一个分布，经验回放缓存 D 中的数据根据该分布采样得到； d^π

为在当前策略下的状态-行为分布； $L_Q(\theta, d^\pi) = \|Q_{tot}(s, a|\theta) - \mathcal{R}Q_{tot}(s, a|\theta)\|_{d^\pi}^2$

作为混合网络的损失函数，其中 $\mathcal{R}$ 为：

$$\mathcal{R}Q(s, \mathbf{a}) := Q(s, \mathbf{a}) + \mathbb{E}_\mu \left[\sum_{t \geq 0} \gamma^t \left(\prod_{i=1}^t c_i \right) (r_t + \gamma \mathbb{E}_\pi Q(s_{t+1}, \cdot) - Q(s_t, \mathbf{a}_t)) \right]$$

- 则当满足 $0 \leq c_i \leq \frac{\pi(a|s)}{\mu(a|s)}$ 条件时，算子 $\mathcal{R}$ 是 γ -收缩映射的。

同策略程度的计算

- d^π 很难准确估计，因为 d^π 需要需要在策略（on-policy）的交互，而交互的资源比较少；
- 在计算 $\omega(s, a) := d^\pi(s, a)/d^D(s, a)$ 的时候，因为 replay buffer D 中的数据是不同时间的策略采到的数据的混合，因此比值也很难计算；
- 采用一组较早生成的轨迹与一组较新生成的轨迹之间的 f -divergence 变分表示来估计密度比。

定理 2 ^[151] 假设 f 在 $[0, +\infty)$ 上具有一阶导数 f' 。 $\forall P, Q \in \mathcal{P}(\mathcal{X})$ ，使得 $P \ll Q$ 且 $\omega : \mathcal{X} \rightarrow \mathbb{R}^+$ ，

$$D_f(P \| Q) \geq \mathbb{E}_P[f'(\omega(x))] - \mathbb{E}_Q[f^*(f'(\omega(x)))] \quad (5-11)$$

其中 f^* 表示凸共轭，当 $\omega = \frac{dP}{dQ}$ 时等号成立。

同策略程度的计算

定理 2 ^[151] 假设 f 在 $[0, +\infty)$ 上具有一阶导数 f' 。 $\forall P, Q \in \mathcal{P}(\mathcal{X})$, 使得 $P \ll Q$ 且 $\omega : \mathcal{X} \rightarrow \mathbb{R}^+$,

$$D_f(P\|Q) \geq \mathbb{E}_P[f'(\omega(x))] - \mathbb{E}_Q[f^*(f'(\omega(x)))] \quad (5-11)$$

其中 f^* 表示凸共轭, 当 $\omega = \frac{dP}{dQ}$ 时等号成立。

- 根据上述公式, $\omega(s, a)$ 可以通过一个参数为 ϕ 的神经网络来拟合, 且损失函数为:

$$L_\omega(\phi) := \mathbb{E}_{D_{off}}[f^*(f'(\omega_\phi(s, \mathbf{a})))] - \mathbb{E}_{D_{on}}[f'(\omega_\phi(s, \mathbf{a}))]$$

- 因为 λ 的值域为 $[0, 1]$, 因此可以通过sigmoid激活函数来激活。在本文中采取的 f 为Jensen-Shannon散度, 即 $f(x) = x \log x + (1 - x) \log(1 - x)$ 。

同策略程度的计算

- 则损失函数计算后成为二项交叉熵损失，即：

$$L_{\omega}(\phi) := BCE_{s, \mathbf{a} \sim D_{off}}(\omega_{\phi}(s, \mathbf{a}), 0) + BCE_{s, \mathbf{a} \sim D_{on}}(\omega_{\phi}(s, \mathbf{a}), 1)$$

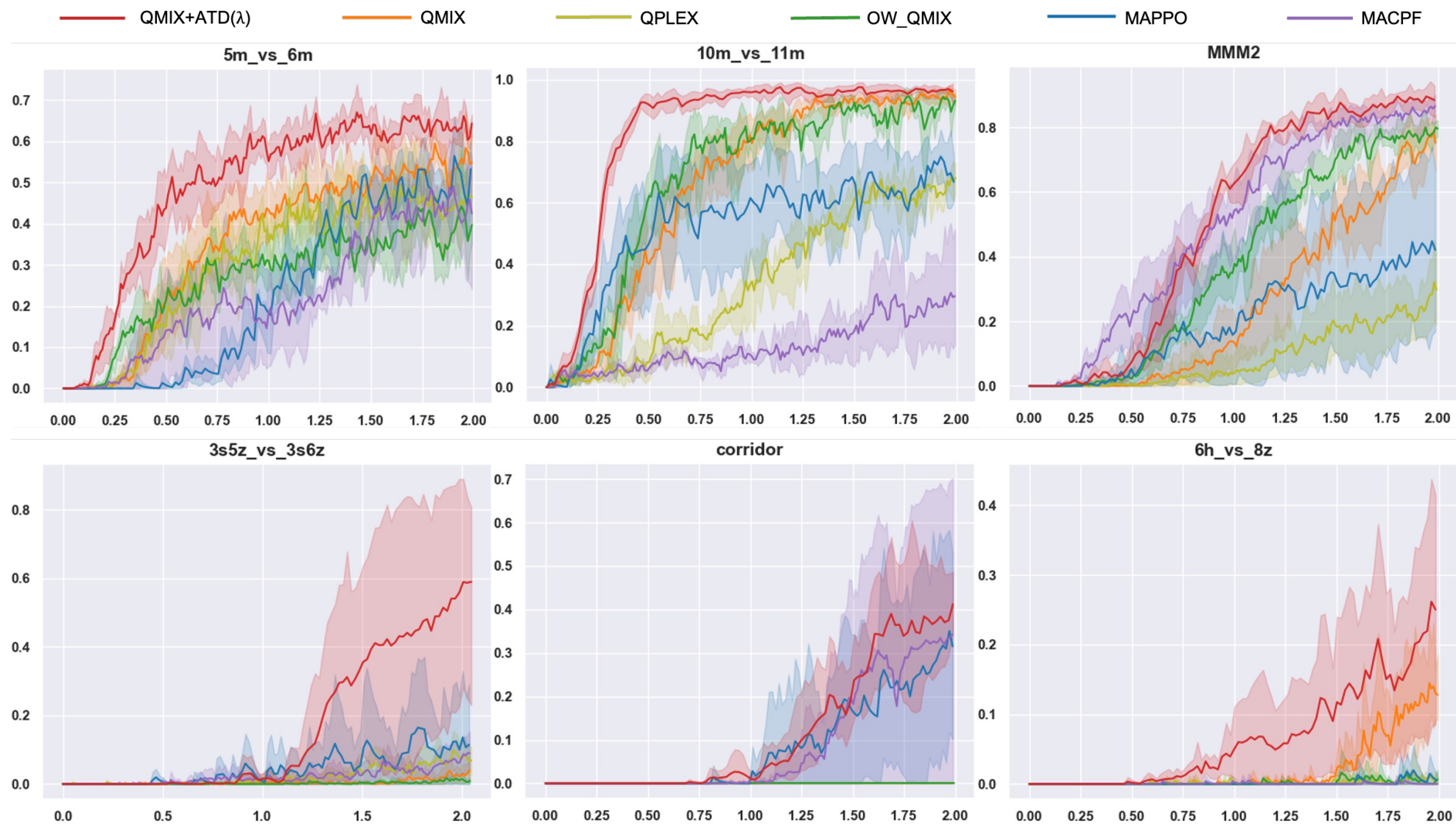
- 则最终的TD更新的目标为：

$$L_Q(\theta; d^{\pi}) \approx L_Q(\theta; D_{off}, \omega_{\phi}) = \mathbb{E}_{(s, \mathbf{a}) \sim D_{off}} [(Q_{tot}(s, \mathbf{a} | \theta) - R_t^{\lambda(s, \mathbf{a})})^2]$$

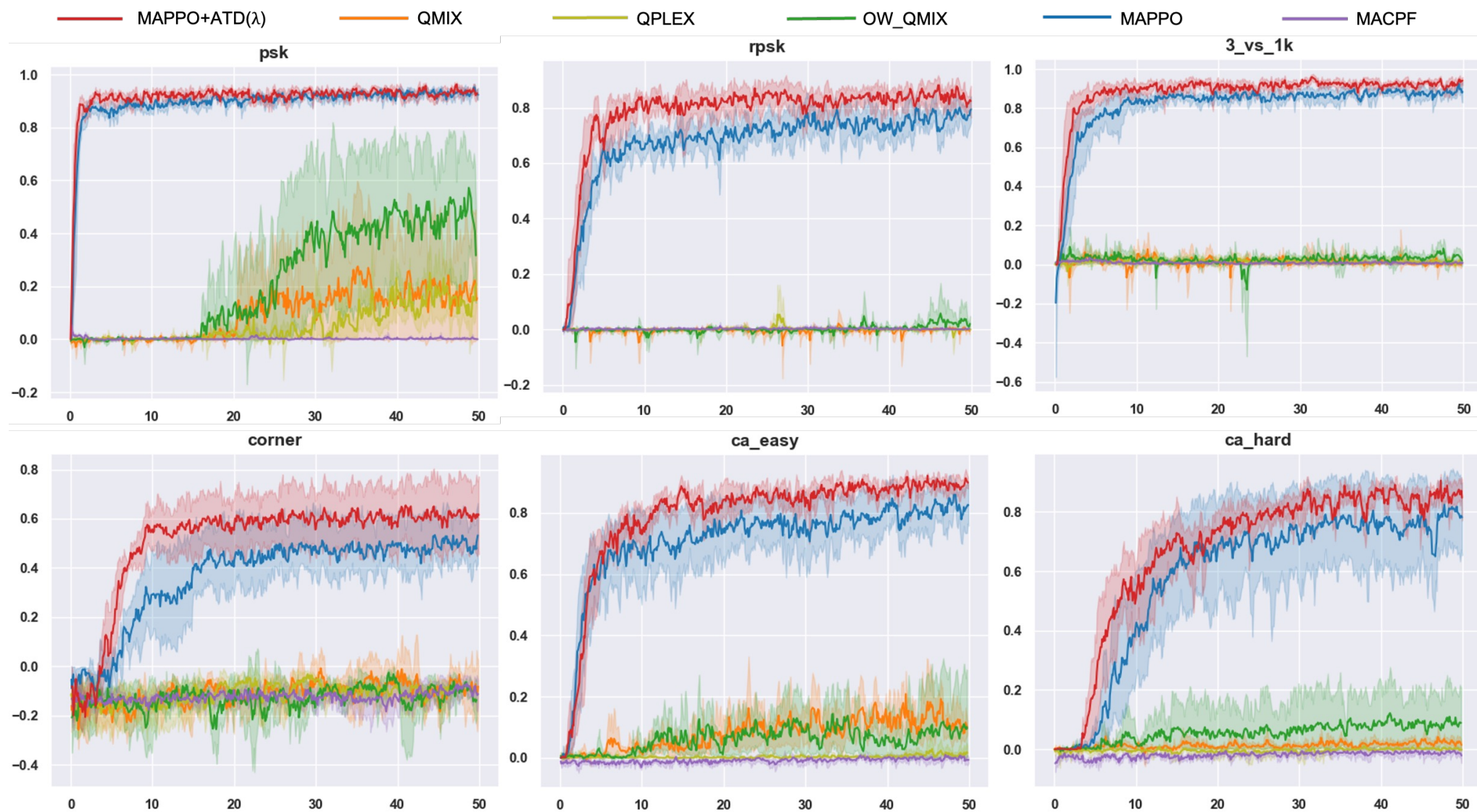
$$R_t^{\lambda(s, a)} = r(s, a) + \gamma[\lambda(s, a) R_{t+1}^{\lambda(s', a')} + (1 - \lambda(s, a) \max_{a' \in A} Q_{tot}(s', a'))]$$

- $\lambda(s, a) = \gamma \sigma(\omega(s, a))$

ATD方法对基于价值的方法的增强



ATD方法对基于策略的方法的增强



主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

问题提出

- Improving Multi-agent Reinforcement Learning with ϵ -greedy;
- Boosting Multi-agent Reinforcement Learning with ϵ -greedy;
- Adaptive TD-Lambda for Cooperative Multi-agent Reinforcement Learning;
- Parallelizing Model-based Multi-agent Reinforcement Learning with Sequence Length;

你的Agent是大模型Agent么???

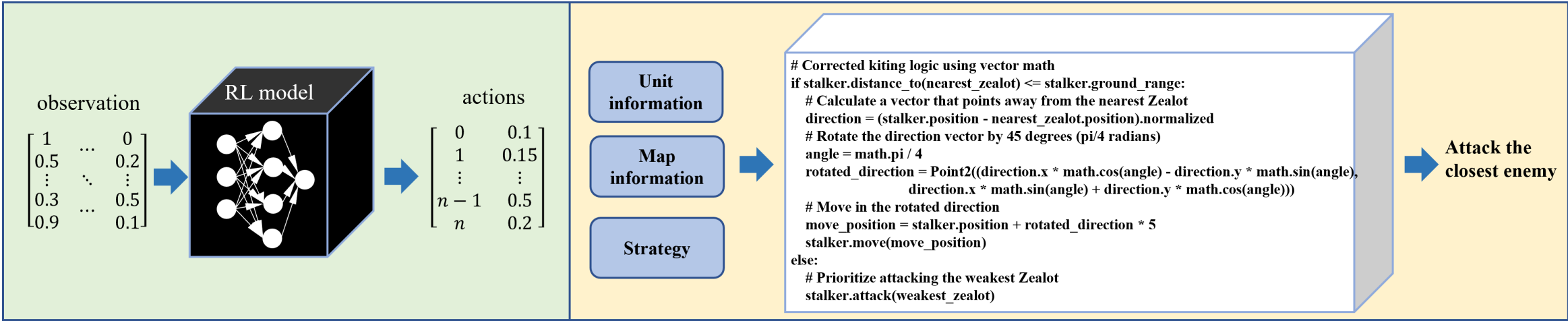
行为树代码生成策略优化

多智能体强化学习：

- 强化学习得到的策略迁移性比较差
- 可能需要数百万步的训练步数
- 深度学习模型的可解释性差

大模型决策树：

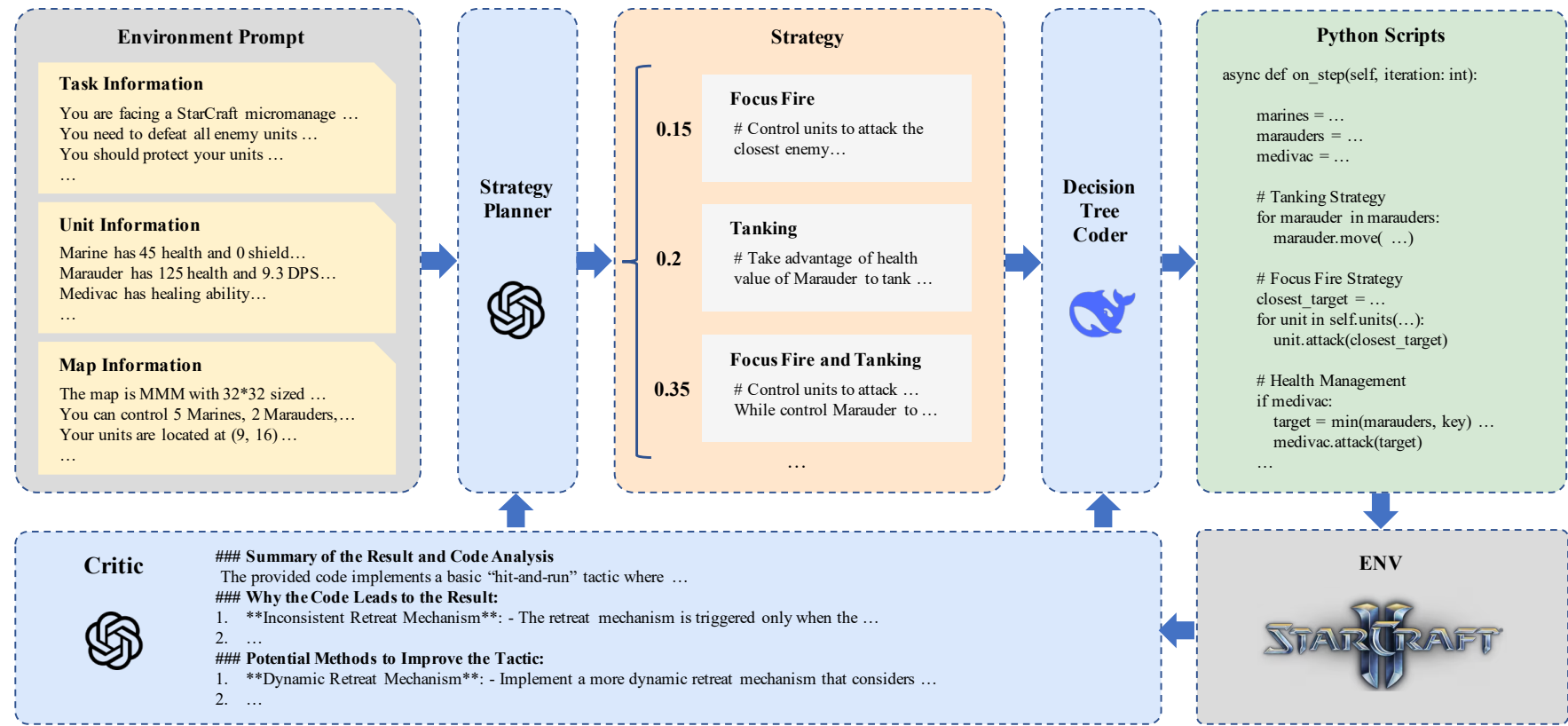
- 白盒决策
- 大模型一轮inference
- 决策树自身具备可解释性



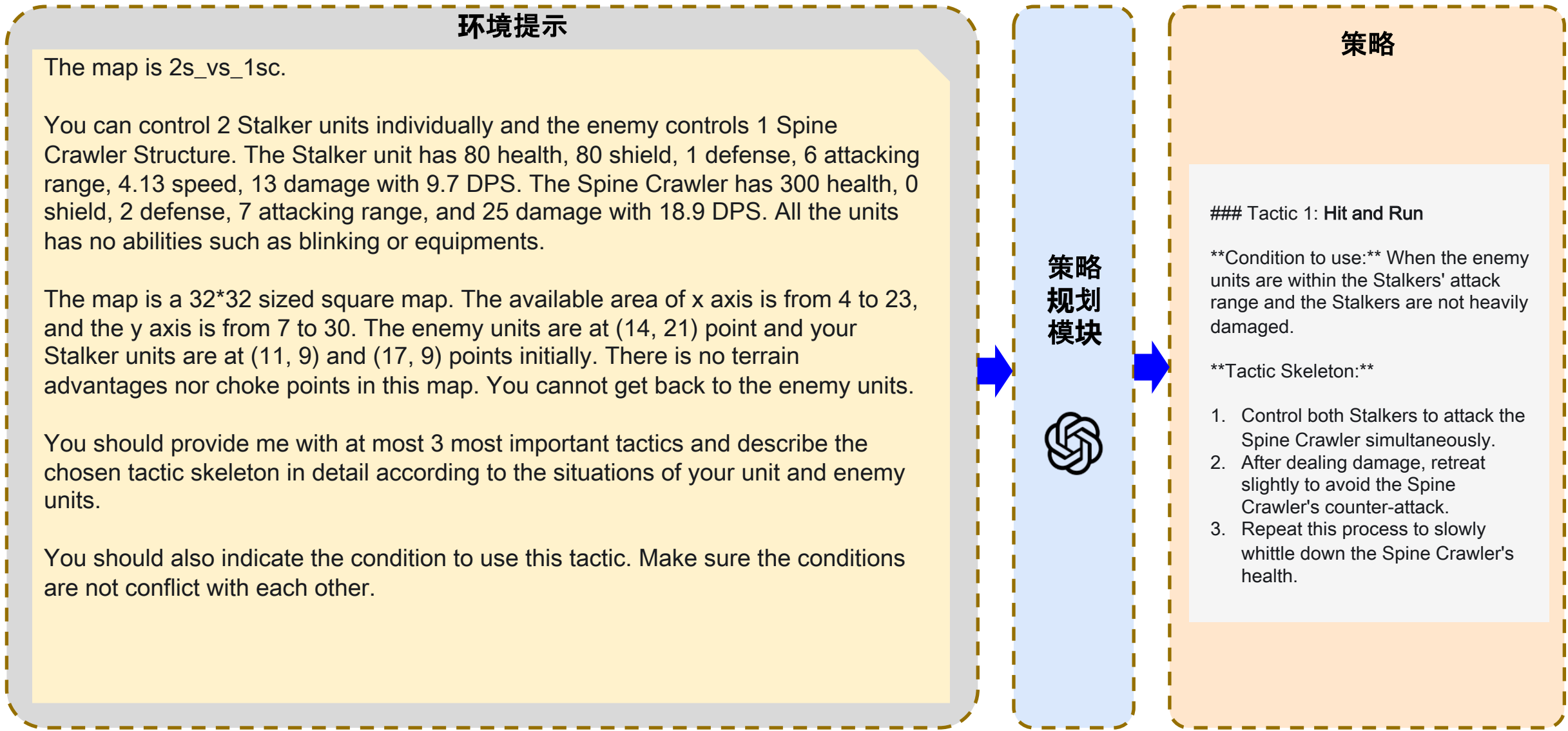
大模型能不能作为元策略，通过生成决策树策略完成决策任务？

基于提示的大语言模型行为树生成

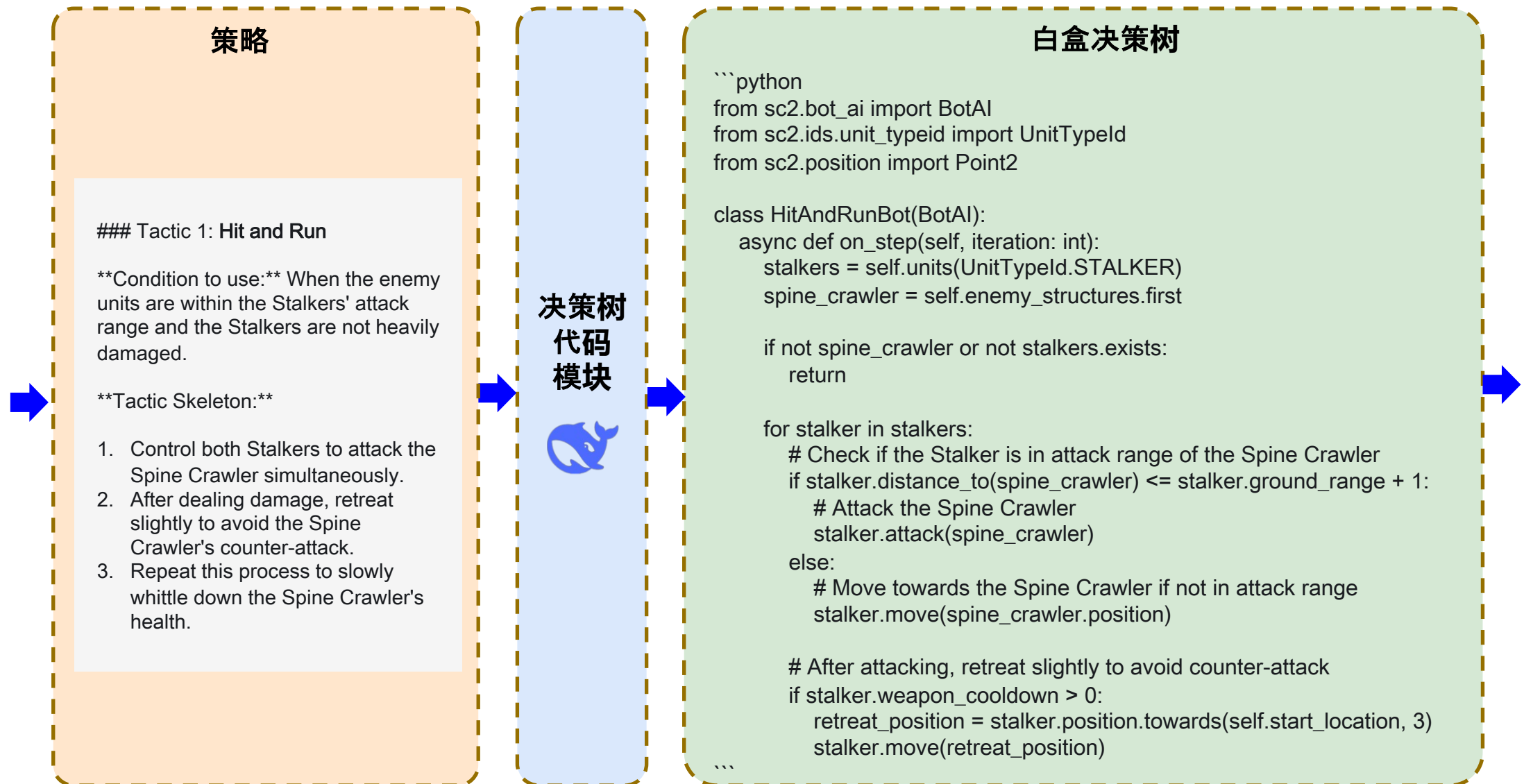
- Planner：规划与学习能力
- Coder：代码实现能力
- Critic：结果分析判别能力



行为树生成（规划）



行为树生成（编码）



行为树生成（执行）

白盒决策树

```
```python
from sc2.bot_ai import BotAI
from sc2.ids.unit_typeid import UnitTypeId
from sc2.position import Point2

class HitAndRunBot(BotAI):
 async def on_step(self, iteration: int):
 stalkers = self.units(UnitTypeId.STALKER)
 spine_crawler = self.enemy_structures.first

 if not spine_crawler or not stalkers.exists:
 return

 for stalker in stalkers:
 # Check if the Stalker is in attack range of the Spine Crawler
 if stalker.distance_to(spine_crawler) <= stalker.ground_range + 1:
 # Attack the Spine Crawler
 stalker.attack(spine_crawler)
 else:
 # Move towards the Spine Crawler if not in attack range
 stalker.move(spine_crawler.position)

 # After attacking, retreat slightly to avoid counter-attack
 if stalker.weapon_cooldown > 0:
 retreat_position = stalker.position.towards(self.start_location, 3)
 stalker.move(retreat_position)

...`
```

## SMAC环境



You Win 4 out of 10 times  
and lose 6 out of 10 times.

You achieve 227.5 scores,  
give 280.0 damages to the  
enemy,  
take 140.6625 damage on  
health,  
and take 170.7625 damage on  
shield on average.

# 行为树生成（批判）

批评家  
模块



### ### Summary of the Result and Code Analysis

The provided code implements a basic "hit-and-run" tactic where Stalkers attack the Spine Crawler and then retreat slightly to avoid counter-attacks. However, the results show that this tactic is not very effective, with a 40% win rate and significant damage taken on both health and shields.

### #### Why the Code Leads to the Result:

1. **\*\*Inconsistent Retreat Mechanism\*\***: - ...
2. **\*\*No Consideration for Health and Shield Levels\*\***: -...
3. **\*\*Fixed Retreat Distance\*\***: ...
4. **\*\*No Group Coordination\*\***: - ...

### #### Potential Methods to Improve the Tactic:

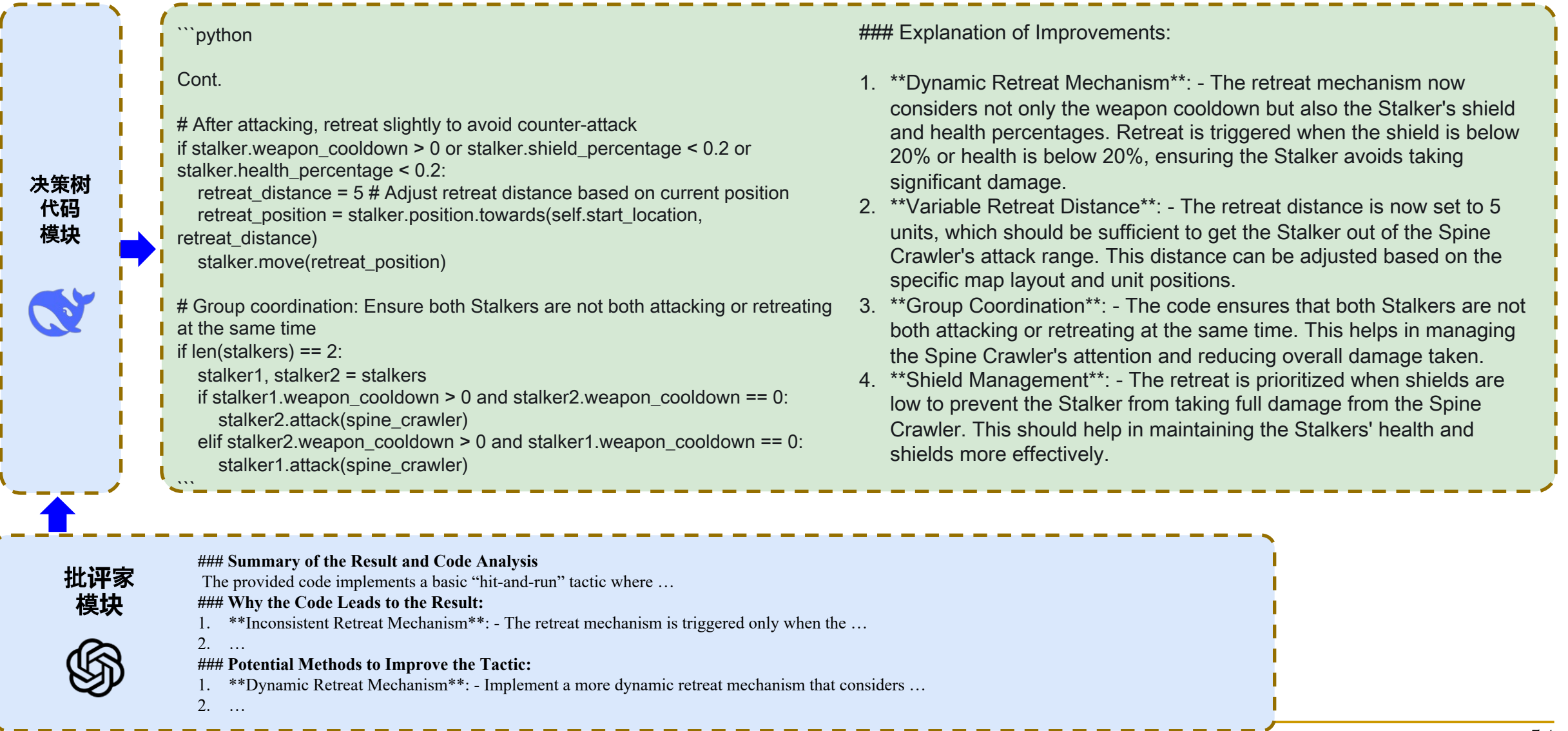
1. **\*\*Dynamic Retreat Mechanism\*\***: - Implement a more dynamic retreat mechanism that considers both the Stalker's weapon cooldown and its current health/shield levels...
2. **\*\*Variable Retreat Distance\*\***: - Use a variable retreat distance based on the Stalker's current position relative to the Spine Crawler. Ensure that the retreat distance is sufficient to get the Stalker out of the Spine Crawler's attack range.
3. **\*\*Group Coordination\*\***: - Coordinate the actions of both Stalkers to ensure that they are not both attacking or retreating at the same time...
4. **\*\*Shield Management\*\***: - Prioritize retreating when shields are low to prevent the Stalker from taking full damage from the Spine Crawler. Consider using a shield recharge mechanic if available in the environment.

### Suggestion: ``[Improve Tactic]``

SMAC环境



# 行为树生成（编码）



# 行为树生成（执行）

```
```python
```

Cont.

```
# After attacking, retreat slightly to avoid counter-attack
if stalker.weapon_cooldown > 0 or stalker.shield_percentage < 0.2 or
stalker.health_percentage < 0.2:
    retreat_distance = 5 # Adjust retreat distance based on current position
    retreat_position = stalker.position.towards(self.start_location,
retreat_distance)
    stalker.move(retreat_position)

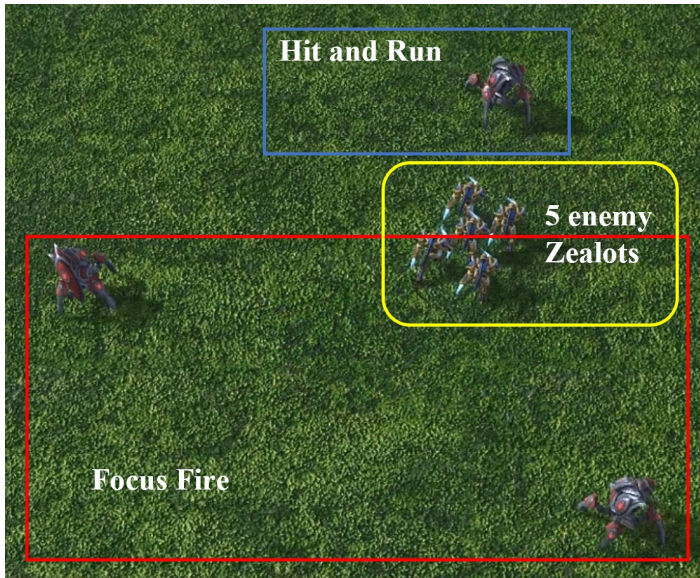
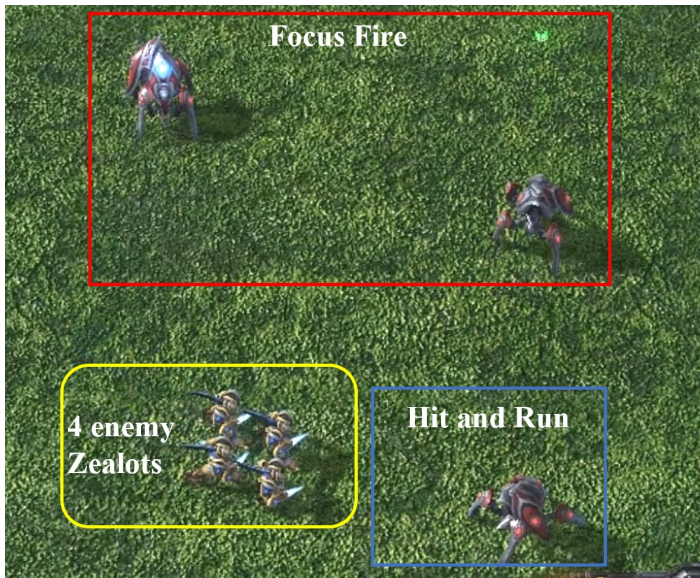
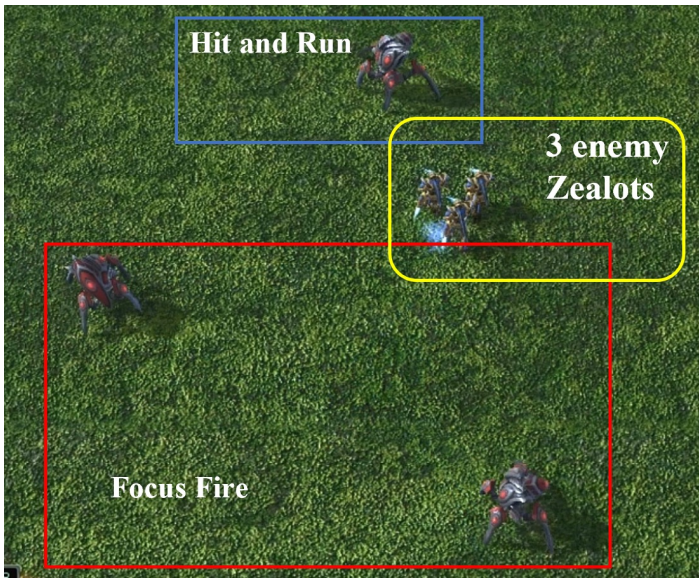
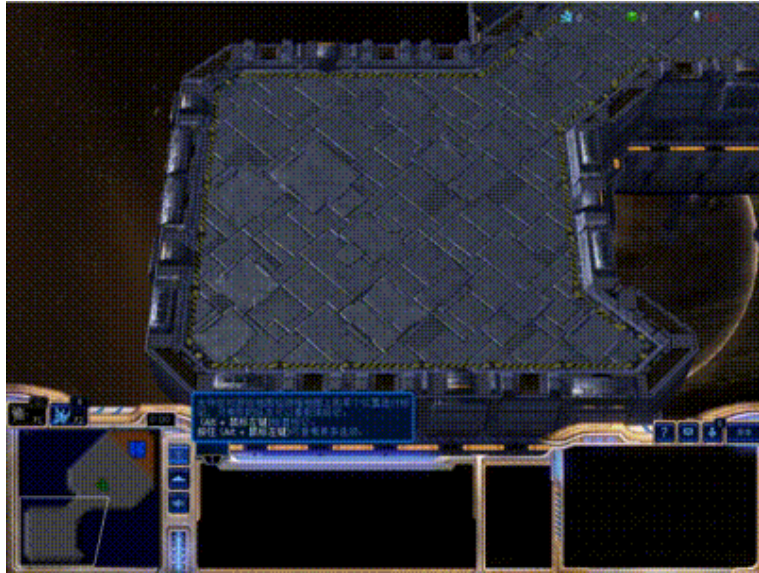
# Group coordination: Ensure both Stalkers are not both attacking or
retreating at the same time
if len(stalkers) == 2:
    stalker1, stalker2 = stalkers
    if stalker1.weapon_cooldown > 0 and stalker2.weapon_cooldown == 0:
        stalker2.attack(spine_crawler)
    elif stalker2.weapon_cooldown > 0 and stalker1.weapon_cooldown == 0:
        stalker1.attack(spine_crawler)
...
```



You Win 10 out of 10 times
and lose 0 out of 10 times.

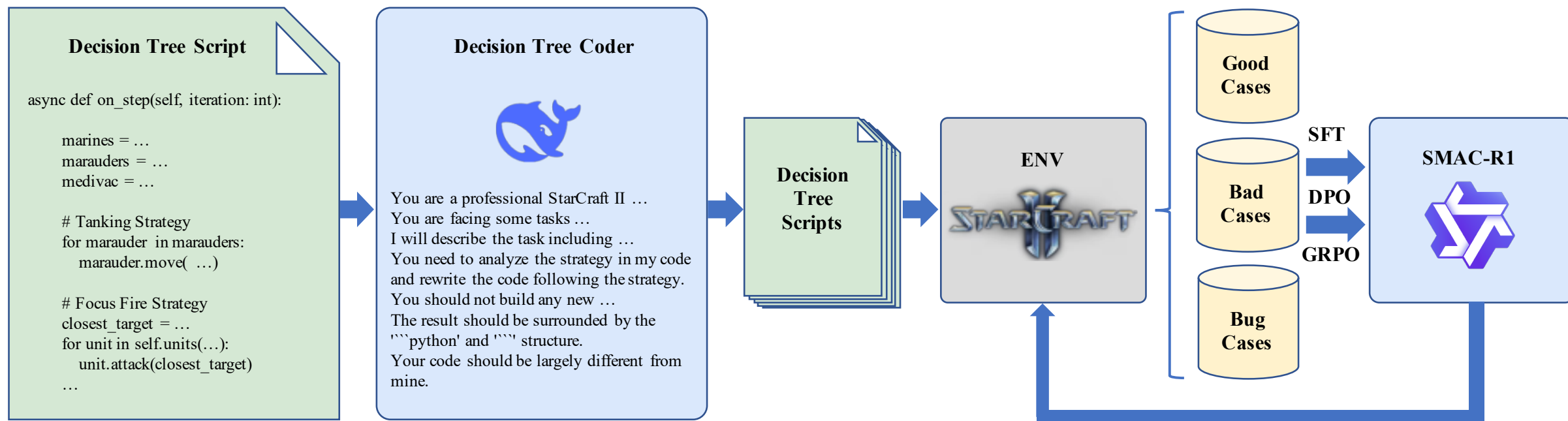
You achieve 350.0 scores,
give 331.2 damages to the
enemy,
take 115.3125 damage on health,
and take 405.0875 damage on
shield on average.

行为树生成实验



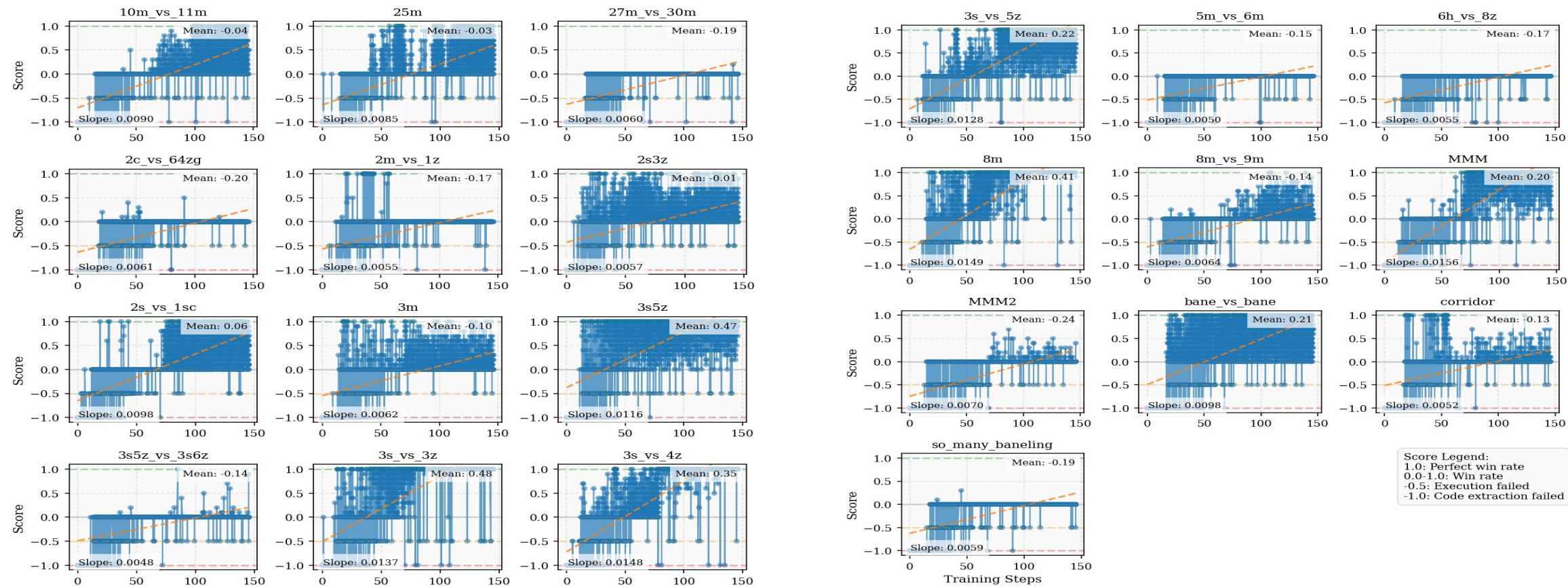
行为树生成能力蒸馏与提升

- 模型太大难以微调，需要将大模型知识蒸馏到小LLM
- 代码数据过少，需要另一个大模型做数据增广
- SMAC作为奖励模型提供代码的真实胜率和分数



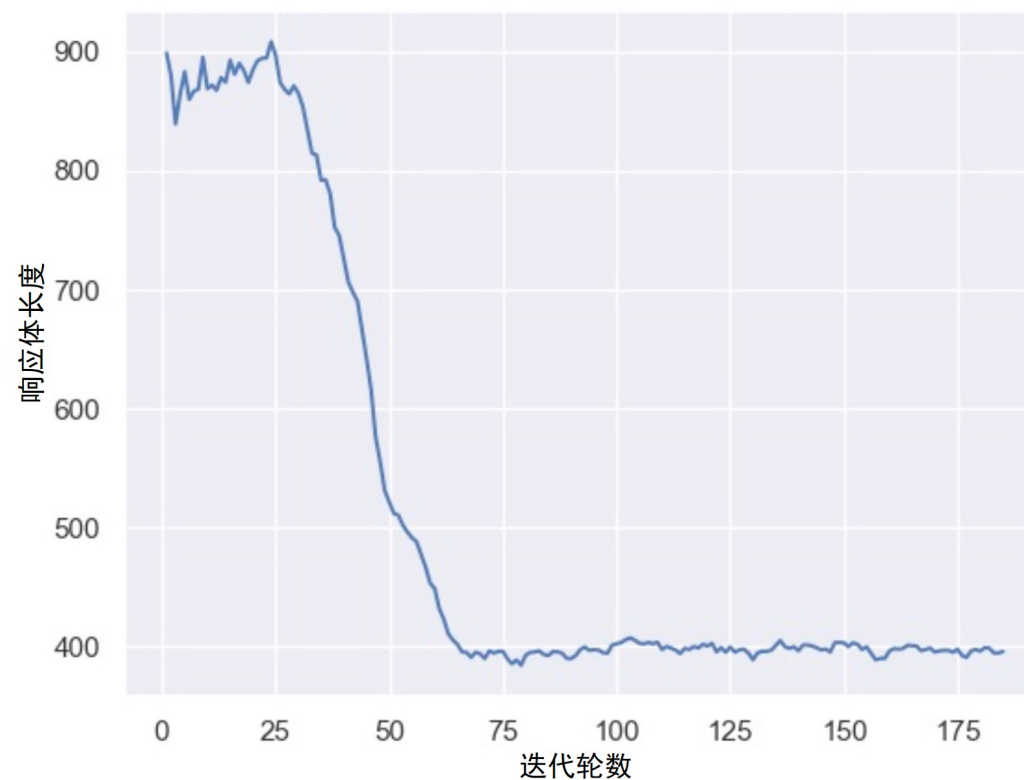
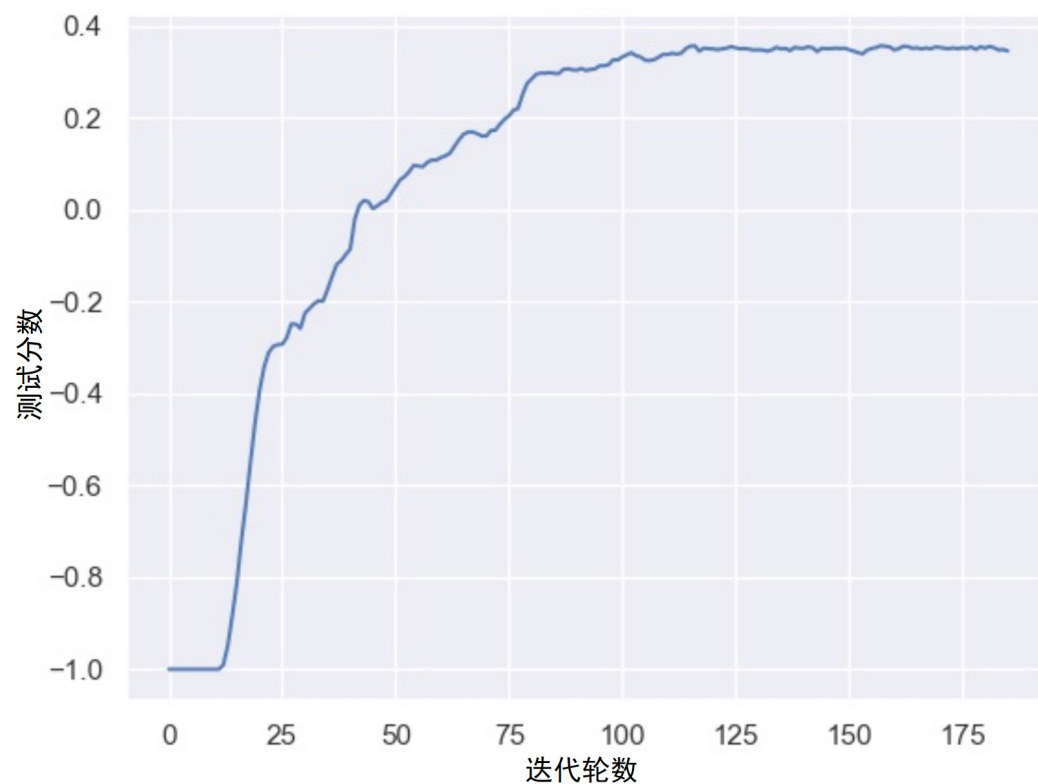
GRPO算法在SMAC任务上的表现

- 1分：无法从大模型的响应中检索到有效行为树；
- 0.5分：行为树脚本中存在错误；
- 0~1分：生成代码在SMAC任务中的胜率。

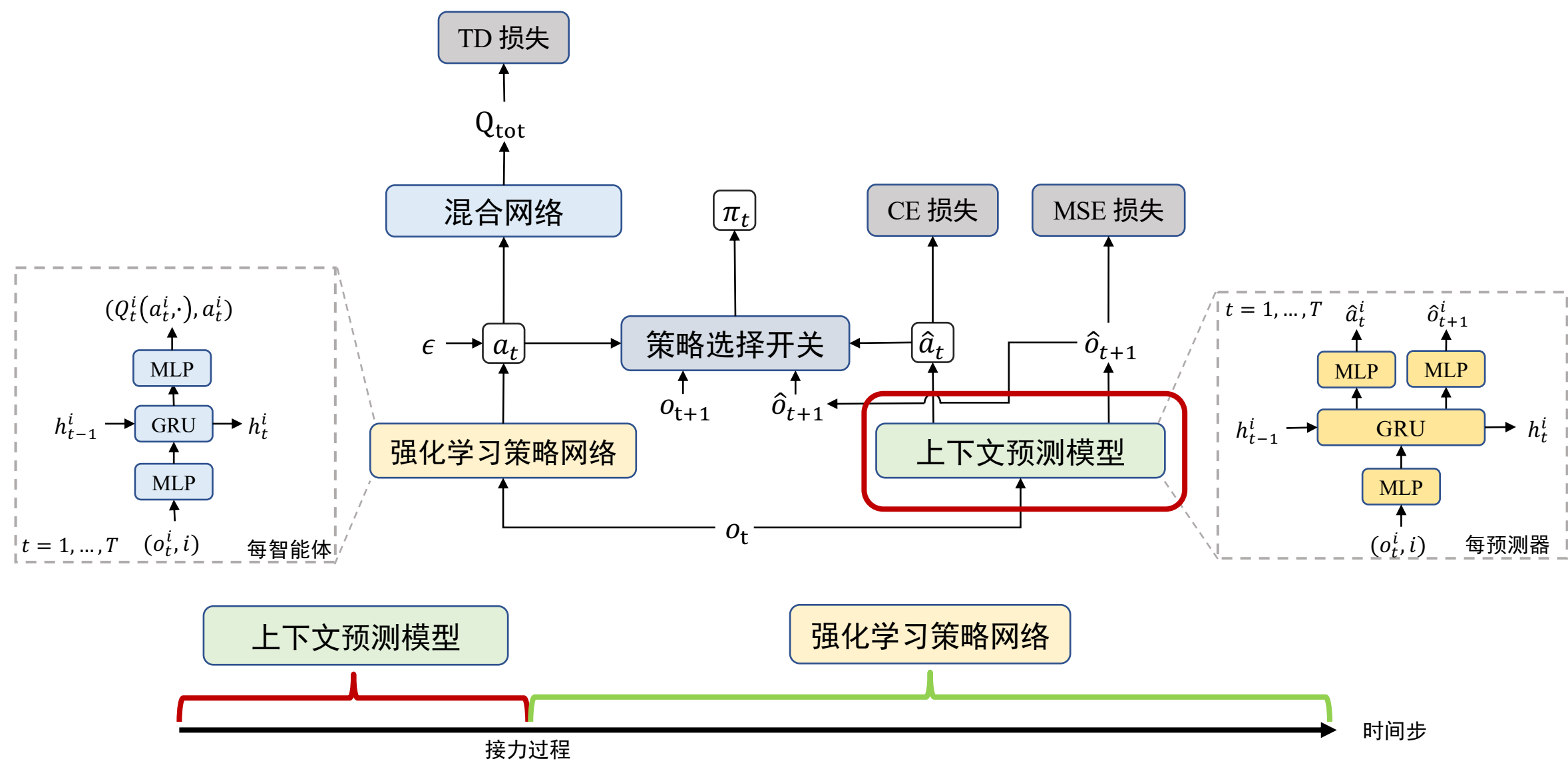


GRPO算法在SMAC任务上的表现

- 与解决数学问题不同，在SMAC任务上，胜率随着思考的内容的减少而增加；
- 在决策问题上，GRPO的优化方向更倾向于更精确、更短的回答。

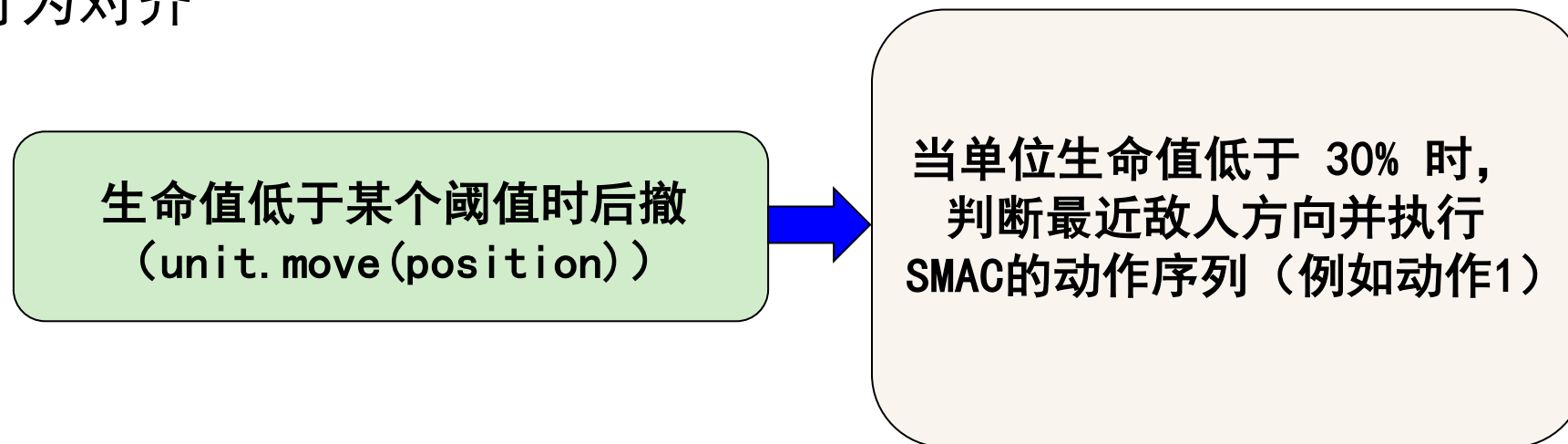


回看：基于经验缓存样本监督的模型接力方法



行为树策略辅助强化学习训练

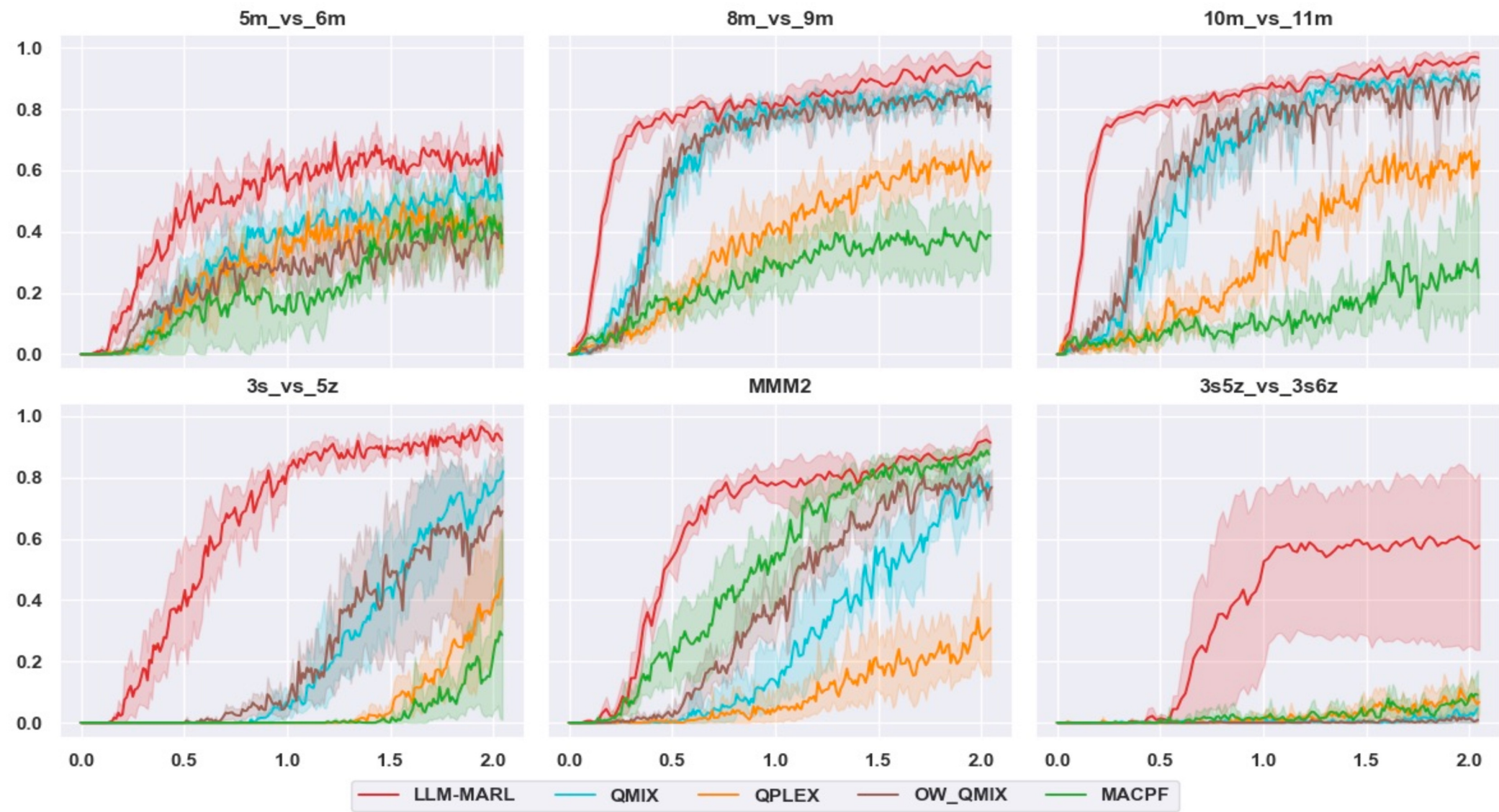
- 策略行为对齐



- “接力” 时间控制退火机制

$$N_t = \max(N_{\min}, N_0 \cdot \exp(-\lambda \cdot t))$$

行为树策略辅助强化学习训练



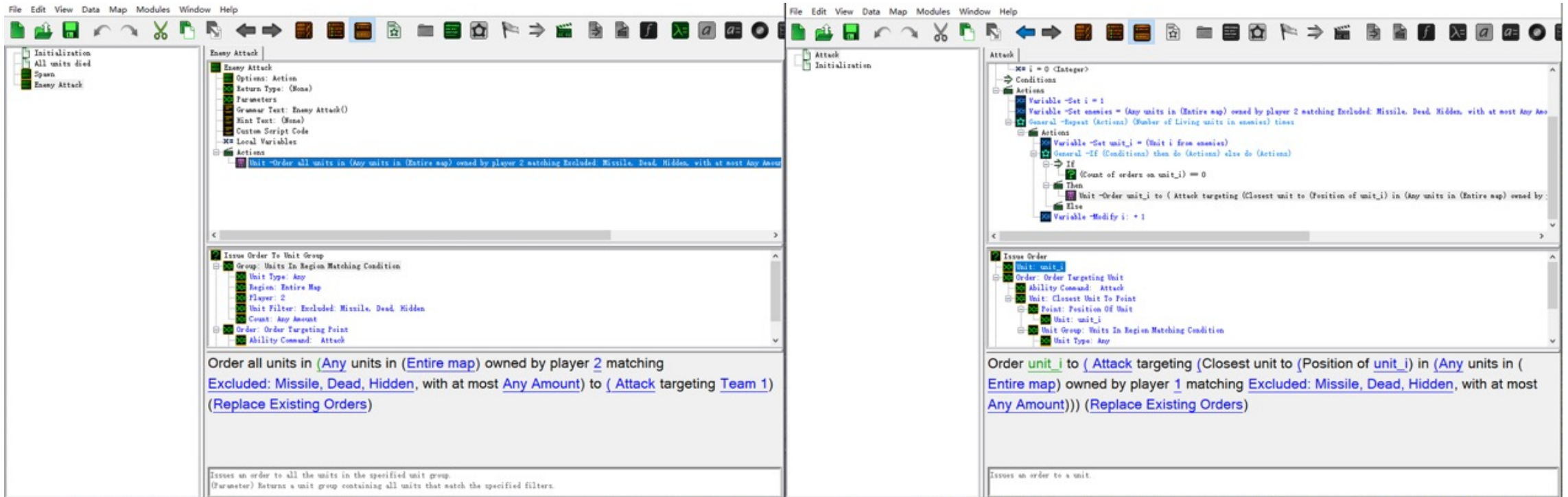
SMAC-Hard

- SMACV1初始化丰富程度不够，相对单一，MARL算法在这个环境中更像树形展开然后得到一条固定的路径；
- 开环控制已然可以解决大部分的SMACV1的场景，即决策仅取决于时间步，状态和观测的信息可以被掩盖。
- SMACV2引出了初始化设定，兵种和位置可以通过概率的形式生成。

SMAC-Hard

- SMACV1的对手是内置脚本， 且是控制所有的角色移动到agent方的出生点， 并攻击沿途的敌人；
- 内置脚本过于简单；
- 存在取巧的获胜方式；
- 是否可以将LLM-SMAC脚本应用为对手策略？

SMAC-Hard



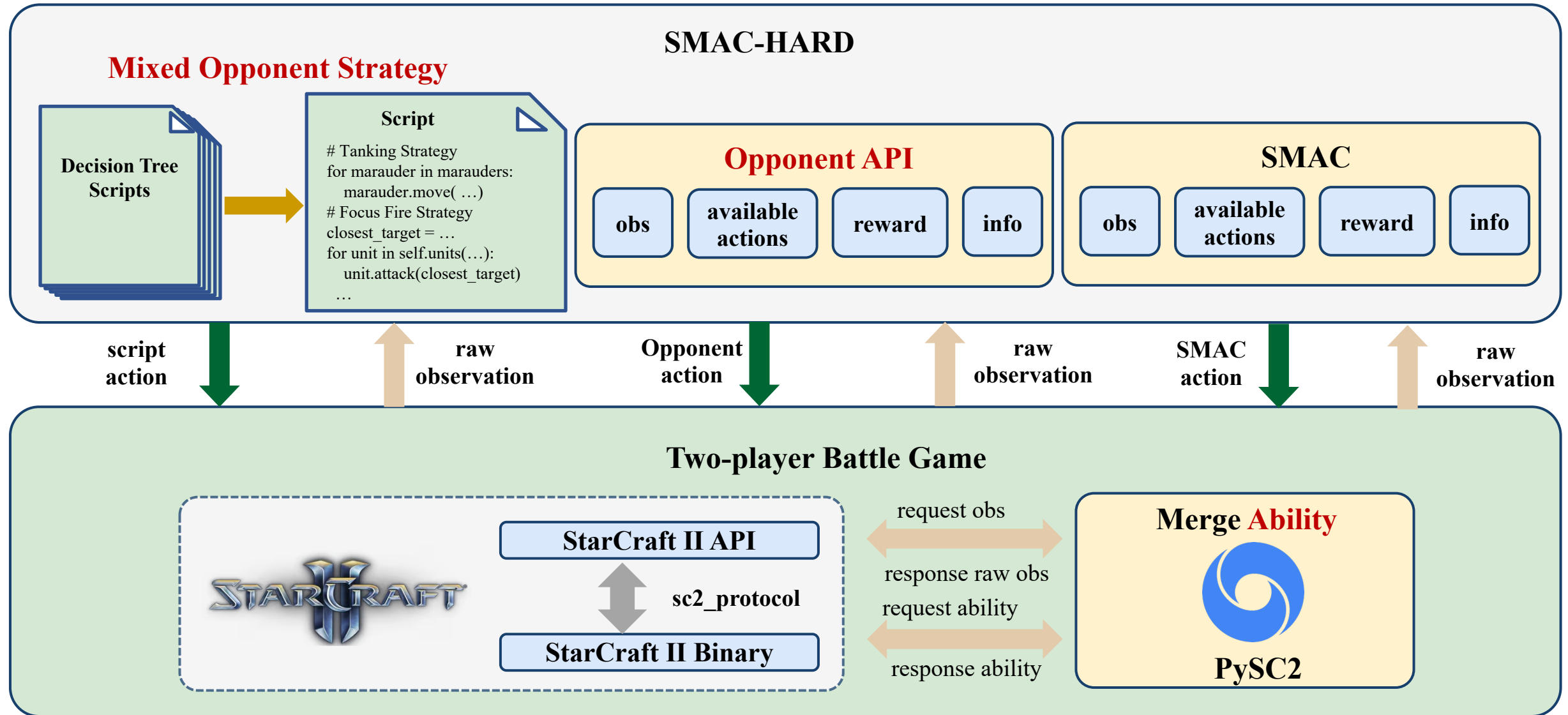
- Order all units in (Any units in (Entire map) owned by player 2 matching Excluded: Missile, Dead, Hidden, with at most Any Amount) to (Attack targeting Team 1) (Replace Existing Orders) (移动到对手位置自动攻击)
- Order unit_i to (Attack targeting (Closest unit to (Position of unit_i) in (Any units in (Entire map) owned by player 1 matching Excluded: Missile, Dead, Hidden, with at most Any Amount)))(Replace Existing Orders) (攻击最近的角色)

SMAC-Hard



- 智能体学习到的策略模型更容易过拟合到某一个对手策略；
- 或利用对手策略的漏洞而拟合到取巧方法上。

SMAC-Hard



SMAC-Hard

相比于SMAC

- ✓ 混合对手脚本
- ✓ 自博弈接口
- ✓ 战争迷雾
- ✓ 技能释放
- ✓ 还原视野
- ✓ 地形优势

随机攻击角色



悬崖下部署



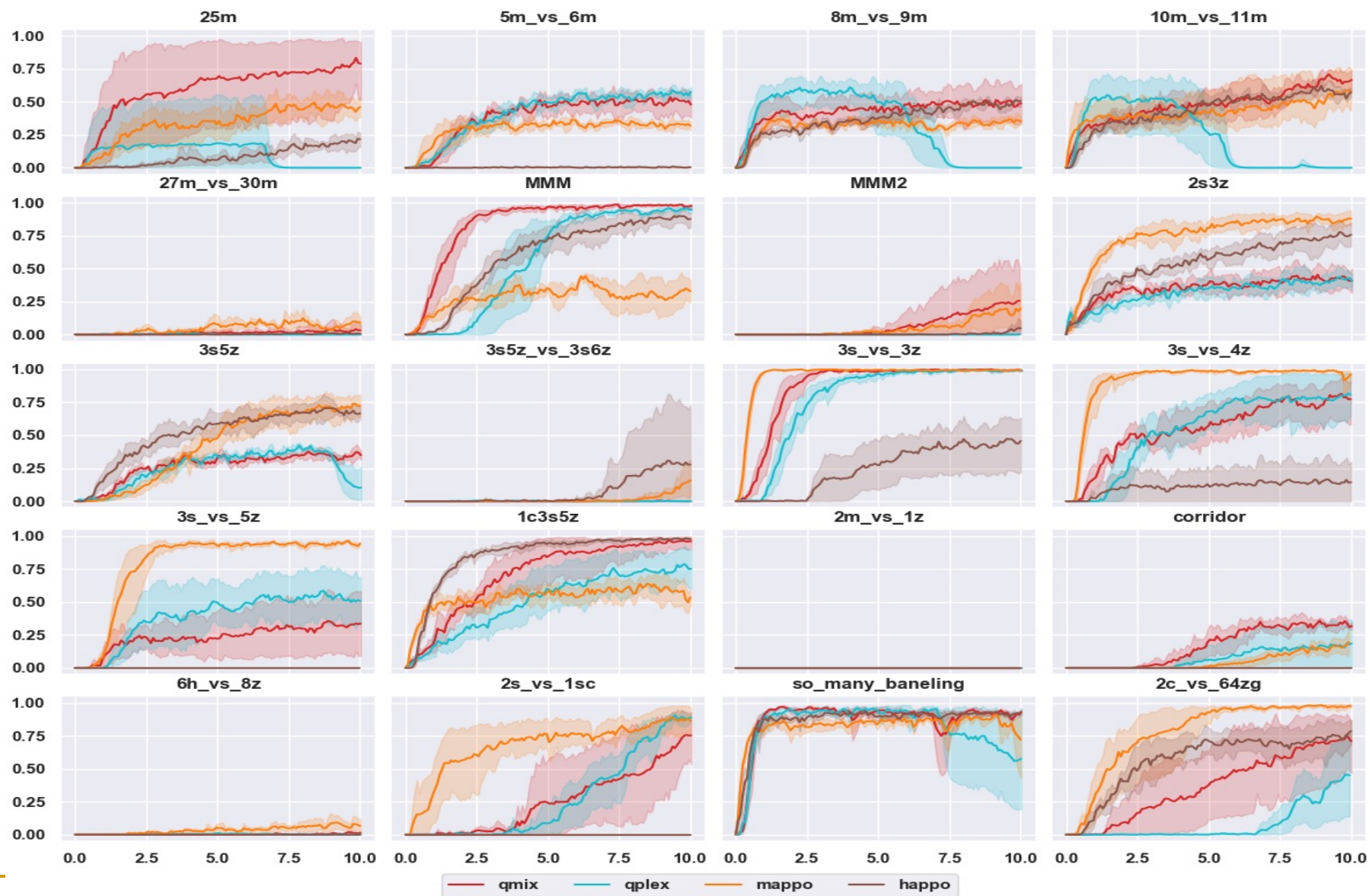
分组攻击角色



地形优势技能



SMAC-Hard



SMAC-Hard

SMAC-Hard提出更大挑战

SMAC-HARD	QMIX	QPLEX	LDSA	MAPPO	HAPPO
3m	0.9938	0.0438	1	1	0.4375
8m	0.9628	0.0875	0.9687	0.6438	0.8625
5m_vs_6m	0.4375	0.6188	0.4	0.3138	0.004
8m_vs_9m	0.4938	0	0.7187	0.3528	0.5006
10m_vs_11m	0.6813	0	0.625	0.5681	0.5672
25m	0.7813	0	0.4375	0.4562	0.2092
27m_vs_30m	0.0313	0	0.125	0.0938	0.0063
2s3z	0.4125	0.4313	0.7737	0.8735	0.7547
3s5z	0.3625	0.1063	0.1876	0.7225	0.6691
3s5z_vs_3s6z	0	0	0.1319	0.1569	0.2573
1c3s5z	0.9625	0.7563	0.9375	0.5445	0.975
3s_vs_3z	0.9913	0.9875	1	0.9875	0.4875
3s_vs_4z	0.7938	0.7938	0.7815	0.9875	0.15
3s_vs_5z	0.3438	0.5	0.875	0.9407	0
bane_vs_bane	0.975	0.2313	0.8229	0.9125	0
so_many_baneling	0.9625	0.5938	0.875	0.8813	0.9188
2s_vs_1sc	0.7563	0.8875	0.4375	0.8375	0
2m_vs_1z	0	0	0	0	0
2c_vs_64zg	0.7891	0.5	0.9062	0.9741	0.7686
MMM	0.9875	0.95	0.875	0.3267	0.8813
MMM2	0.275	0.0006	0.5312	0.1925	0.0425
6h_vs_8z	0.0188	0	0.0625	0.0607	0
corridor	0.3063	0.1688	0	0.1804	0

SMAC-Hard 黑盒测试

SMAC-HARD	QMIX	QPLEX	MAPPO	HAPPO
3m	0.0117	0.0898	0.1028	0.0573
8m	0.0375	0	0.0513	0.1217
5m_vs_6m	0	0	0	0
8m_vs_9m	0	0	0	0
10m_vs_11m	0	0	0	0
25m	0	0	0	0
27m_vs_30m	0	0	0	0
2s3z	0.0508	0	0.1217	0.0959
3s5z	0.0313	0	0.0375	0.0705
3s5z_vs_3s6z	0	0	0	0
1c3s5z	0.0703	0	0.1027	0.0912
3s_vs_3z	0.3359	0.3555	0.4125	0.3724
3s_vs_4z	0.7461	0.5052	0.4467	0
3s_vs_5z	0.5305	0.8573	0.5359	0
bane_vs_bane	0.2578	0	0.3484	0
so_many_baneling	0	0	0	0
2s_vs_1sc	0	0	0	0
2m_vs_1z	0	0	0	0
2c_vs_64zg	0.6238	0	0.3419	0.2565
MMM	0	0	0	0
MMM2	0	0	0	0
6h_vs_8z	0.0898	0	0	0
corridor	0	0	0	0

主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

高性能模型学习

基于模型的强化学习表现很好，但代价是是额外的**计算开销**，**内存开销**，和**训练时间**

Atari 100K 基准上的训练开销（大约2小时的游玩数据）

MBRL Method	Wall-clock Time	Hardware Requirement
SimPLE [KBM+'2019]	5 days	1 x P100
EfficientZero [YLK+'2021]	7 hours	4 x RTX3090
IRIS [MAF+'2023]	7 days	1 x A100
TWM [RHU+'2023]	10 hours	1 x A100
DreamerV3 [HPB+'2023]	12 hours	1 x V100
STORM [ZWS+'2023]	9.3 hours	1 x RTX3090

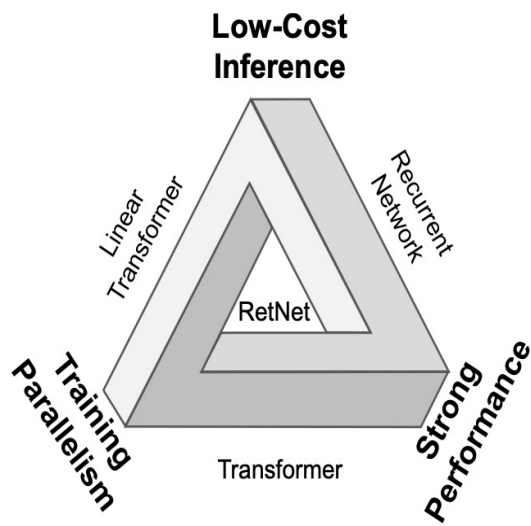
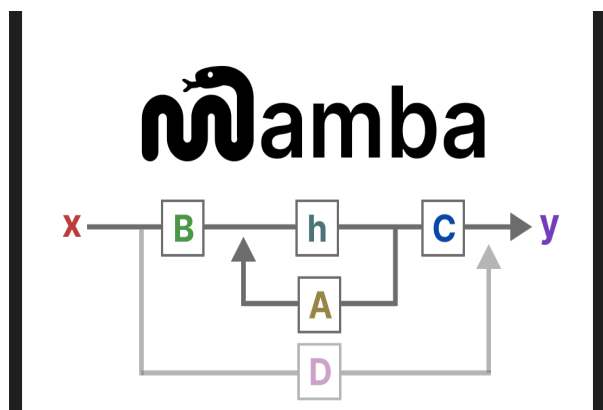
高性能模型学习

如何克服这一挑战？

可以从大模型领域中 Transformer 模型的挑战者中汲取灵感

核心思想：并行训练 与 串行推理

核心技术：并行扫描（Parallel Scan）



RWKV

高性能模型学习

给定序列 $a_1, a_2, \dots, a_T$ 和 $b_1, b_2, \dots, b_T$ ，状态变量为 $x_1, x_2, \dots, x_T$ 线性动力系统可以通过以下方程计算得到：

$$x_1 = b_1,$$

$$x_2 = a_2 x_1 + b_2,$$

$$\vdots$$

$$x_T = a_T x_{T-1} + b_T,$$

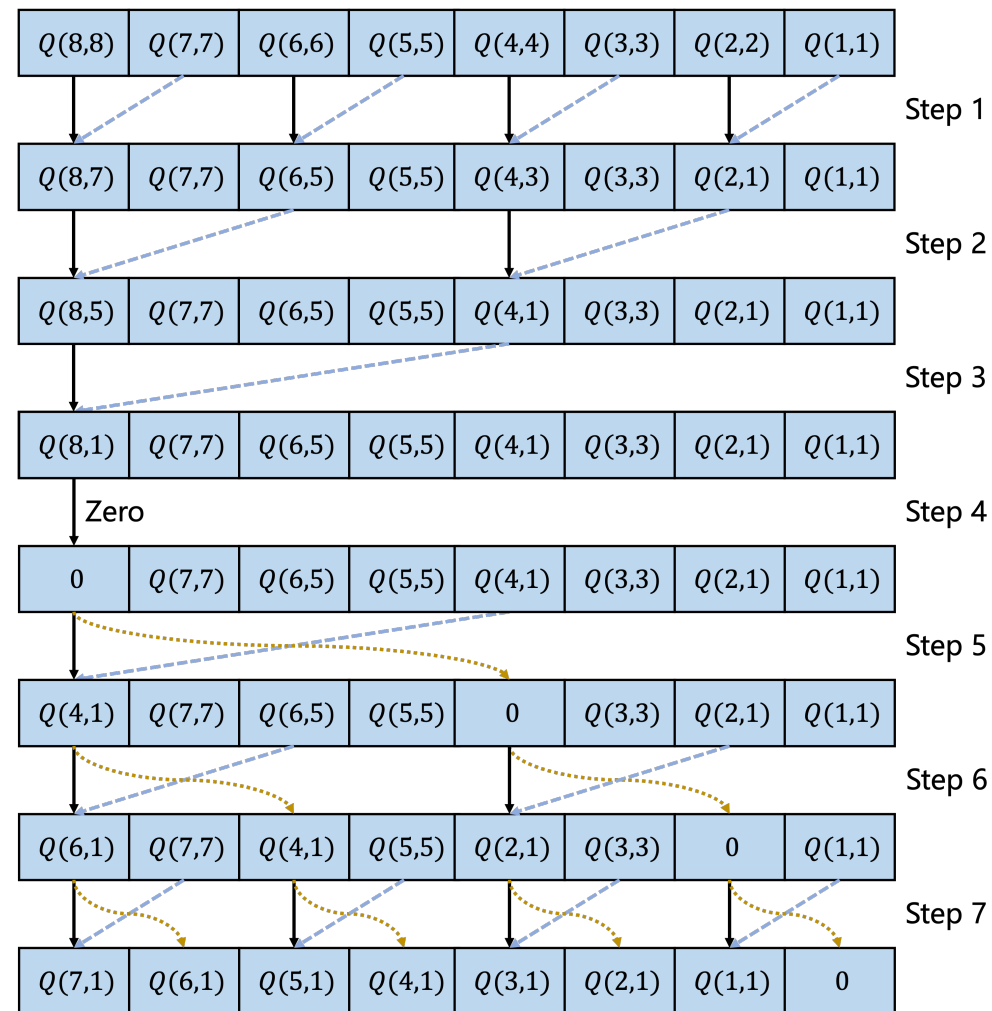
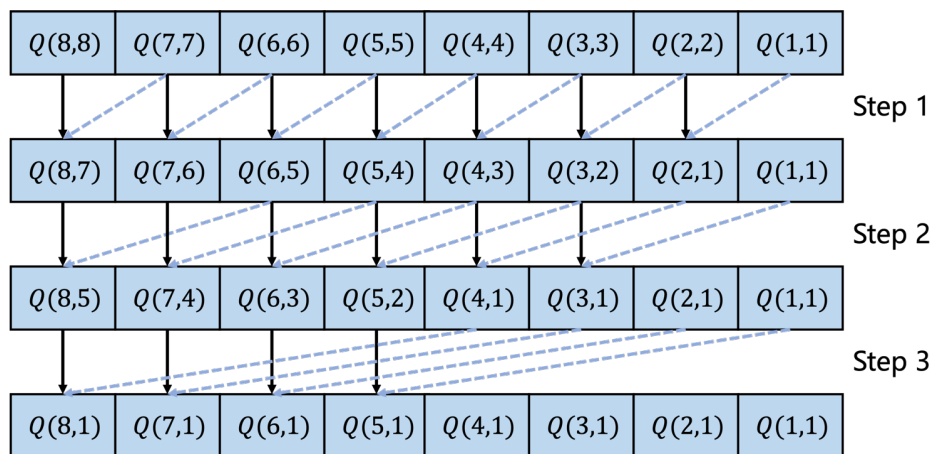
得到状态 $x_1, x_2, \dots, x_T$ 需要进行多少步计算？



看起来需要 T 步，但是实际上通过并行扫描只需要更少的步数

高性能模型学习

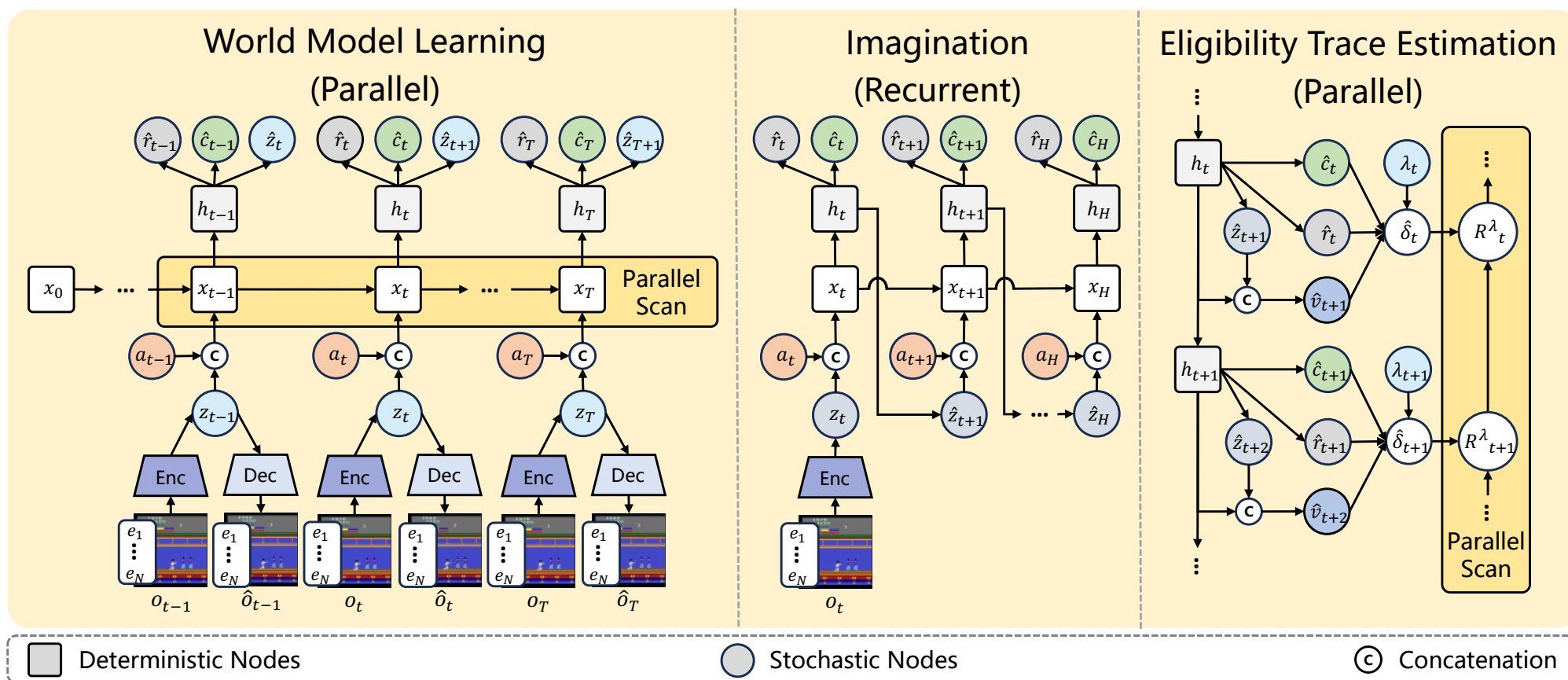
考虑以下当 $T = 8$ 的情况，当采用不同的并行扫描算法时，只需要分别进行 $\lceil \log_2 T \rceil$ 和 $\lceil 2\log_2 T \rceil$ 步计算



高性能模型学习

提出了 **Parallelized Model-based RL** framework

该框架能够将并行扫描算法同时应用于世界模型学习 & 策略学习



高性能模型学习

针对强化学习的任务特性，在世界模型中使用了一种 **Linear Attention** 的变体结构

Linear Attention 的经典核函数

$$\begin{aligned} q_t, k_t &= 1 + \text{ELU}(u_t W_q), 1 + \text{ELU}(u_t W_k) \\ v_t &= \text{Sigmoid}(u_t W_r) \odot u_t W_v, \\ g_t &= \text{Sigmoid}(\mu \odot u_t + (1 - \mu) \odot u_{t-1}) W_g, \\ x_t &= g_t \odot x_{t-1} + k_t^\top v_t, \\ y_t &= \text{RMSNorm}(q_t x_t) W_h + u_t, \\ h_t &= \text{SiLU}(y_t W_g) \odot y_t W_y + y_t. \end{aligned}$$

Token mixing 模块

Data-dependent decay rate (遗忘门)

Post-Norm

Architecture	Minimum complexity			More expressive		
	Training	Inference step	Imagination step	Parallel	Resetable	Selective
Atten	$\mathcal{O}(L^2)$	$\mathcal{O}(L^2)$	$\mathcal{O}((L + H)^2)$	✓	✓	✓
RNN	$\mathcal{O}(L)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	✗	✓	✓
SSM (FFT)	$\mathcal{O}(L \log L)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	✓	✗	✗
SSM (Scan)	$\mathcal{O}(L)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	✓	✓	✗
Lin-Atten (Scan)	$\mathcal{O}(L)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	✓	✓	✓

高性能模型学习

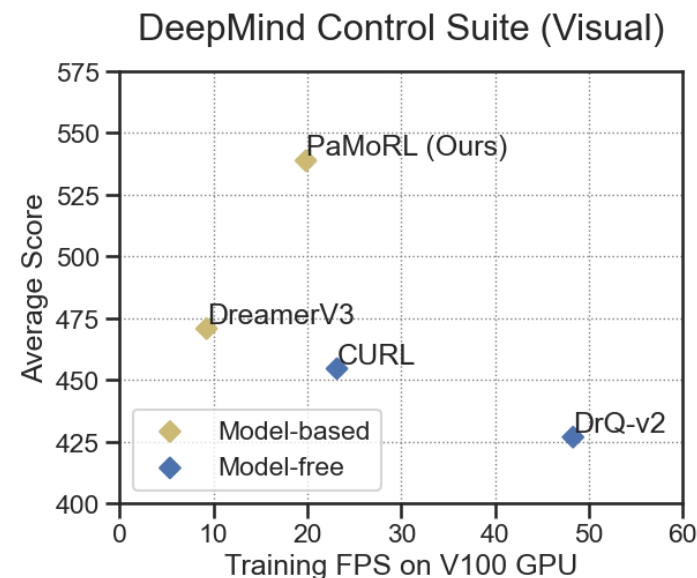
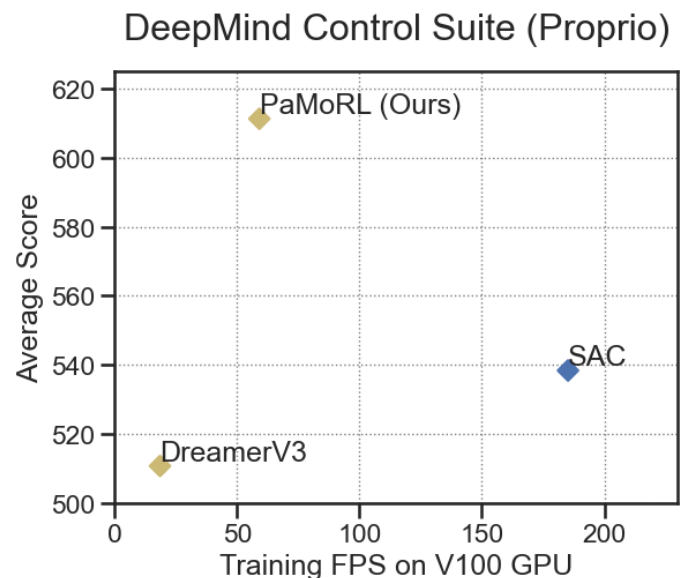
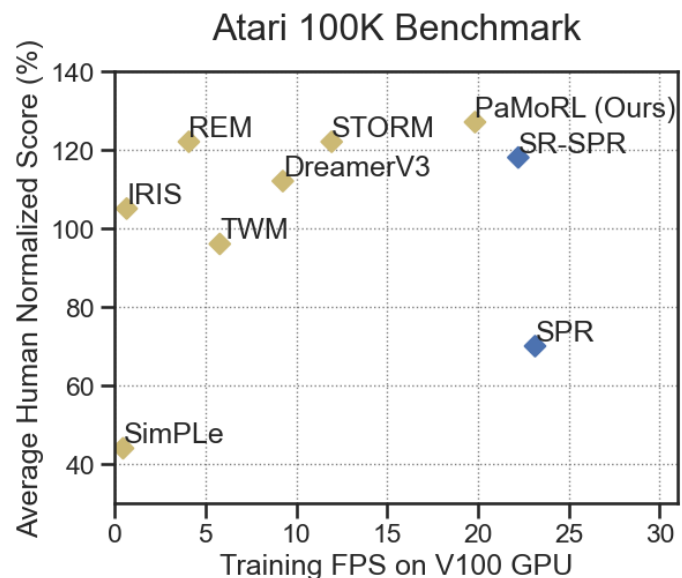
$$\begin{aligned}\text{TD-}\lambda: \quad R_t^\lambda &= \hat{r}_t + (\gamma \hat{c}_t) [(1 - \lambda)v_\phi(s_{t+1}) + \lambda R_{t+1}^\lambda] \\ &= (\lambda \gamma \hat{c}_t) R_{t+1}^\lambda + [\hat{r}_t + (1 - \lambda)(\gamma \hat{c}_t)v_\phi(s_{t+1})]\end{aligned}$$

$$\text{资格迹: } R_t = v_\phi(s_t) + E_t,$$

$$E_t = (\gamma \hat{c}_t \lambda_t) E_{t+1} + \rho_t [\hat{r}_t + (\gamma \hat{c}_t)v_\phi(s_{t+1}) - v_\phi(s_t)]$$

此外，还注意到强化学习中的**资格迹估计**（**Eligibility Trace Estimations**）
的计算过程也能够使用并行扫描实现加速 🤖

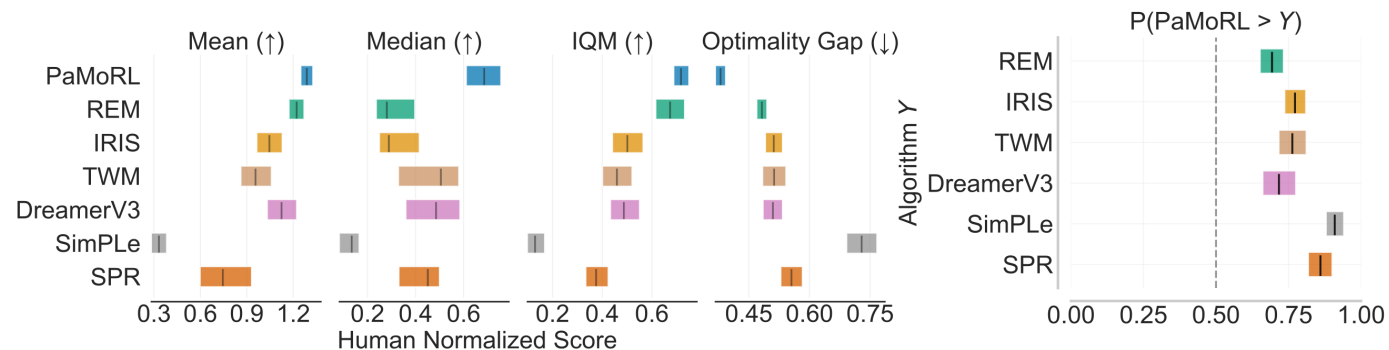
高性能模型学习



Task	Atari 100K	Proprio (easy)	Proprio (hard)	Vision (easy)	Vision (hard)
Runtime	3.5 hours	0.94 hours	1.88 hours	2.74 hours	7.1 hours

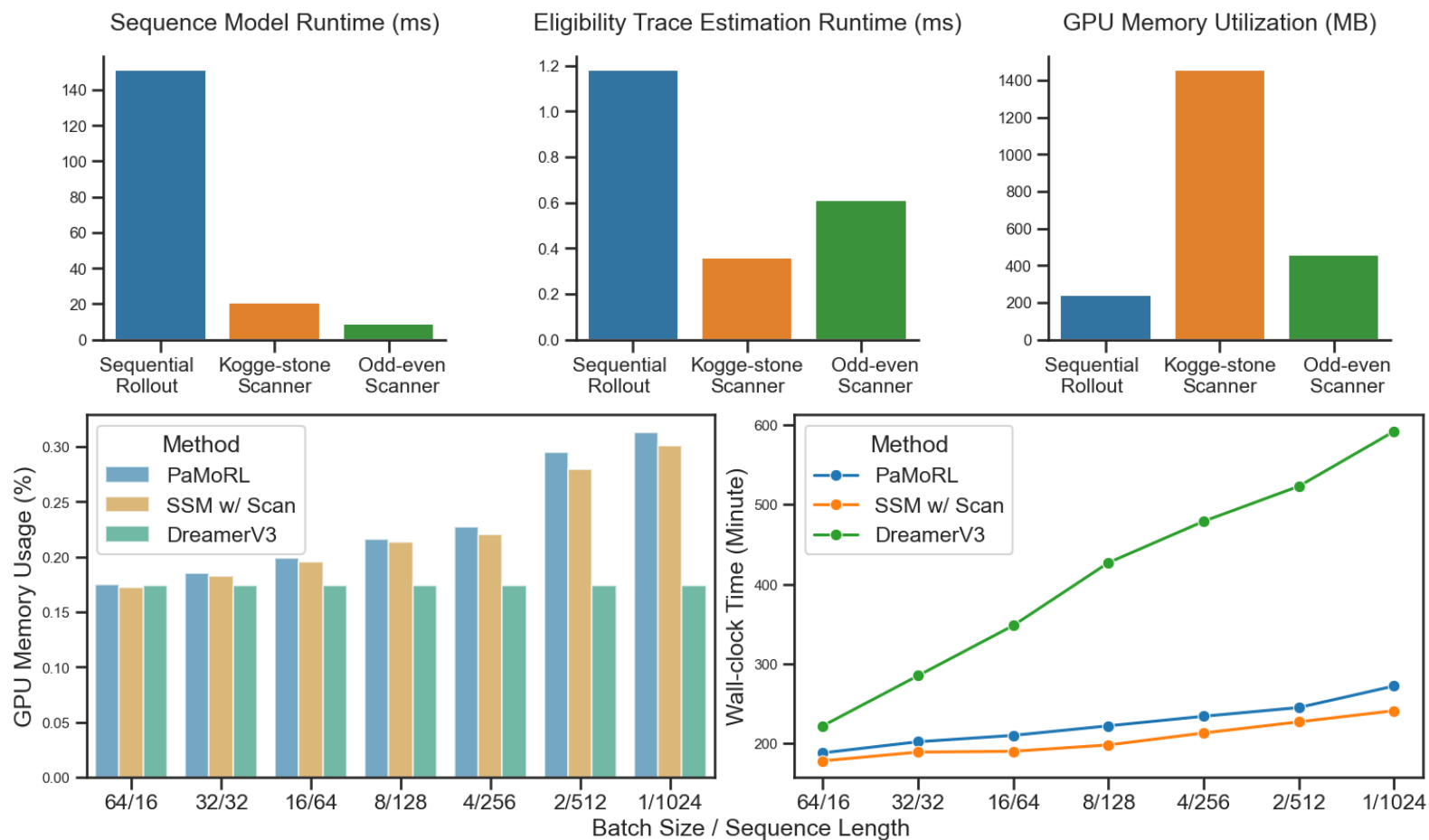
PaMoRL 同时实现了 **MBRL-level** 的样本效率，与
MFRL-level 的计算效率

高性能模型学习



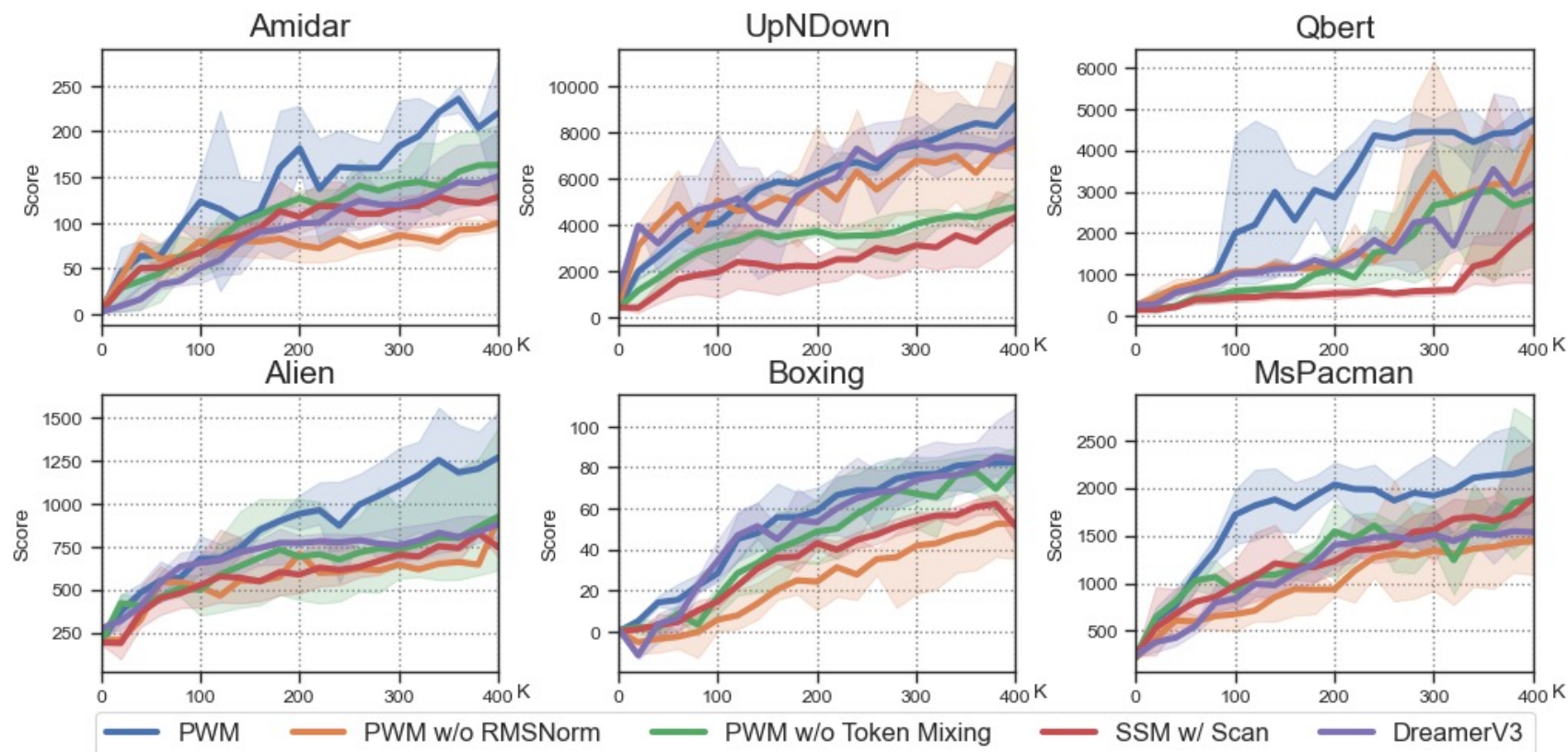
Task	Proprio Control			Vision Control			
	SAC	DreamerV3	PaMoRL (Ours)	CURL	DrQ-v2	DreamerV3	PaMoRL (Ours)
Cartpole Balance	997.6	839.6	994.7	963.3	965.5	956.4	610.3
Cartpole Balance Sparse	993.1	559	997.4	999.4	1000	813	996.5
Cartpole Swingup	861.6	527.7	773.6	765.4	756	374.8	281.9
Cup Catch	949.9	729.6	957.9	932.3	468	947.7	966.3
Finger Spin	900	765.8	835.8	850.2	459.4	633.2	765.3
Pendulum Swingup	158.9	830.4	707.1	144.1	233.3	619.3	26.6
Reacher Easy	744	693.4	761.6	467.9	722.1	441.4	950.2
Reacher Hard	646.5	768	645.9	112.7	202.9	120.4	103.7
Cartpole Swingup Sparse	256.6	172.7	542.3	8.8	81.2	392.4	263.6
Cheetah Run	680.9	400.8	313.2	405.1	418.4	587.3	935.6
Finger Turn Easy	630.8	560.5	617.1	371.5	286.8	366.6	886.2
Finger Turn Hard	414	474.2	389.7	236.3	268.4	258.5	500.1
Hopper Hop	0.1	9.7	387.5	84.5	26.3	76.3	426.9
Hopper Stand	3.8	296.1	151.5	627.7	290.2	652.5	189.7
Quadruped Run	139.7	289	246.7	170.9	339.4	168	344.8
Quadruped Walk	237.5	256.2	457.9	131.8	311.6	122.6	371.6
Mean	538.4	510.8	611.2	454.5	426.8	470.7	538.7
Median	638.7	543.4	631.5	388.3	325.5	416.9	463.5

高性能模型学习



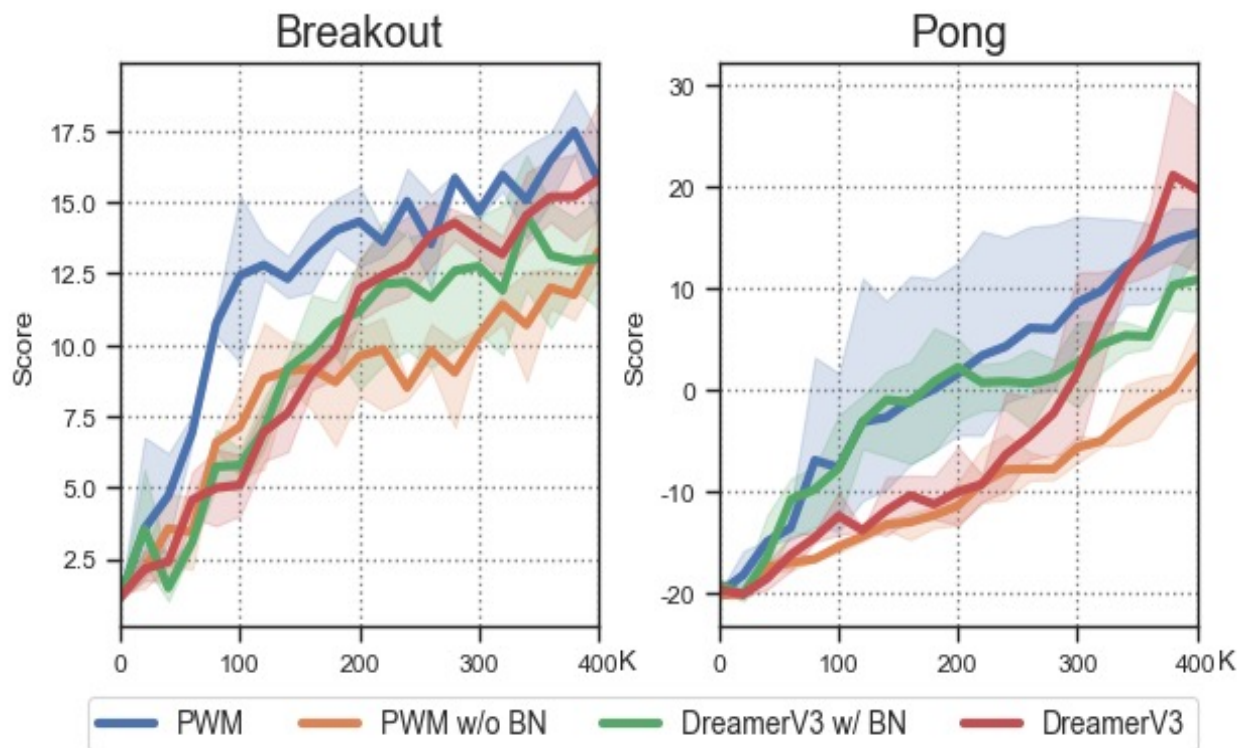
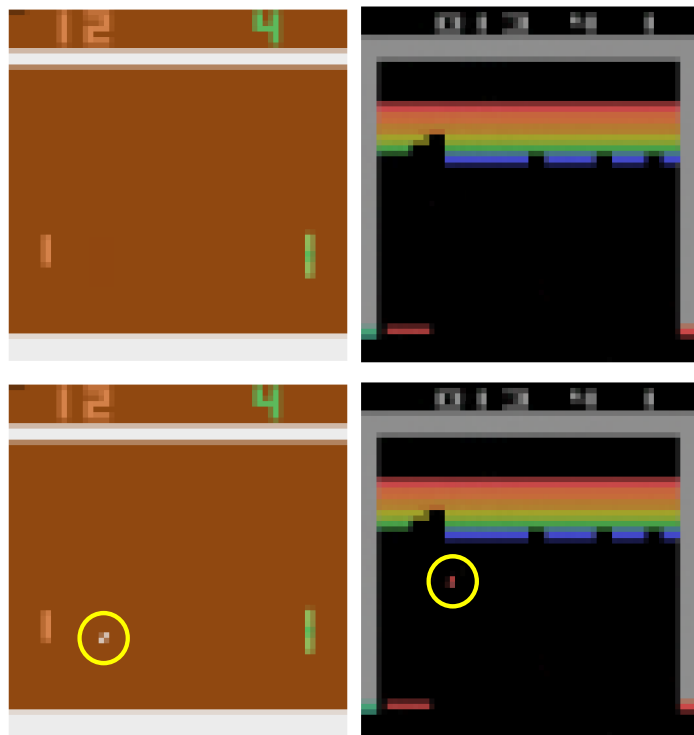
PaMoRL 带来了可接受的额外内存开销，但是能够换来显著的计算加速

高性能模型学习



RMSNorm, **Token Mixing**, 和 **Data-dependent Decay Rate** 对强化学习任务的训练稳定性很重要

高性能模型学习



BatchNorm Trick 对于提取图像输入中的细节信息具有非常重要的作用

主要内容

1. 基于经验缓存样本监督的模型接力方法
2. 基于历史样本建模的多智能博弈探索研究
3. 基于异/同策略样本分布密度的自适应TD(λ)计算方法研究
4. 行为树代码生成策略优化与辅助训练多智能体强化学习研究
5. PaMoRL基于模型的强化学习方法
6. 具身智能

具身智能

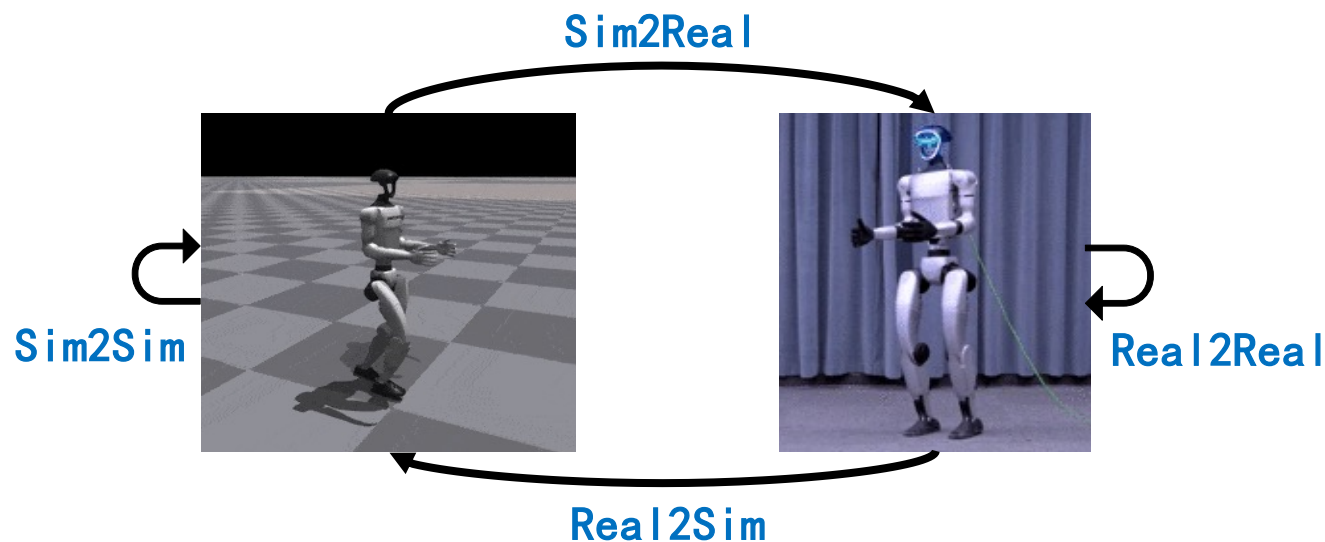
具身智能面临的核心问题？

- 物理实体采样效率低

（强化学习的样本量一般会达到上千万，在现实环境中很难采集如此数量级的样本）

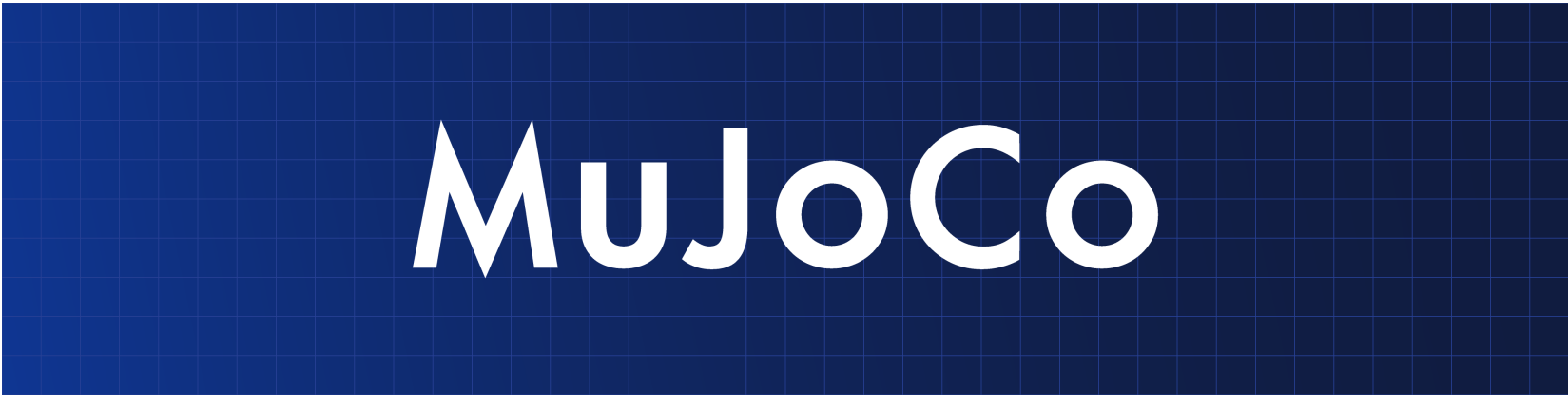
- 安全问题

（强化学习需要大量试错，该特性可能会损伤机器人自身，也会对周围环境甚至生物造成伤害）



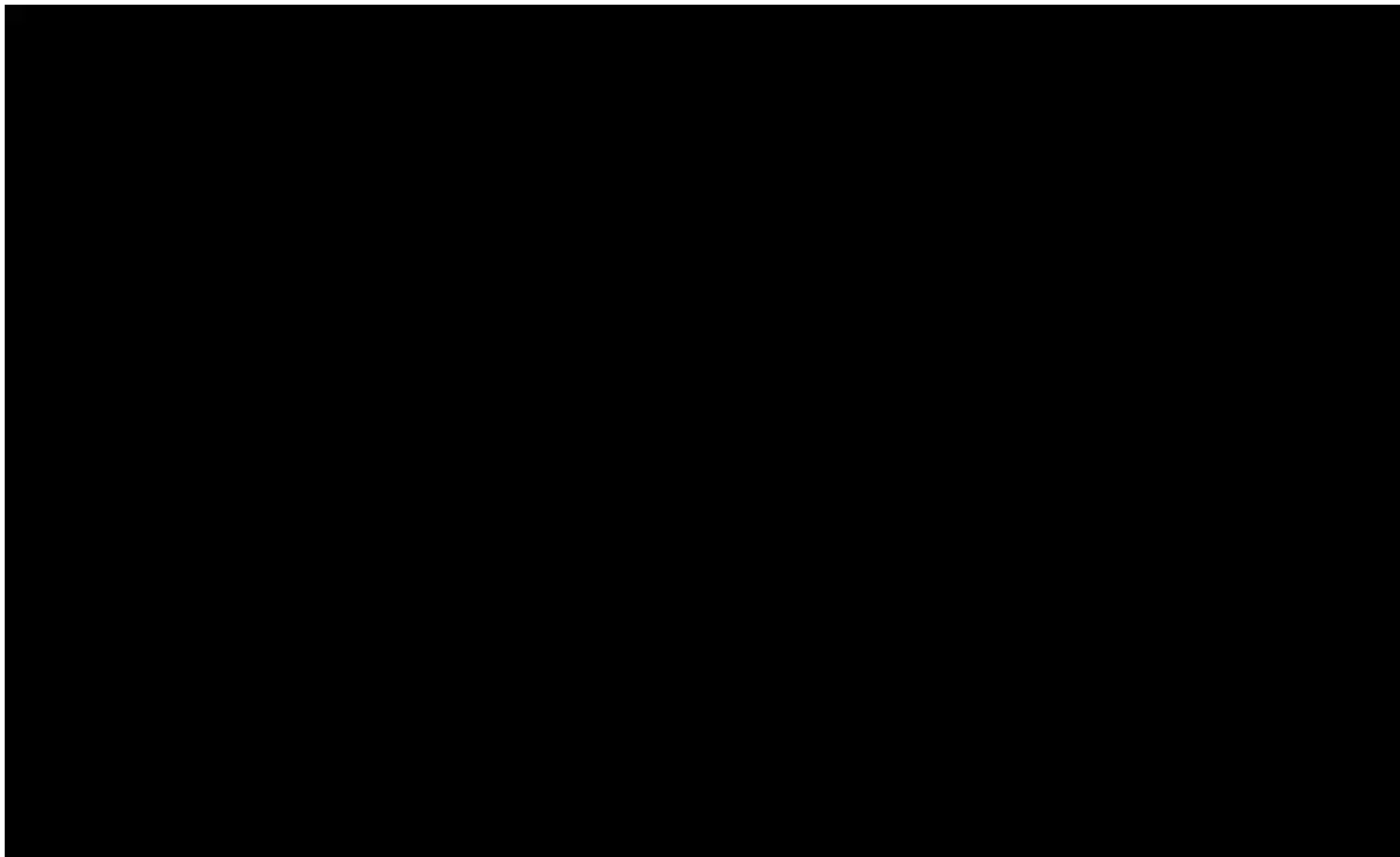
具身智能仿真平台

- MuJoCo 是"带接触的多关节动力学"（Multi-Joint dynamics with Contact）的缩写
- 作为一款通用物理引擎，它致力于推动机器人学、生物力学、图形动画、机器学习等领域的研究与开发，这些领域都需要对关节结构与环境的交互进行快速精确的模拟

The MuJoCo logo is displayed in white text on a dark blue rectangular background. The background features a subtle grid pattern. The text "MuJoCo" is centered and rendered in a clean, sans-serif font.

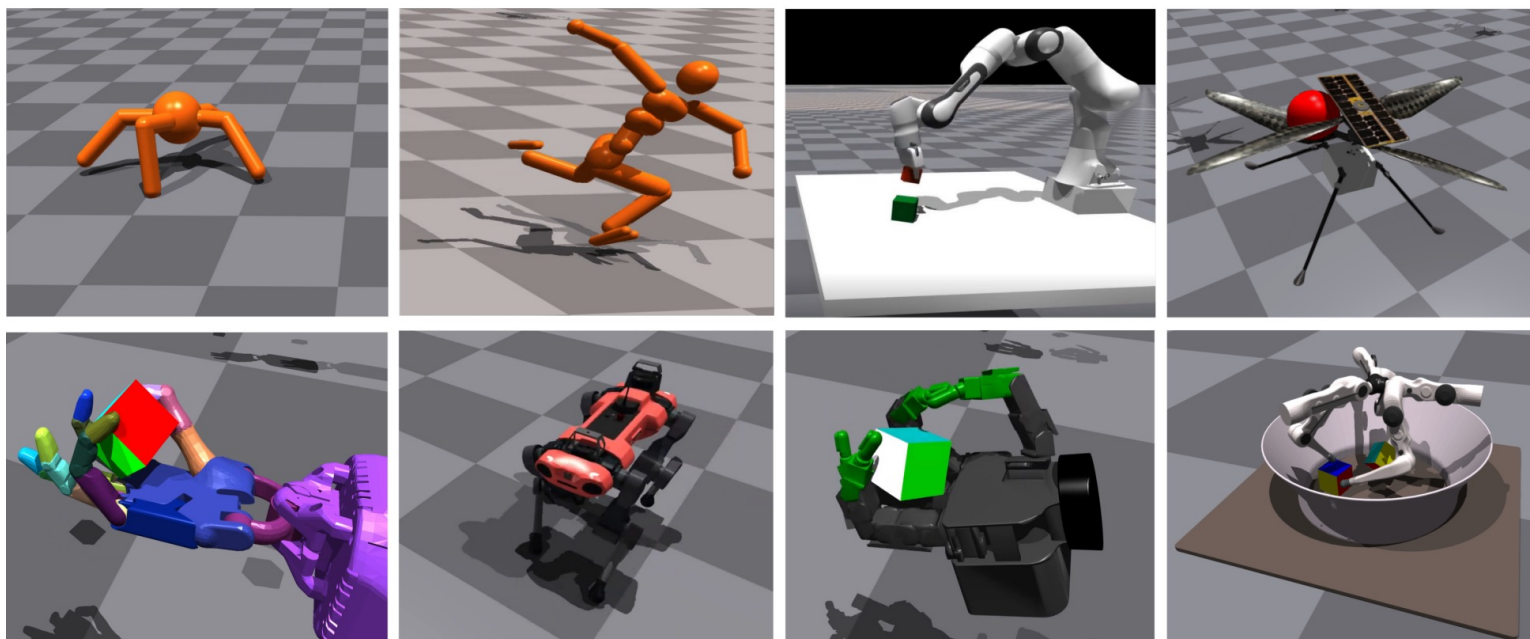
MuJoCo

具身智能仿真平台



具身智能仿真平台

- Isaac Gym 是 NVIDIA 为强化学习打造的物理模拟环境
- Isaac Gym 支持基于 GPU 的大规模并行物理仿真，支持导入多种机器人描述文件，并自动进行凸分解以用于物理模拟，其支持多种环境传感器、多种物理参数的域随机化

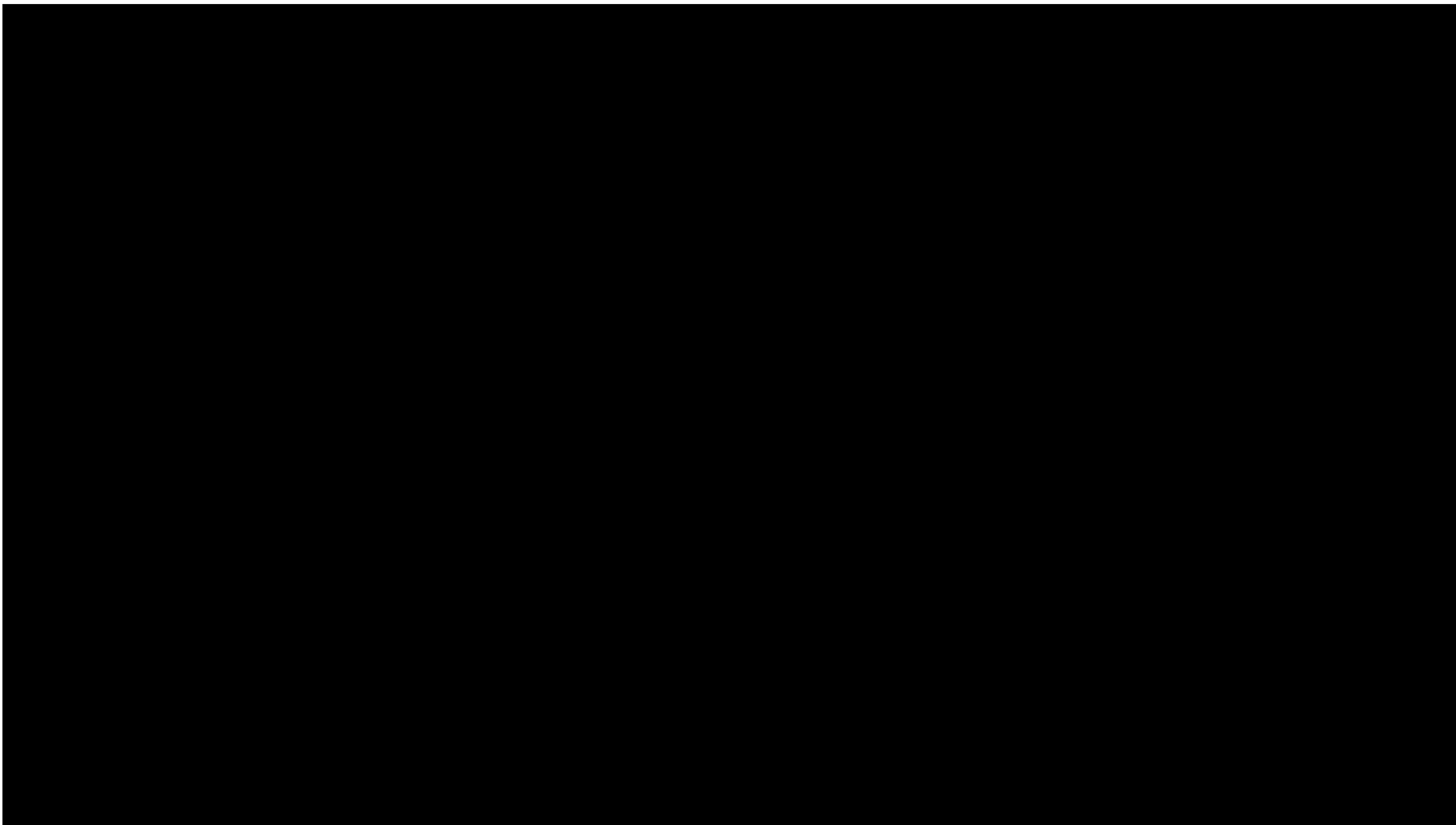


具身智能仿真平台

- Isaac Lab 是一个统一且模块化的机器人学习框架，旨在简化机器人研究中的常见工作流程
- 该框架基于 NVIDIA Isaac Sim 构建，能够利用最新仿真技术实现逼真场景渲染及高效快速仿真



具身智能仿真平台



具身智能仿真平台

- SAPIEN 是一个高度拟真、物理特性丰富的仿真环境，内含大规模铰接物体数据集
- 其主要面向精细操作领域，支持多种需要精细部件级理解的各类机器人视觉与交互任务



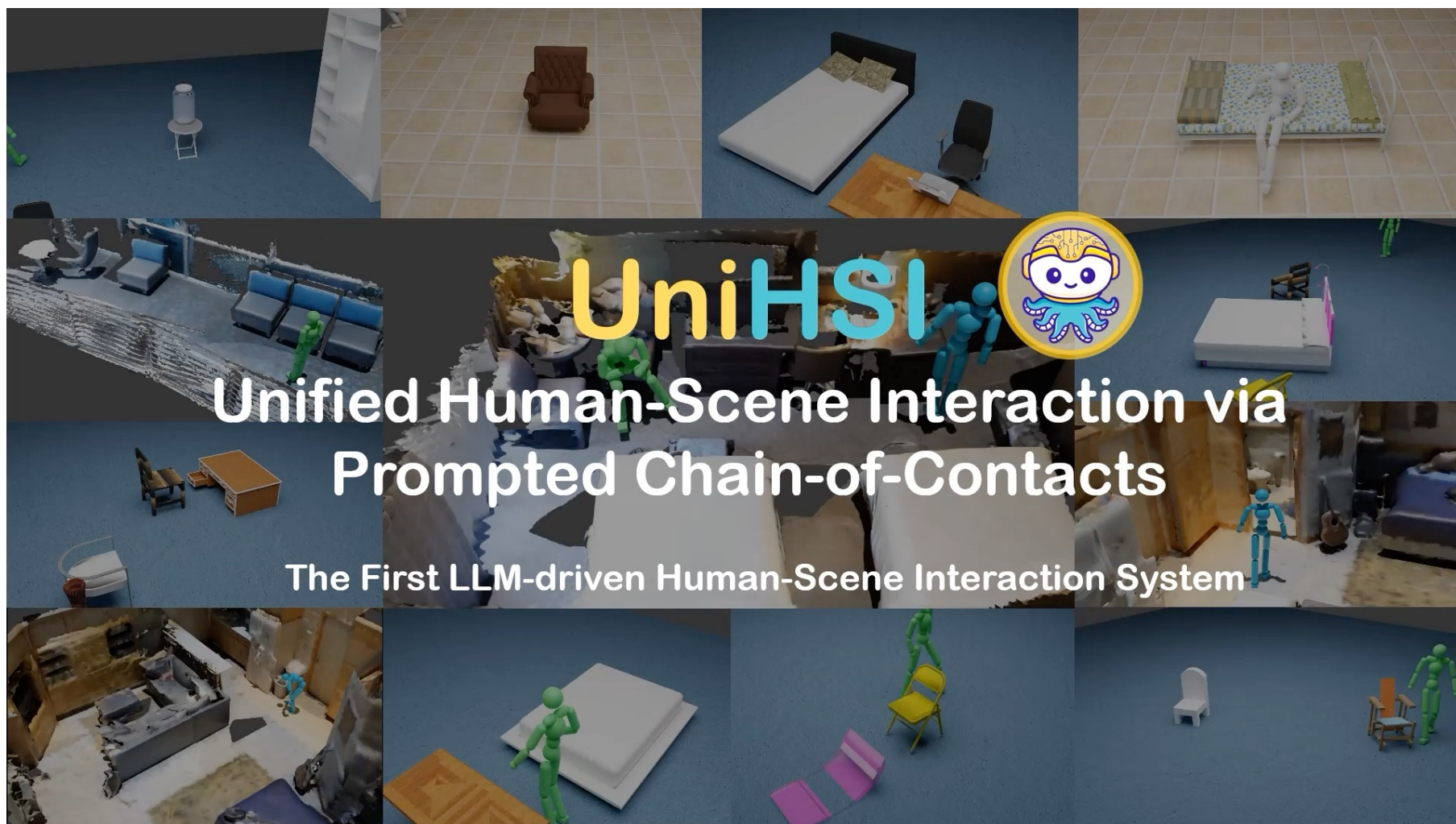
机械臂 & 灵巧手



机械臂 & 灵巧手

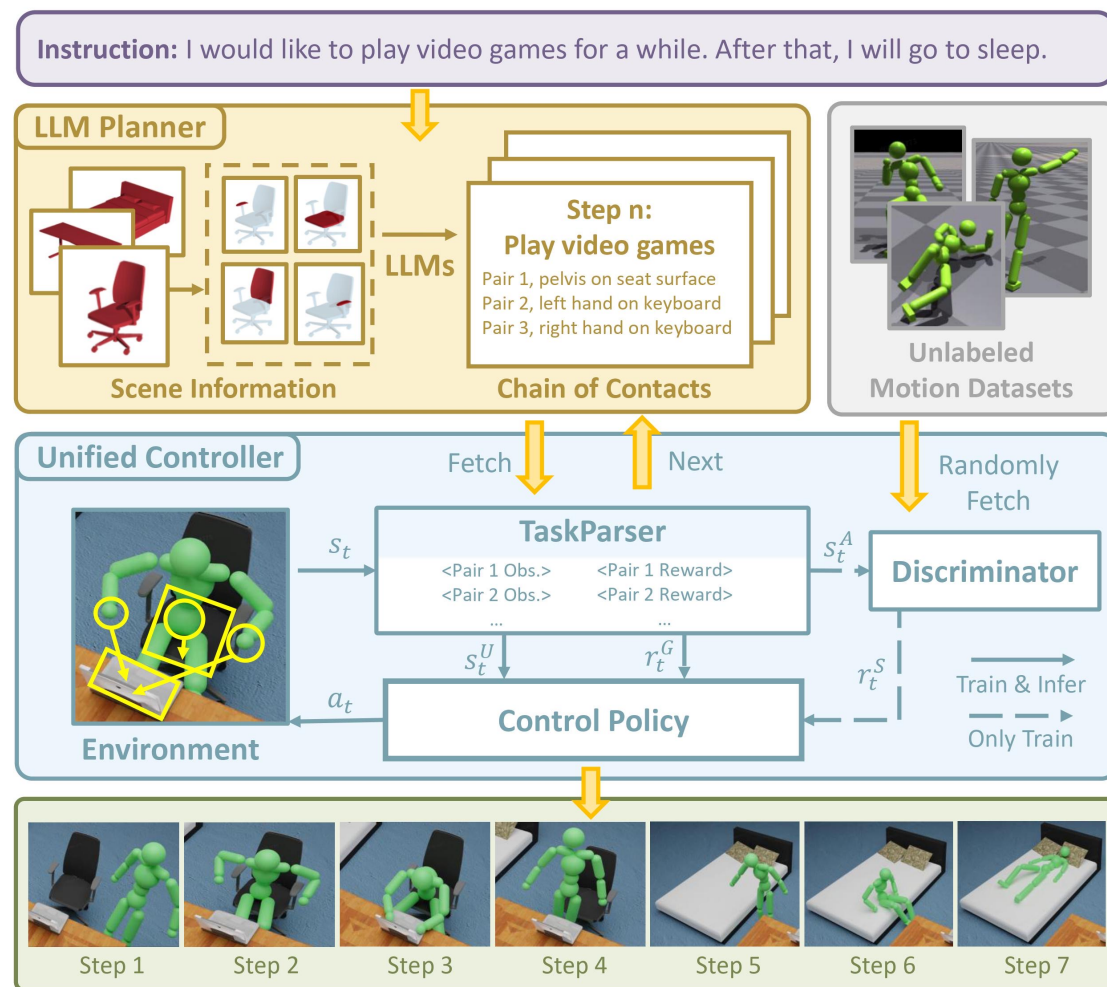


人型机器人场景交互



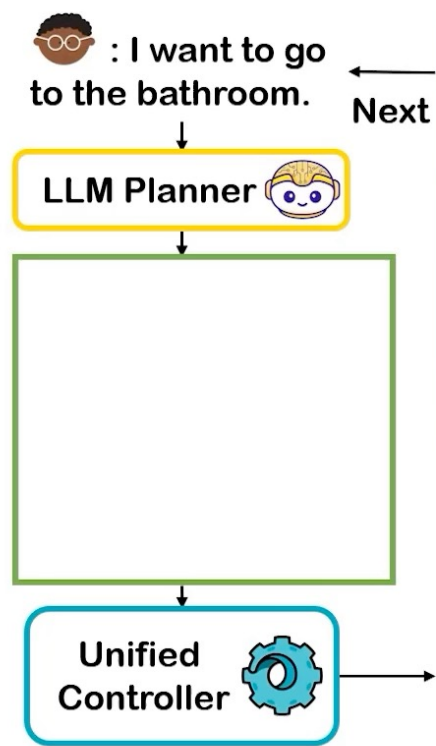
人型机器人场景交互

- UniHSI 是首个由大语言模型驱动的统一交互框架
- UniHSI 将场景交互建模为接触链，用人体与物体部分的接触来描述交互行为
- UniHSI 包括一个上游大语言模型规划器，将语言输入转化为交互接触链形式的任务计划，以及一个位于下游的人型机器人强化学习运动控制策略



人型机器人场景交互

- UniHSI 可以进行**长程交互**、多物品交互以及同一物品的多样化交互，支持基于大语言模型的“多智能体”交互



人型机器人场景交互

- UniHSI 可以进行长程交互、**多物品交互**以及同一物品的**多样化交互**，支持基于大语言模型的“多智能体”交互



👤 :



👤 :



人型机器人场景交互

- UniHSI 可以进行长程交互、多物品交互以及同一物品的多样化交互，支持基于大语言模型的 **“多智能体”交互**



:



:



谢谢

